



第3章 数据库SQL语言

主讲教师：陈安龙

电子科技大学 信息与软件学院

本章主要内容

- 1、了解SQL语言的特点
- 2、理解SQL语言的功能
- 3、掌握SQL语言创建数据库、表、视图、索引
- 4、掌握SQL语言的查询语句

一、SQL简介

- **SQL (Structured Query Language)**：是一种结构化查询语言，用于管理关系型数据库系统的标准化语言；提供了对数据库进行访问与操作的数据语言。
- SQL（结构化查询语言）语言的发展**历程是数据库技术演进的重要组成部分**。
SQL 作为一种用于管理和操作关系数据库的语言，自20世纪70年代以来经历了多个阶段的演变和标准化。

SQL标准的发展历程

1. 1970年代：SQL的起源

- ① 1970年：埃德加·F·科德（Edgar F. Codd）提出了关系模型，奠定了关系数据库的理论基础。
- ② 1974年：IBM 的研究团队开发了 **SEQUEL**（**Structured English Query Language**），这是 **SQL** 的前身，旨在操作关系数据库。
- ③ 1979年：IBM 发布了第一个商业关系数据库管理系统（RDBMS）**System R**，采用了 **SEQUEL** 作为查询语言。

2. 1980年代：SQL的标准化

- ① 1986年：美国国家标准协会（ANSI）发布了第一个 SQL 标准，称为 SQL-86。这个标准定义了 SQL 的基本语法和功能，包括数据查询、数据定义和数据操纵等。
- ② 1989年：ANSI 发布了 SQL-89 标准，对 SQL-86 进行了修订，增加了一些新特性，如外连接和更严格的语法规则。

3. 1990年代：SQL的扩展与发展

- ① 1992年：ANSI 发布了 SQL-92 标准，这是一个重要的版本，增加了许多新特性，如子查询、集合操作、触发器和视图等。SQL-92 标准使 SQL 更加功能丰富，广泛被数据库厂商采用。
- ② 1999年：发布了 SQL:1999 标准，增加了对对象关系模型的支持，引入了对象导向的特性，如用户定义类型（UDT）、表继承和多态等。

4. 2000年代：SQL的进一步发展

- ① 2003年：发布了 SQL:2003 标准，增加了对 XML 数据的支持，允许在 SQL 中处理 XML 数据类型。
- ② 2006年：发布了 SQL:2006 标准，进一步扩展了对 XML 的支持，增加了 XML 查询功能。

5. 2010年代：SQL的现代化

- ① 2011年：发布了 SQL:2011 标准，增加了对时间序列数据的支持，定义了新的时间数据类型和相关操作。
- ② 2016年：发布了 SQL:2016 标准，增加了对 JSON 数据的支持，允许在 SQL 中处理 JSON 数据类型，增强了对大数据和非关系型数据的支持。

SQL语言的使用与功能

- SQL语句可嵌入到编程语言(如JAVA、C++等)中执行或在DBMS工具中执行。
- SQL不仅仅是一个查询工具，还用于控制DBMS提供给用户的所有功能，包括：
 - ① 数据定义(Data definition)
 - ② 数据检索(Data retrieval)
 - ③ 数据操纵(Data manipulation)
 - ④ 存取控制(Access control)
 - ⑤ 数据共享(Data sharing)
 - ⑥ 数据完整性(Data integrity)

SQL应用情况

- 绝大多数主流的关系型DBMS采用了SQL语言标准：Oracle、Sybase、DB2、Microsoft SQL Server、postgreSQL、MySql
- 其中某些数据库管理系统对SQL语句进行了再开发和扩展：
 - ① Sybase、Microsoft SQL Server将操作语言扩展为T-SQL;
 - ② Oracle将操作语言扩展为PL/SQL;
 - ③ Postgre SQL扩展为PL/PGSQL;
 - ④ mySQL是采用的标准SQL

SQL语言的类型

1. 数据定义语言（Data Definition Language, DDL）是SQL语言中用于创建或删除表。也可以定义索引（键），规定表之间的约束。

- ① **CREATE DATABASE** - 创建新数据库
- ② **ALTER DATABASE** - 修改数据库
- ③ **CREATE TABLE** - 创建新表
- ④ **ALTER TABLE** - 变更（改变）数据库表
- ⑤ **DROP TABLE** - 删除表
- ⑥ **CREATE INDEX** - 创建索引（搜索键）
- ⑦ **DROP INDEX** - 删除索引

2. 数据操纵语言（Data Manipulation Language, DML）是SQL语言中用于查询和增删修改表数据的语句

- ① **SELECT** - 从数据库表中获取数据
- ② **UPDATE** - 更新数据库表中的数据
- ③ **DELETE** - 从数据库表中删除数据
- ④ **INSERT** - 向数据库表中插入数据

3. **数据控制语言（Data Control Language, DCL）**是SQL语言中用于对数据访问权进行控制的语句。

① **GRANT** – 授予用户对数据对象的权限

② **DENY** – 用于拒绝特定用户或角色对数据库对象的访问权限；即使该用户或角色在其他地方被授予了相应的权限，**DENY** 仍然会覆盖这些权限。

③ **REVOKE** – 撤消用户对数据对象的权限

4. 事务处理语言（Transaction Process Language, TPL）是SQL语言中用于数据库内部事务处理的语句。

① BEGIN TRANSACTION – 开始事务

② COMMIT – 提交事务

③ ROLLBACK – 回滚事务

4. 游标控制语言（Cursor Control Language, CCL）是SQL语言中用于数据库游标操作的语句。

① **DECLARE CURSOR** – 定义游标

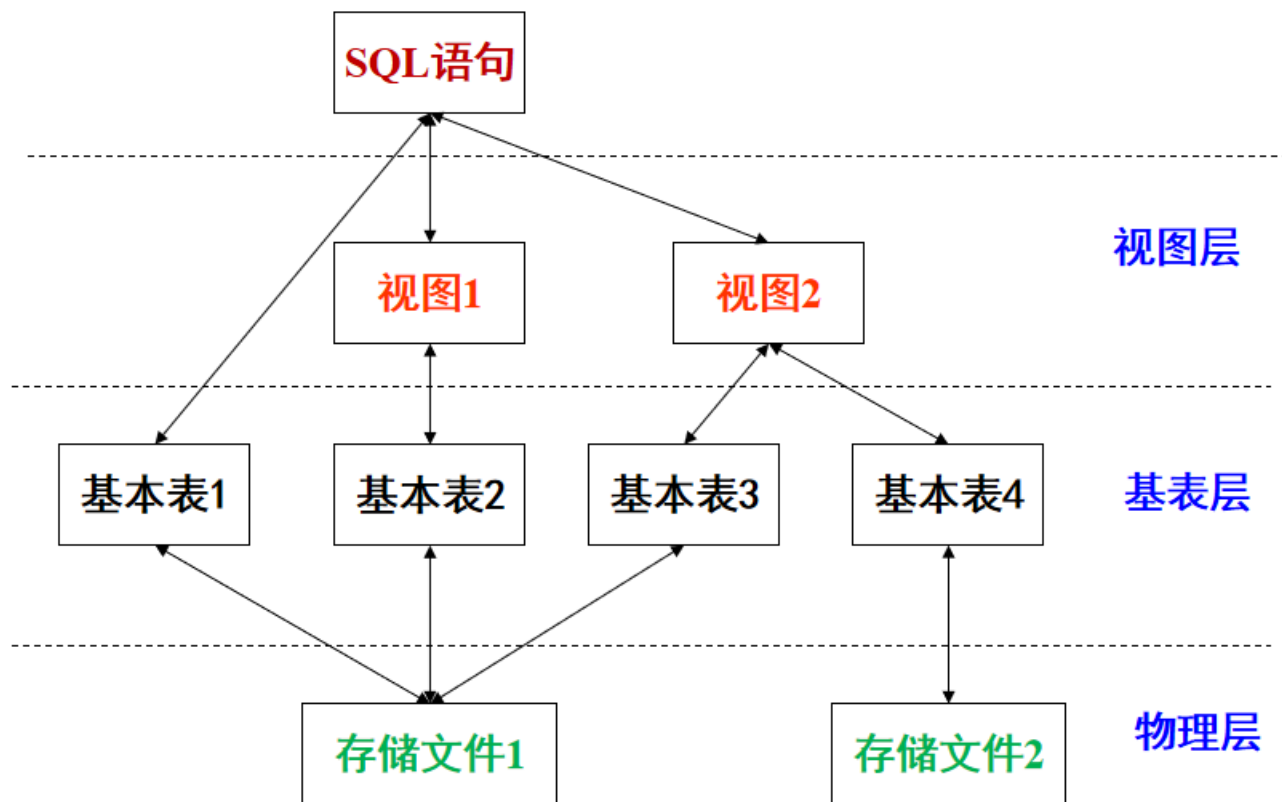
② **FETCH INTO** – 提交游标数据

③ **CLOSE CURSOR**– 关闭游标

SQL语言特点

- 多种功能集成：DDL、DML、DCL的等功能集成
- 高度非过程化：
 - ① 只需提出“做什么”，而无需指明“怎么做”；
 - ② 无需了解存取路径，存取路径的选择以及操作过程由系统自动完成。
- 面向集合的操作方式：操作对象和结果均为集合
- 语言简洁，易学易用，核心功能只需9个动词
 - ① DDL：CREATE、DROP、ALTER
 - ② DML：SELECT、INSERT、UPDATE、DELETE
 - ③ DCL：GRANT、REVOKE

SQL语言在数据库中的工作层次

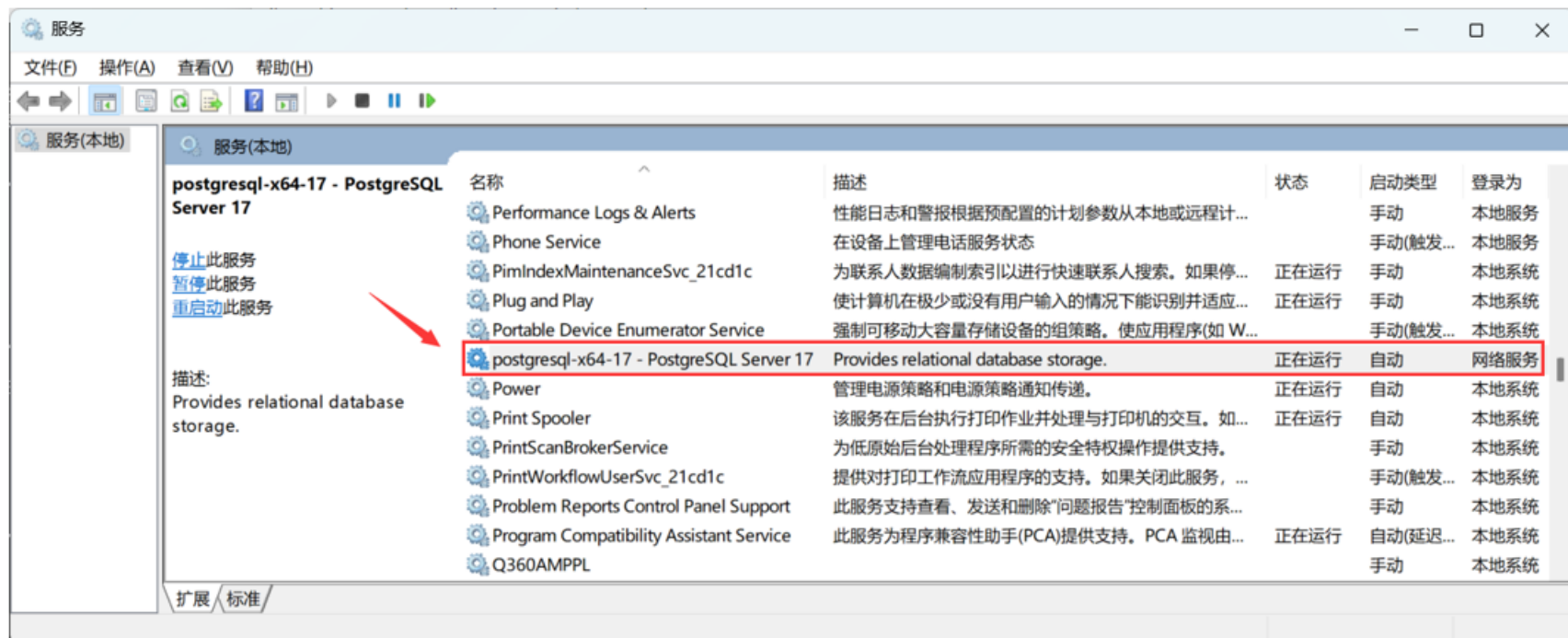


二、PostgreSQL 连接数据库

- 1、命令行工具 psql;
- 2、图形化工具（如 pgAdmin）；
- 3、在高级编程语言中使用数据库驱动连接数据库（**自学**）。

连接数据库前的准备：确保 PostgreSQL 服务正在运行

- ① 按 Win + R 打开运行窗口，输入 **services.msc** 并按回车。
- ② 在服务列表中找到 PostgreSQL 服务（通常以版本号命名，如 PostgreSQL 17）。
- ③ 确保服务状态为“正在运行”。如果没有运行，右键单击该服务并选择“启动”。



1、命令行工具 psql

使用以下命令连接到 PostgreSQL 数据库：

`psql -U username -h hostname -p port -d database_name`

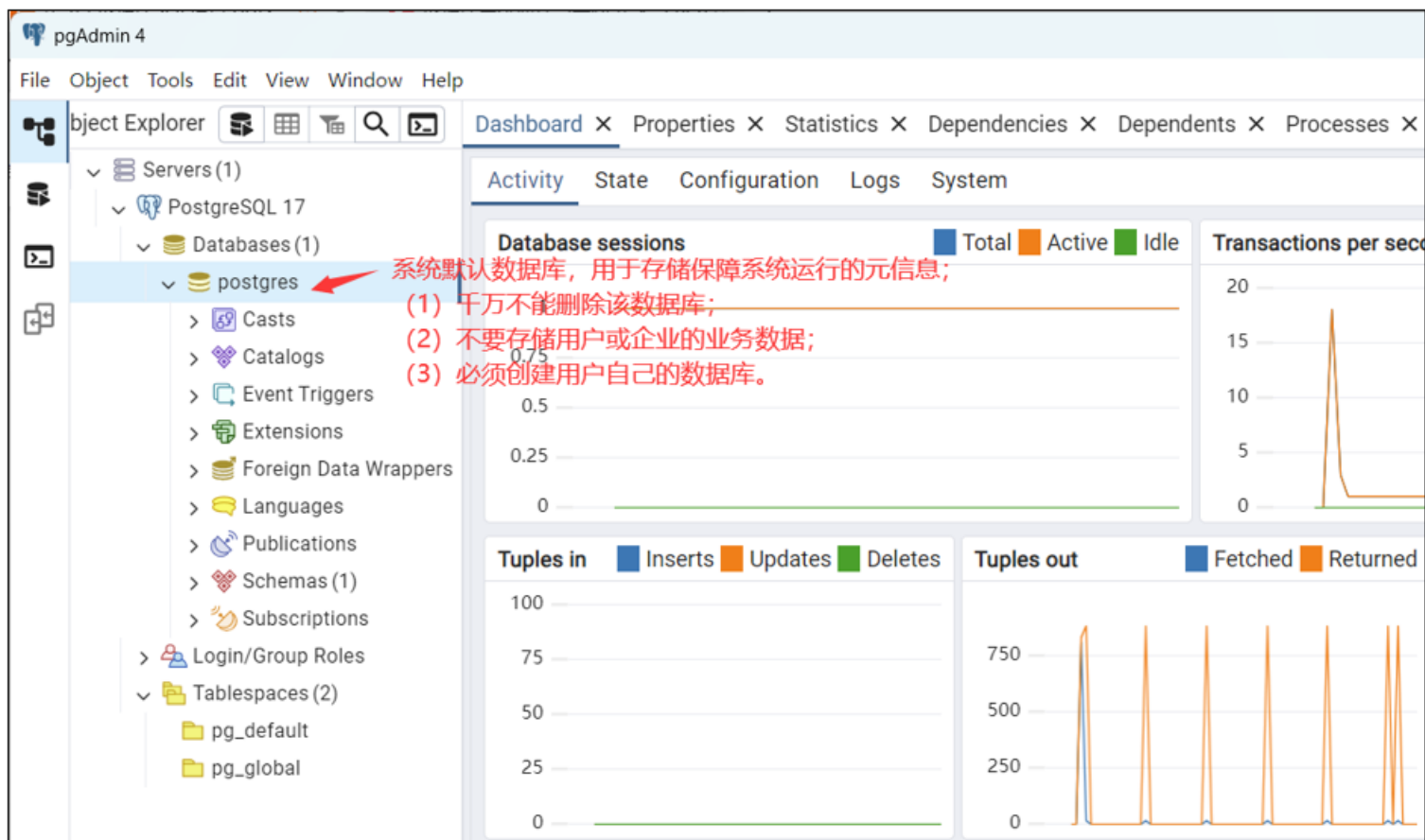
- ① **username**：您的 PostgreSQL 用户名。
- ② **hostname**：数据库服务器的主机名（通常为 localhost，表示本地）。
- ③ **port**：PostgreSQL 的端口号（默认是 5432）。
- ④ **database_name**：要连接的数据库名称。

```
命令提示符 - psql -U postgres
Microsoft Windows [版本 10.0.26100.3194]
(c) Microsoft Corporation. 保留所有权利。

C:\Users\chena>psql -U postgres -h localhost -p 5432 -d postgres
用户 postgres 的口令:
psql (17.3)
输入 "help" 来获取帮助信息。

postgres=#
```


2、启动 pgAdmin 应用程序连接数据库



三、创建数据库

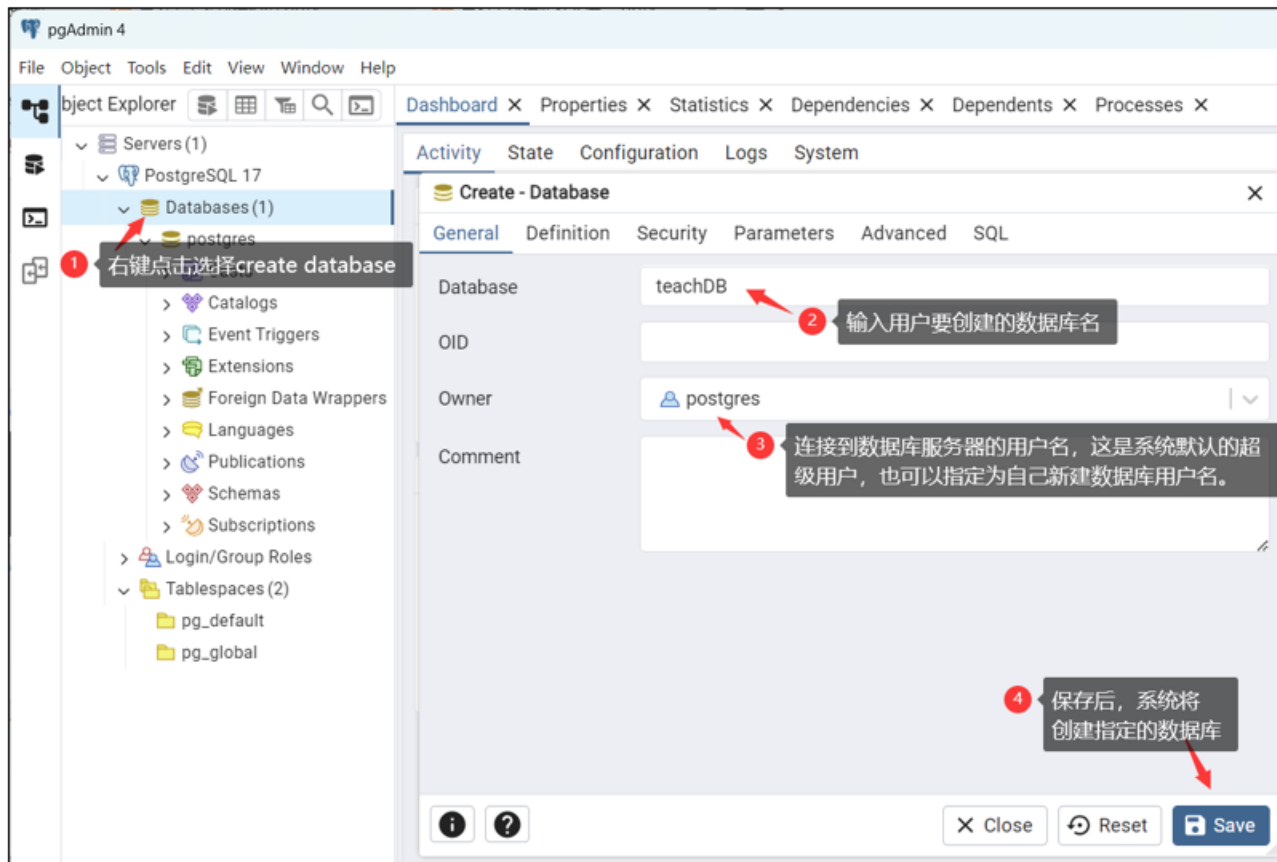
PostgreSQL 创建数据库可以用以下方式：

- 1、使用 **pgAdmin** 图形向导工具；
- 2、使用 **CREATE DATABASE** 语句来创建；

1、使用 pgAdmin 图形向导工具

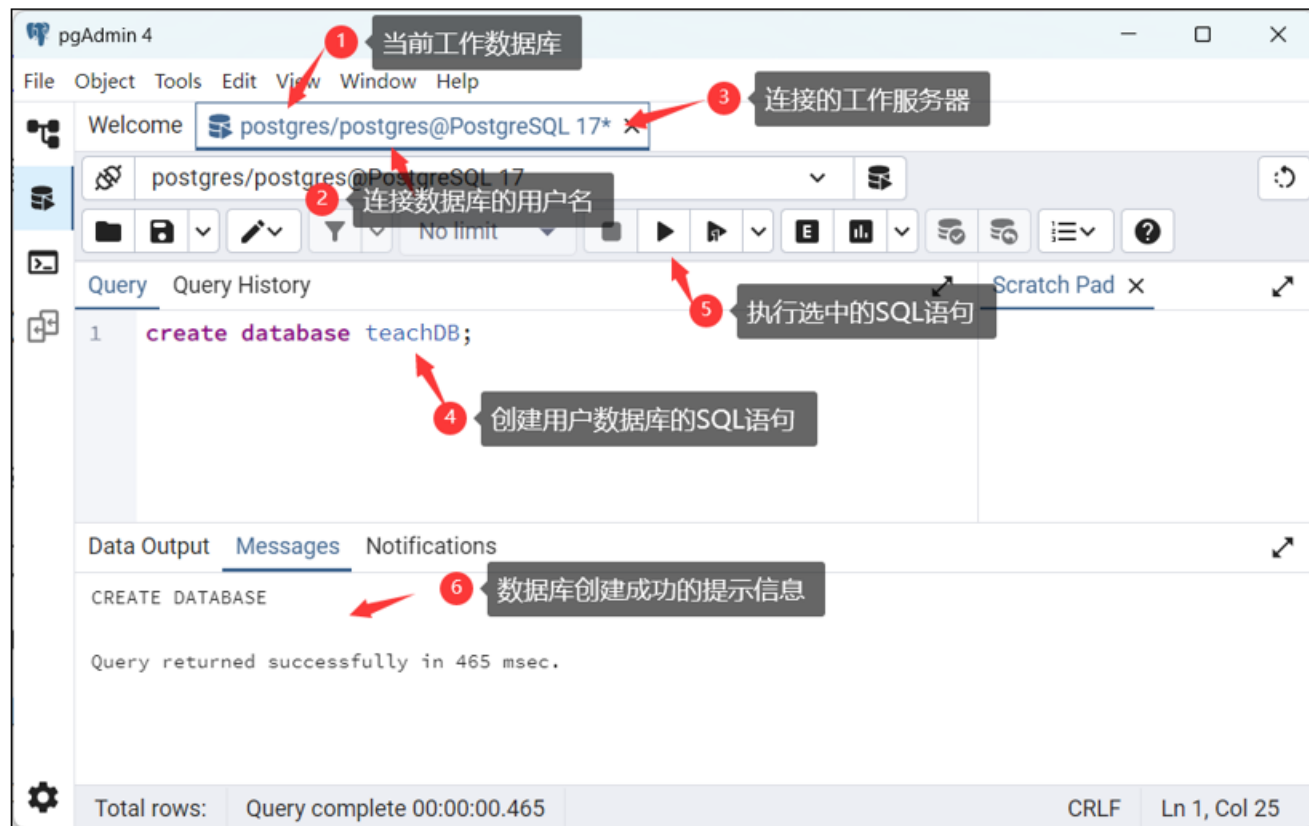
① 选择服务器：在左侧树形结构中，展开您刚刚连接的服务器，找到 **Databases**（数据库）节点。

② 右键单击**Databases**：在**Databases**节点上右键单击，选择 **Create（创建） > Database...（数据库...）**。



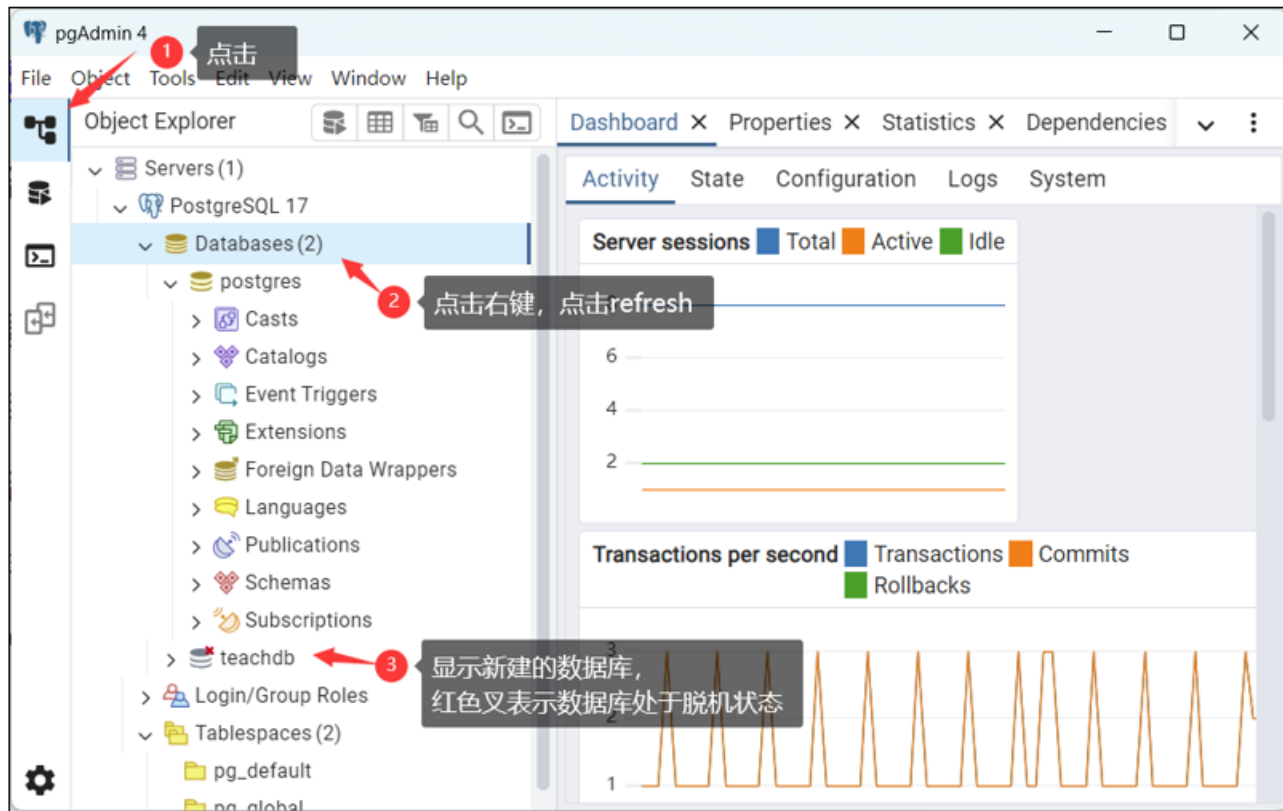
2、使用 CREATE DATABASE 语句来创建

- ① 选择数据库：在左侧树形结构中，选择您要连接的数据库（通常是 postgres 数据库，或者您可以选择其他已存在的数据库）。
- ② 打开查询工具：在 pgAdmin 的顶部菜单中，点击 Tools，然后选择 Query Tool。这将打开一个新的查询窗口。



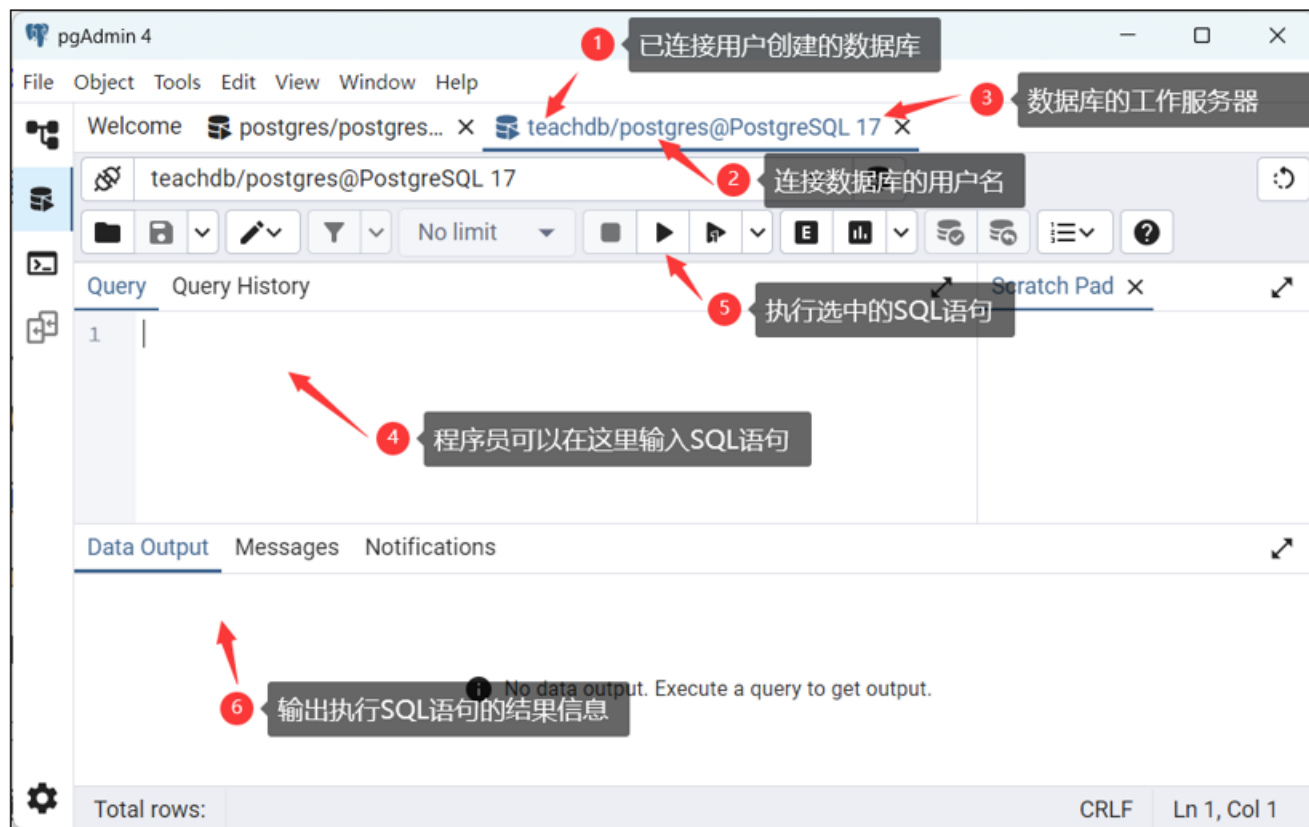
成功创建数据库后的状态

- ① 选择服务器：在左侧树形结构中，展开您刚刚连接的服务器，找到 **Databases**（数据库）节点。
- ② 右键单击**Databases**：在**Databases**节点上右键单击，选择 **refresh**（刷新）。



3、在用户创建的数据库上执行SQL操作

- ① 选择数据库：在左侧树形结构中，选择要连接的用户已创建的数据库（如：teachDB数据库）。
- ② 打开查询工具：在pgAdmin 的顶部菜单中，点击Tools，然后选择Query Tool。这将打开一个新的查询窗口。



四、数据库修改SQL语句

在 PostgreSQL 中，可以使用 SQL 语句对数据库进行多种修改操作，包括修改数据库的名称、所有者、表空间等。以下是一些常见的 SQL 语句及其用法，帮助您了解如何在 PostgreSQL 中修改数据库。

语句基本格式： `ALTER DATABASE` <数据库名> <修改内容>;

修改数据库的名称： `ALTER DATABASE` old_database_name `RENAME TO` new_database_name;

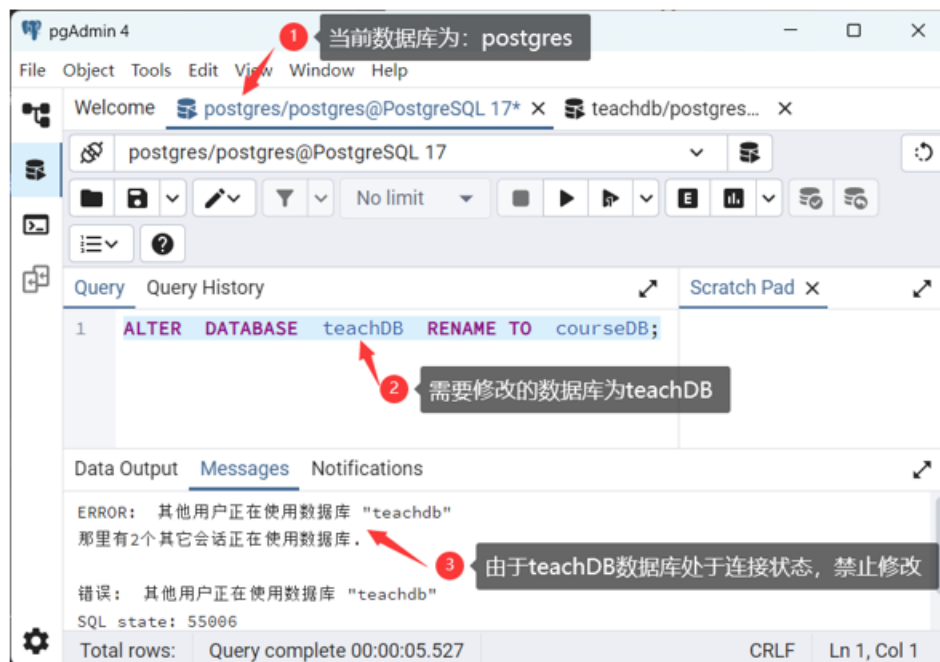
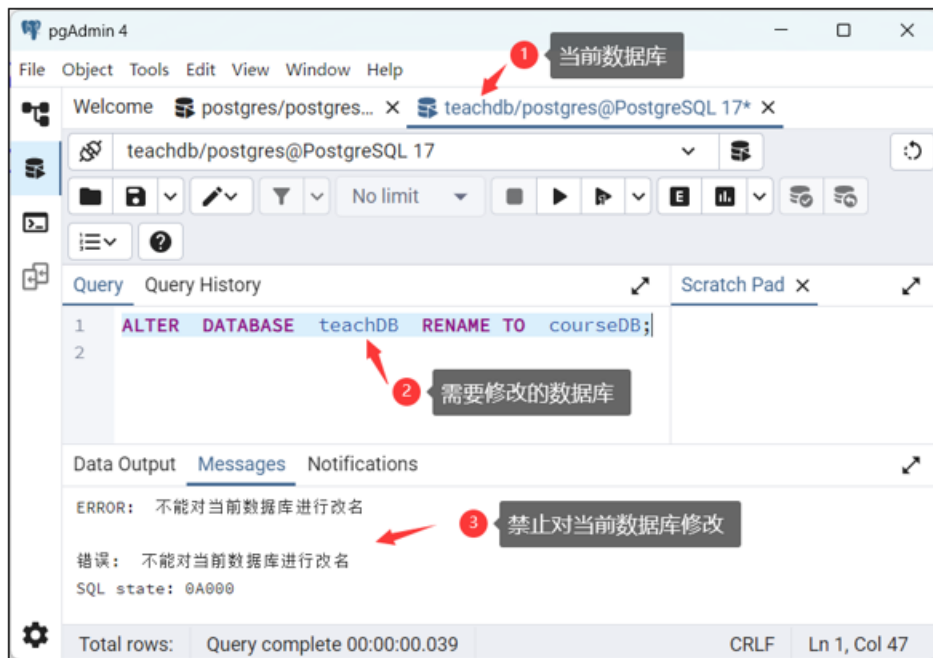
修改数据库的所有者： `ALTER DATABASE` database_name `OWNER TO` new_owner;

修改数据库的连接限制： `ALTER DATABASE` database_name `CONNECTION LIMIT` new_limit;

修改数据库的表空间： `ALTER DATABASE` database_name `SET TABLESPACE` new_tablespace;

1、修改数据库的名称

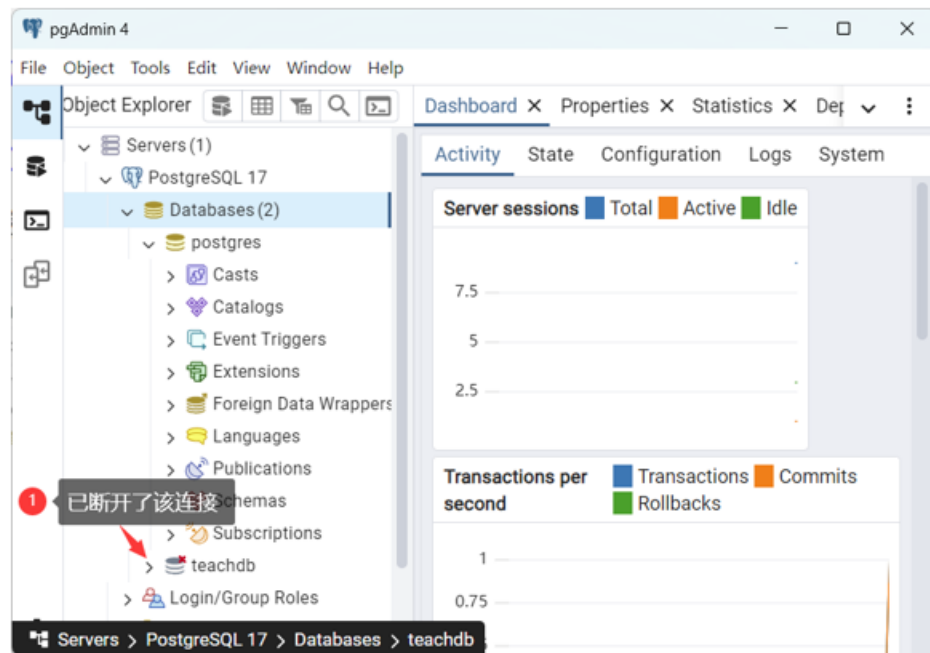
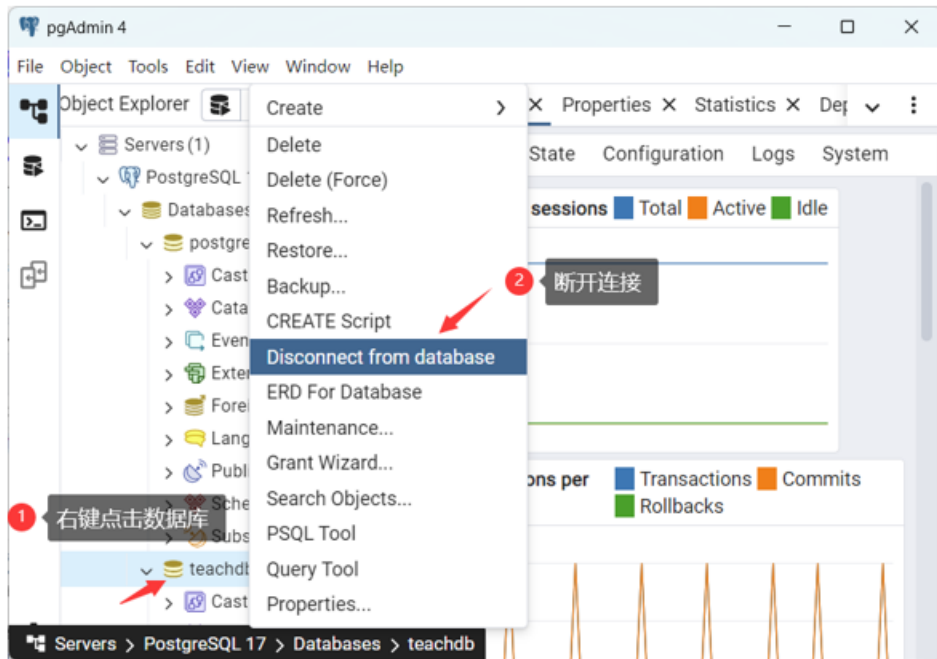
① ALTER DATABASE teachDB RENAME TO courseDB;



1、修改数据库的名称

① **ALTER DATABASE teachDB RENAME TO courseDB;**

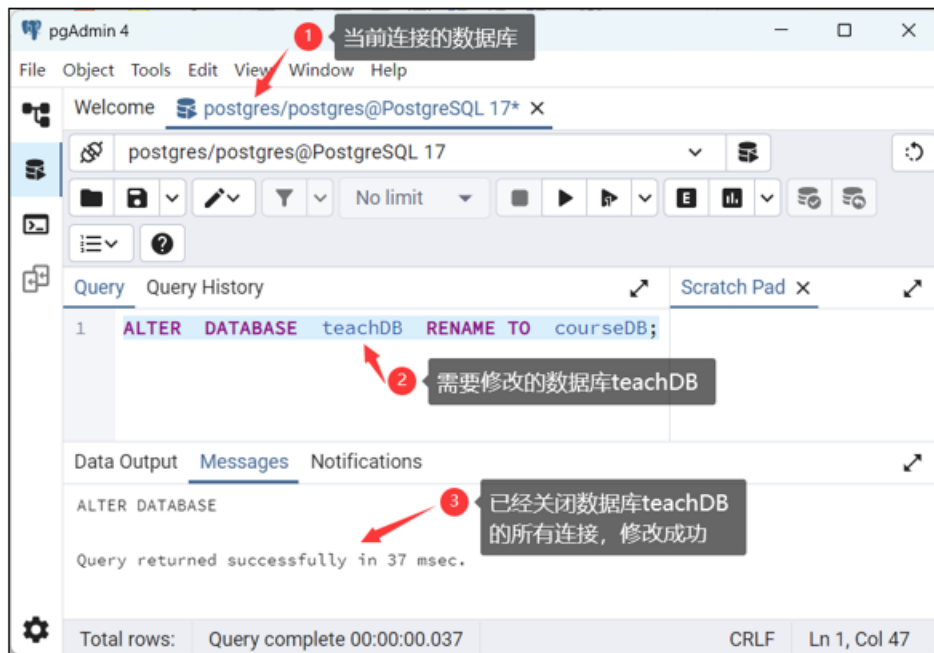
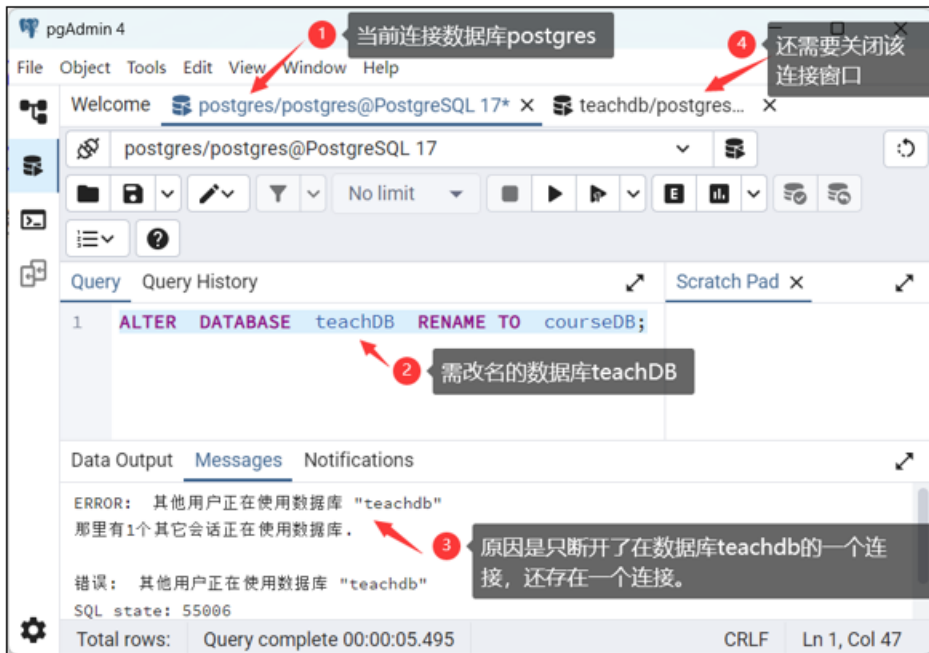
② 需要断开数据库的所有连接；



1、修改数据库的名称

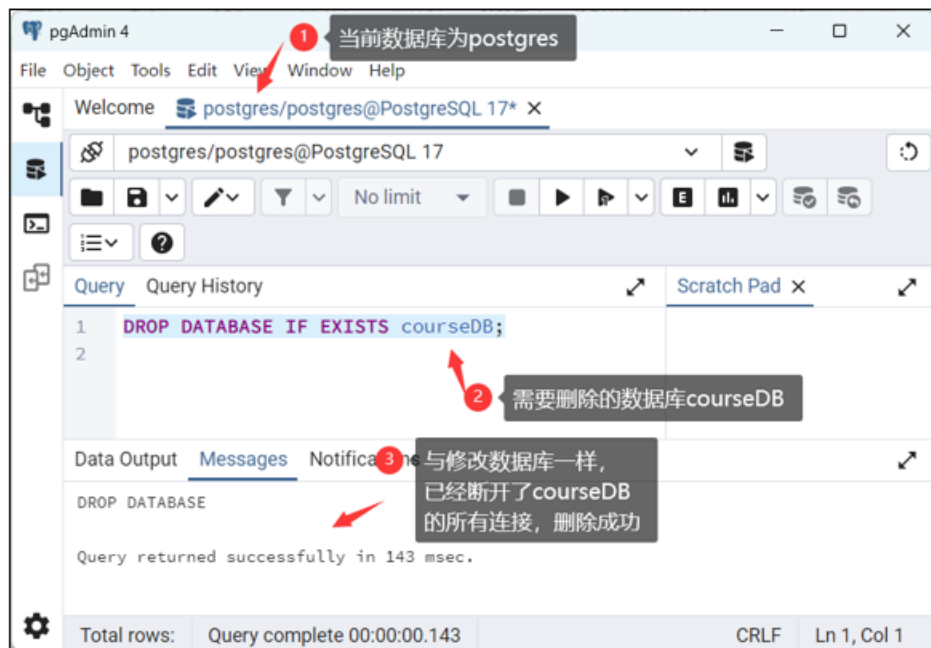
① **ALTER DATABASE teachDB RENAME TO courseDB;**

② 需要断开数据库的所有连接;



2、删除数据库

- ① **DROP DATABASE IF EXISTS** database_name;
- ② 使用 **IF EXISTS** 选项，确保当数据库存在时执行删除。
- ③ 与修改数据库一样，需要断开与数据库的所有连接；



五、创建数据库表

CREATE TABLE 用于创建新表，用于定义表的结构，包括列、数据类型、约束等。

```
CREATE TABLE table_name (  
    column1 data_type [constraints],  
    column2 data_type [constraints],  
    ...  
    [table_constraints]  
);
```

- ① **table_name**: 要创建的表的名称。
- ② **column1, column2, ...**: 表中的列名。
- ③ **data_type**: 列的数据类型，例如: **INTEGER**、**VARCHAR(n)**、**DATE** 等。
- ④ **constraints**: 可选的约束条件，例如: **NOT NULL**、**UNIQUE**、**PRIMARY KEY** 等。
- ⑤ **table_constraints**: 可选的表级约束，例如: **PRIMARY KEY**、**FOREIGN KEY** 等。

1、完整性约束关键词

- ① **PRIMARY KEY**——主键
- ② **NOT NULL**——非空值
- ③ **NULL**——空值
- ④ **UNIQUE**——值唯一
- ⑤ **CHECK**——有效性检查
- ⑥ **DEFAULT**——缺省值

2、在 PostgreSQL 常用的数据类型

(1) 数值类型：

- ① **INTEGER**：整型，4 字节。
- ② **BIGINT**：大整型，8 字节。
- ③ **SMALLINT**：小整型，2 字节。
- ④ **DECIMAL(p, s)**或**NUMERIC(p, s)**：精确小数，p 为总位数，s 为小数位数。
- ⑤ **REAL**：4 字节，单精度浮点数。
- ⑥ **DOUBLE PRECISION**：8 字节，双精度浮点数。

(2) 字符类型:

- ① **VARCHAR(n)**: 变长字符串, 最大长度为 **n**。
- ② **CHAR(n)**: 定长字符串, 长度为 **n**。
- ③ **TEXT**: 可变长度字符串, 没有长度限制。

(3) 日期和时间类型:

- ① **DATE**: 日期数据, 格式为 **YYYY-MM-DD**, 范围从 **4713 BC** 到 **5874897 AD**。
- ② **TIME**: 时间数据, 格式为 **HH:MM:SS**, 可以带时区。
- ③ **TIMESTAMP**: 时间戳数据, 格式为 **YYYY-MM-DD HH:MM:SS**。
- ④ **INTERVAL**: 表示时间间隔, 支持多种单位 (如天、小时、分钟等) 。

(4) 自增长类型:

在 PostgreSQL 中, SERIAL 是一种用于创建自增长整数列的特殊数据类型。它通常用于表的主键, 以确保每一行都有一个唯一的标识符。使用 SERIAL 类型时, PostgreSQL 会自动为新插入的行生成一个唯一的整数值。提供了三种不同大小的 SERIAL 类型:

- ① **SERIAL**: 对应于 INTEGER, 范围是从 1 到 2,147,483,647。
- ② **BIGSERIAL**: 对应于 BIGINT, 范围是从 1 到 9,223,372,036,854,775,807。
- ③ **SMALLSERIAL**: 对应于 SMALLINT, 范围是从 1 到 32,767。

(5) 布尔类型: BOOLEAN: 存储布尔值, 取值为 TRUE、FALSE 或 NULL

(6) 地理空间类型:

① POINT: 表示二维空间中的点。

② LINE: 表示无限长的直线。

③ LSEG: 表示线段。

④ BOX: 表示矩形。

⑤ POLYGON: 表示多边形。

⑥ PATH: 表示路径 (开或闭)。

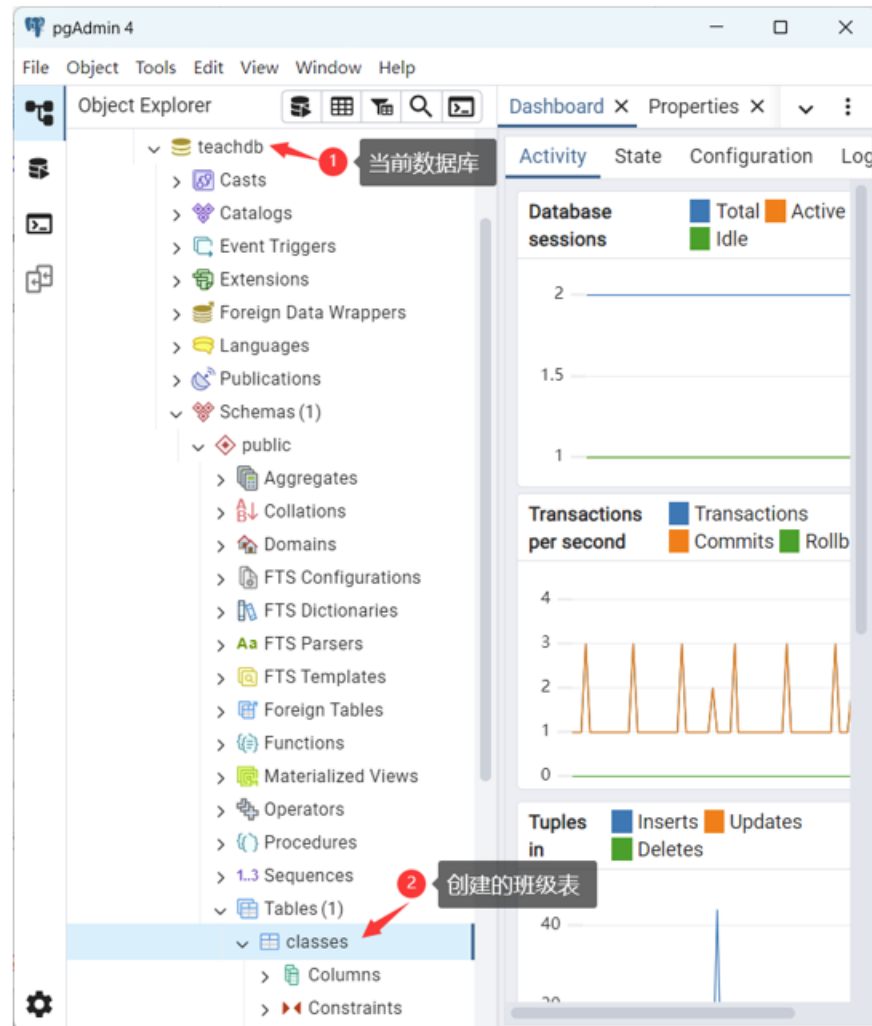
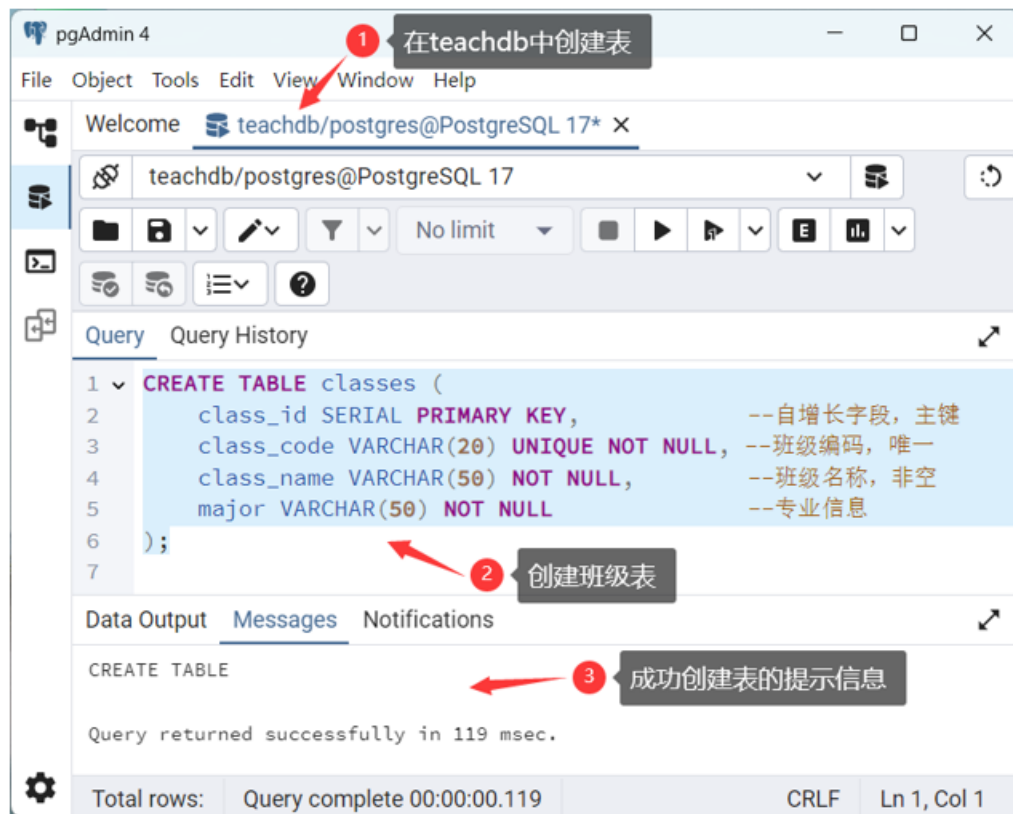
⑦ CIRCLE: 表示圆。

示例1：创建学生班级表

```
CREATE TABLE classes (  
    class_id SERIAL PRIMARY KEY,           --自增长字段，主键  
    class_code VARCHAR(20) UNIQUE NOT NULL, --班级编码，唯一  
    class_name VARCHAR(50) NOT NULL,       --班级名称，非空  
    major VARCHAR(50) NOT NULL            -- 专业信息  
);
```

- ① 自增长字段（SERIAL）在插入新记录时自动生成，确保每条记录都有一个唯一且不变的标识符。
- ② 业务字段可能会因为各种原因发生变化，这会导致主键的变更，从而影响到与其他表的关联。
- ③ 自增长的整数主键在索引中表现更好，查询和连接操作的性能更高。在处理整数时比处理字符串更快。
- ④ 自增长字段通常占用较少的存储空间，在大数据量的情况下，使用整数作为主键可显著降低存储成本。

执行创建班级表的SQL语句



示例2：创建学生信息表

```
CREATE TABLE students (  
    student_id SERIAL PRIMARY KEY,           --自增长字段，主键  
    student_number VARCHAR(20) UNIQUE NOT NULL, -- 学号，唯一  
    student_name VARCHAR(50) NOT NULL,        -- 姓名，非空  
    gender CHAR(1) CHECK (gender IN ('M', 'F')) DEFAULT 'M', -- 性别，M=男，F=女，缺省值为M  
    class_id INT,  
    CONSTRAINT classid_fk FOREIGN KEY (class_id) REFERENCES classes(class_id)  
);
```

注意： **CHECK (gender IN ('M', 'F'))**定义检查约束，gender字段取值只能是M或F

DEFAULT 'M' 定义gender的缺省值为M

CONSTRAINT定义了约束名为 classid_fk的外键约束，

class_id字段的取值参照班级表classes的字段class_id的取值。

执行创建学生表的SQL语句

pgAdmin 4

当前工作数据库

teachdb/postgres@PostgreSQL 17*

teachdb/postgres@PostgreSQL 17

Query

```
7
8 CREATE TABLE students (
9     student_id SERIAL PRIMARY KEY,
10    student_number VARCHAR(20) UNIQUE NOT NULL,
11    student_name VARCHAR(50) NOT NULL,
12    gender CHAR(1) CHECK (gender IN ('M', 'F')) DEFAULT 'M',
13    class_id INT,
14    CONSTRAINT classid_fk FOREIGN KEY (class_id) REFERENCES classes(class_id)
15 );
16
```

定义检查约束

定义缺省值

--自增长字段，主键
-- 学号，唯一
-- 姓名，非空
-- 性别，M=男，F=女，缺省值为M

定义约束名

定义外键约束

定义参照的表的主键

Messages

CREATE TABLE

Query returned successfully in 32 msec.

Total rows: Query complete 00:00:00.032 CRLF Ln 12, Col 82

7 执行成功的信息

示例3：创建教师表

```
CREATE TABLE teachers (  
    teacher_id SERIAL PRIMARY KEY,           --自增长字段，主键  
    employee_number VARCHAR(20) UNIQUE NOT NULL, -- 职工号，唯一  
    teacher_name VARCHAR(50) NOT NULL,        -- 职工姓名，非空  
    gender CHAR(1) CHECK (gender IN ('M', 'F')) DEFAULT 'M' -- 性别，M=男，F=女，缺省为M  
);
```

注意： **CHECK (gender IN ('M', 'F'))**定义检查约束，gender字段取值只能是M或F
DEFAULT 'M' 定义gender的缺省值为M

执行创建教师表的SQL语句

The screenshot shows the pgAdmin 4 interface with the following elements and annotations:

- 1 当前数据库teachdb**: Points to the database name in the top bar.
- 2 主键约束**: Points to the `PRIMARY KEY` constraint in the SQL statement.
- 3 唯一性约束**: Points to the `UNIQUE` constraint in the SQL statement.
- 4 非空约束**: Points to the `NOT NULL` constraint in the SQL statement.
- 5 检查约束**: Points to the `CHECK` constraint in the SQL statement.
- 6 缺省约束**: Points to the `DEFAULT` constraint in the SQL statement.
- 7 执行结果信息**: Points to the message "Query returned successfully in 37 msec." in the Messages pane.
- 8 执行按钮**: Points to the Execute button (a green play icon) in the toolbar.

The SQL statement being executed is:

```
1 CREATE TABLE teachers (  
2     teacher_id SERIAL PRIMARY KEY,  
3     employee_number VARCHAR(20) UNIQUE NOT NULL,  
4     teacher_name VARCHAR(50) NOT NULL,  
5     gender CHAR(1) CHECK (gender IN ('M', 'F')) DEFAULT 'M'  
6 );  
7
```

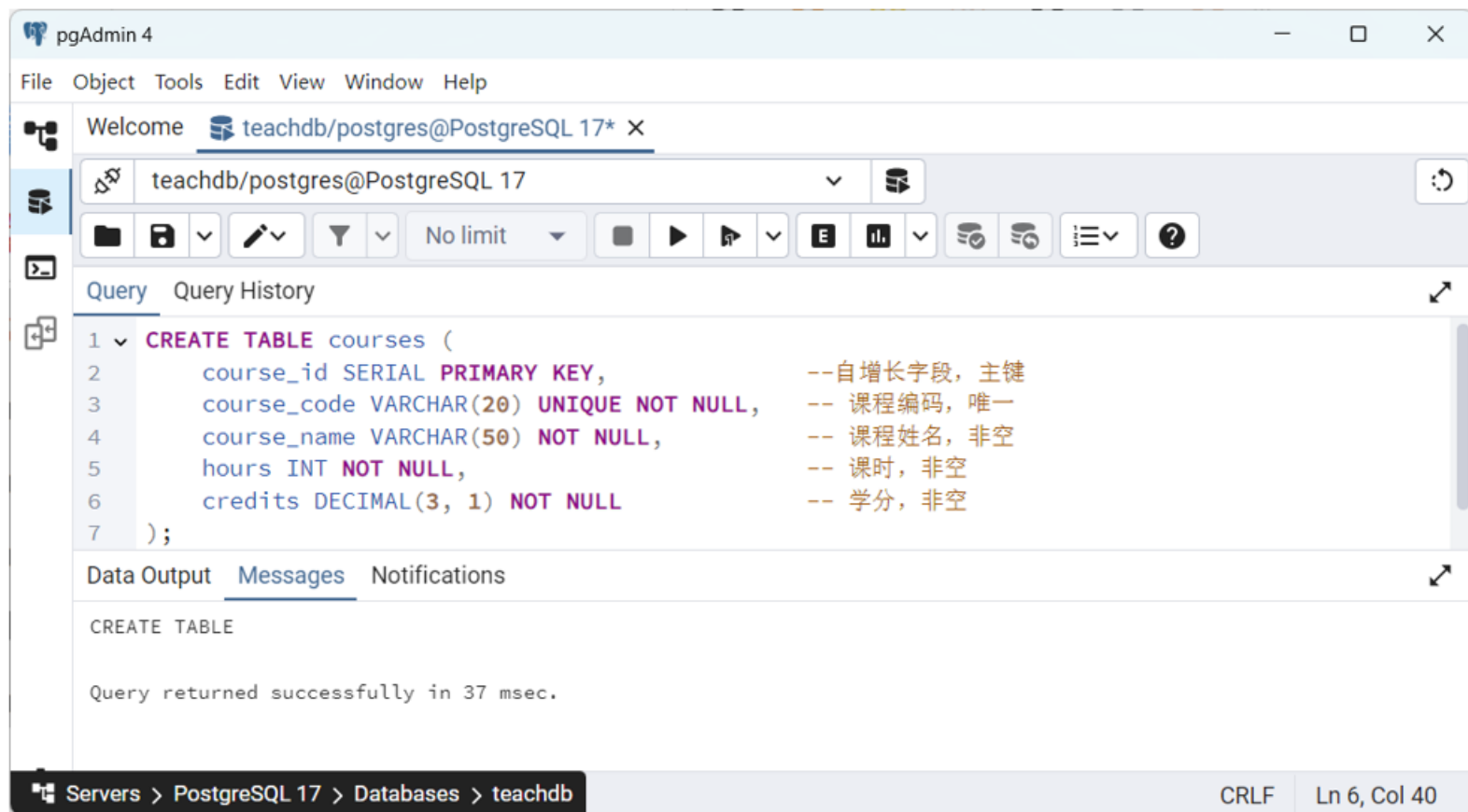
The Messages pane shows the output:

```
CREATE TABLE  
  
Query returned successfully in 37 msec.
```


示例4：创建课程表

```
CREATE TABLE courses (  
    course_id SERIAL PRIMARY KEY,           --自增长字段，主键  
    course_code VARCHAR(20) UNIQUE NOT NULL, -- 课程编码，唯一  
    course_name VARCHAR(50) NOT NULL,       -- 课程姓名，非空  
    hours INT NOT NULL,                     -- 课时，非空  
    credits DECIMAL(3, 1) NOT NULL         -- 学分，非空  
);
```

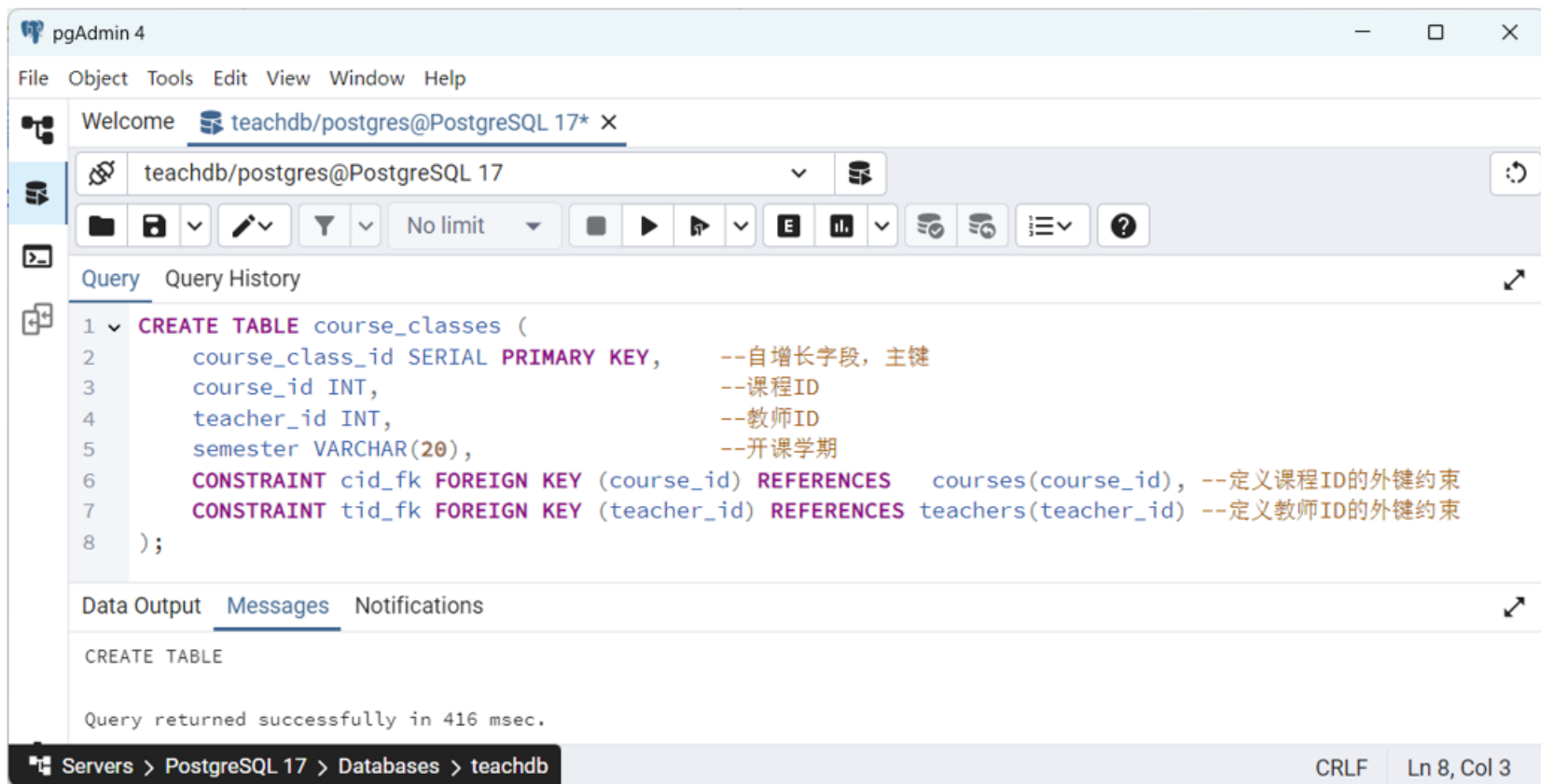
执行创建课程表的SQL语句



示例5：创建授课班级表

```
CREATE TABLE course_classes (  
    course_class_id SERIAL PRIMARY KEY, --自增长字段，主键  
    course_id INT,                      --课程ID  
    teacher_id INT,                    --教师ID  
    semester VARCHAR(20),              --开课学期  
    CONSTRAINT cid_fk FOREIGN KEY (course_id) REFERENCES courses(course_id), --定义课程ID外键约束  
    CONSTRAINT tid_fk FOREIGN KEY (teacher_id) REFERENCES teachers(teacher_id) --定义教师ID外键约束  
);
```

执行创建授课班级表的SQL语句



示例6：创建选课表

```
CREATE TABLE enrollments (  
    student_id INT,                                --学生ID  
    course_class_id INT,                            --课程班ID  
    score DECIMAL(5, 2),                            --考试分数  
    CONSTRAINT enrollment_pk PRIMARY KEY (student_id, course_class_id), --定义复合主键  
    CONSTRAINT sid_fk FOREIGN KEY (student_id) REFERENCES students(student_id), --定义学生ID外键约束  
    CONSTRAINT course_cid_fk FOREIGN KEY (course_class_id) REFERENCES course_classes(course_class_id)  
    --定义课程班ID外键约束  
);
```

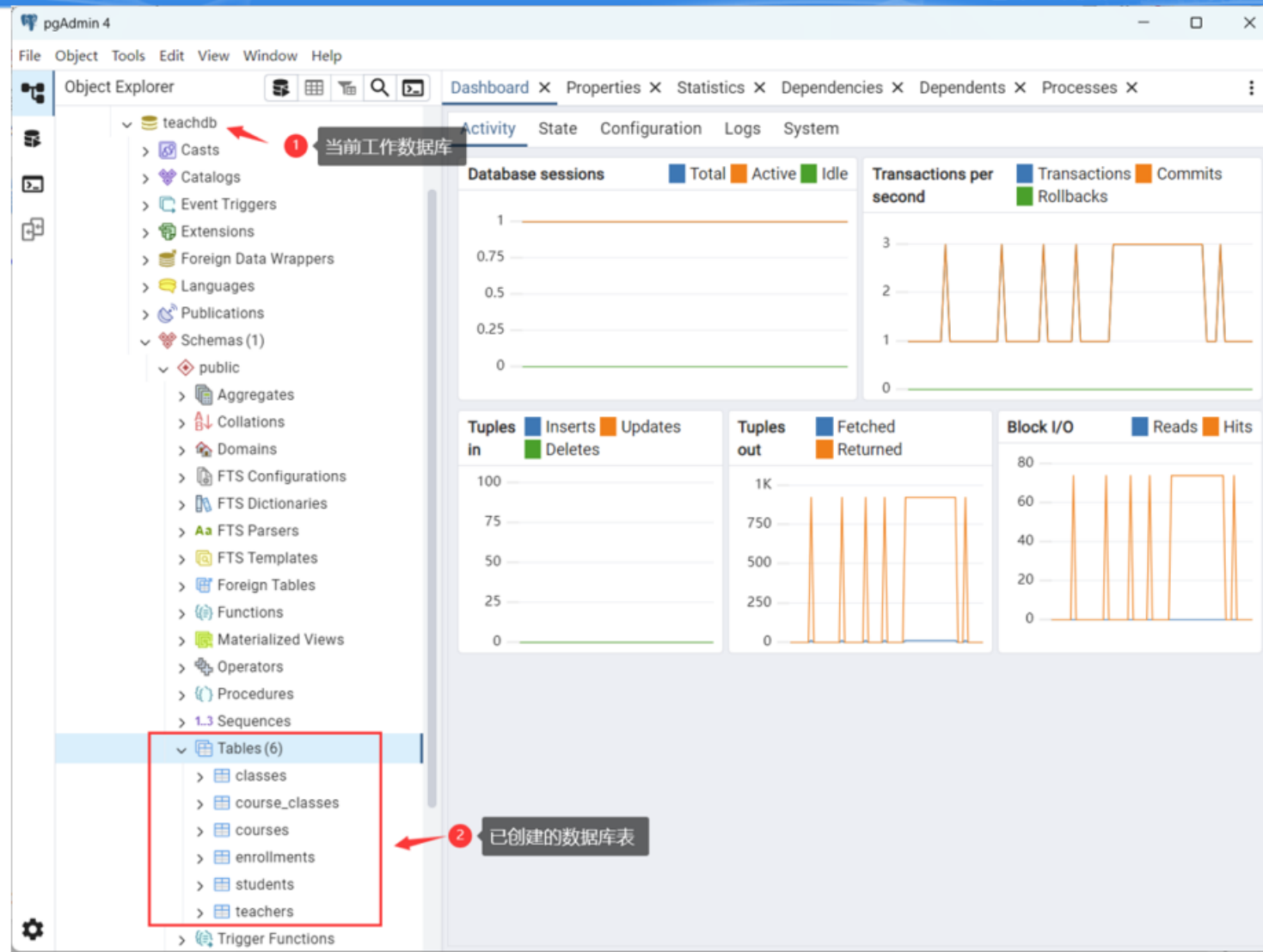
执行创建授课班级表的SQL语句

The screenshot shows the pgAdmin 4 web interface. The top menu bar includes File, Object, Tools, Edit, View, Window, and Help. The main toolbar contains icons for connecting, saving, editing, filtering, and executing queries. The 'Query' tab is active, displaying the following SQL statement:

```
1 CREATE TABLE enrollments (  
2     student_id INT,           --学生ID  
3     course_class_id INT,      --课程班ID  
4     score DECIMAL(5, 2),      --考试分数  
5     CONSTRAINT enrollment_pk PRIMARY KEY (student_id, course_class_id), --定义复合主键  
6     CONSTRAINT sid_fk FOREIGN KEY (student_id) REFERENCES students(student_id), --定义学生ID外键约束  
7     CONSTRAINT course_cid_fk FOREIGN KEY (course_class_id) REFERENCES course_classes(course_class_id) --定义课程班ID外键约束  
8 );
```

Below the query editor, the 'Messages' tab is active, showing the output: 'CREATE TABLE' and 'Query returned successfully in 416 msec.' The bottom status bar indicates the current location: 'Servers > PostgreSQL 17 > Databases > teachdb' and the cursor position: 'Ln 2, Col 30'.

已创建的 数据库表



六、修改数据库表结构SQL语句

1.语句基本格式

ALTER TABLE <表名> <修改方式>;

2.主要语句类型

1) **ADD**修改方式，用于增加新列或列完整性约束

ALTER TABLE <表名> **ADD** <新列名称><数据类型>[[完整性约束]

2) DROP修改方式，用于删除指定列或列的完整性约束条件

ALTER TABLE <表名> DROP COLUMN <列名>;

ALTER TABLE <表名> DROP CONSTRAINT <完整性约束名>;

3) RENAME修改方式，用于修改表名称、列名称

ALTER TABLE <表名> RENAME TO <新表名>;

ALTER TABLE <表名> RENAME <原列名> TO <新列名>;

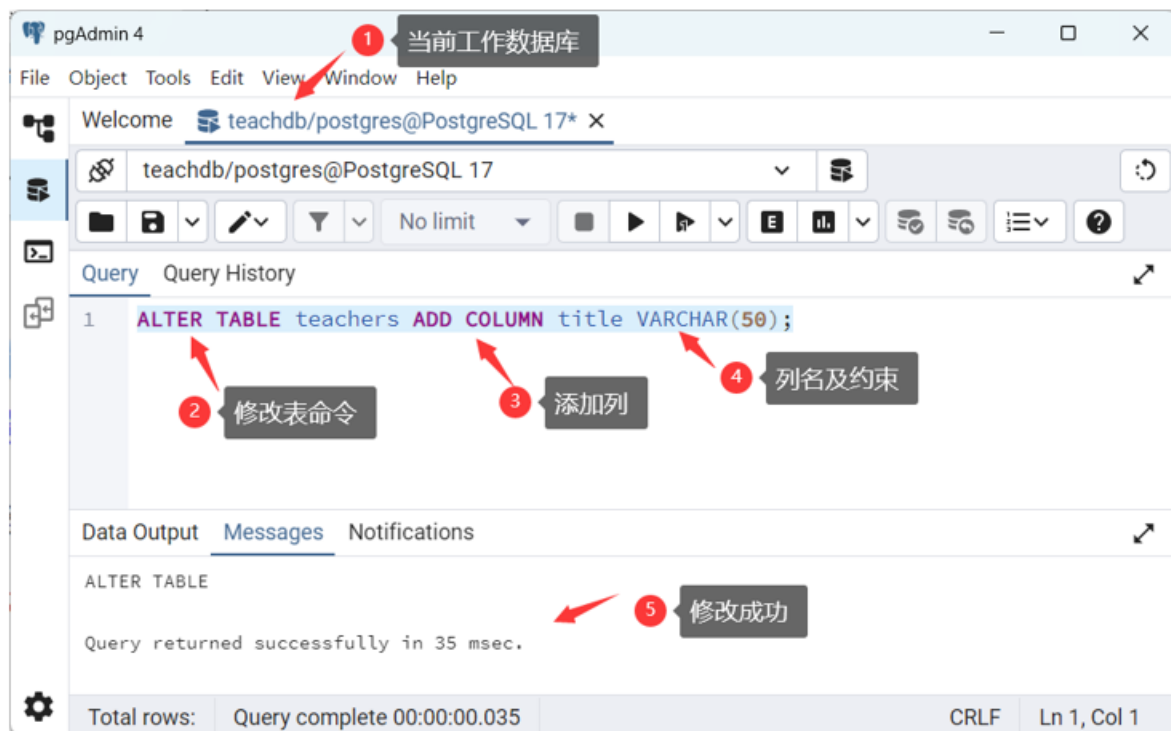
4) ALTER修改方式，用于修改列的数据类型

ALTER TABLE <表名> ALTER COLUMN <列名> TYPE <新的数据类型>;

3.修改表结构的示例

修改教师表teachers，添加职称字段。

ALTER TABLE teachers **ADD COLUMN** title **VARCHAR(50);**



七、删除表结构SQL语句

1.语句基本格式

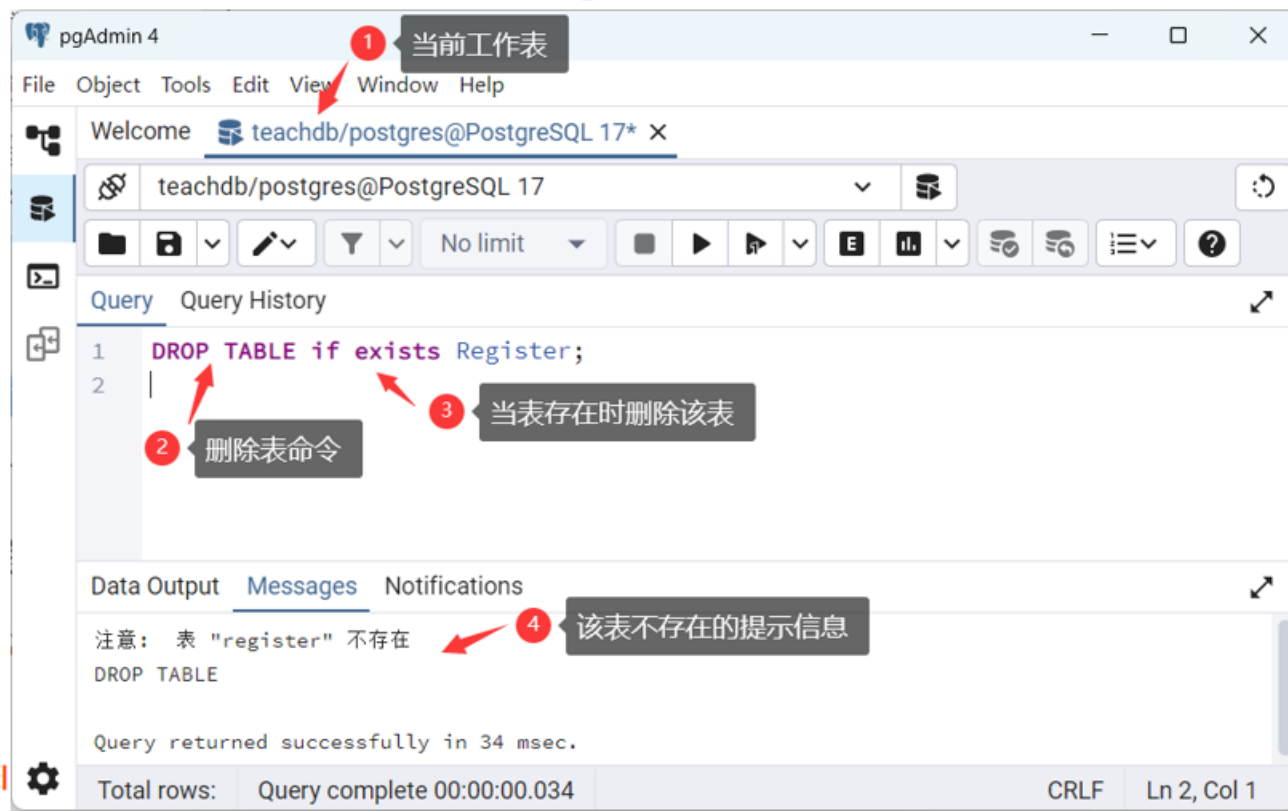
DROP TABLE <表名>;

注意: 该语句将删除该表的所有数据及其结构

2.表删除举例

例：删除注册表Register表及其数据,可以使用如下语句。

DROP TABLE if exists Register;



八、向表中插入记录

1.插入单行数据的语法

```
INSERT INTO table_name (column1, column2, column3, ...)  
VALUES (value1, value2, value3, ...);
```

- ① **table_name**: 要插入数据的表的名称。
- ② **column1, column2, ...**: 要插入数据的列名。
- ③ **value1, value2, ...**: 对应列的值。

插入向班级表一条指令插入一条记录的示例：

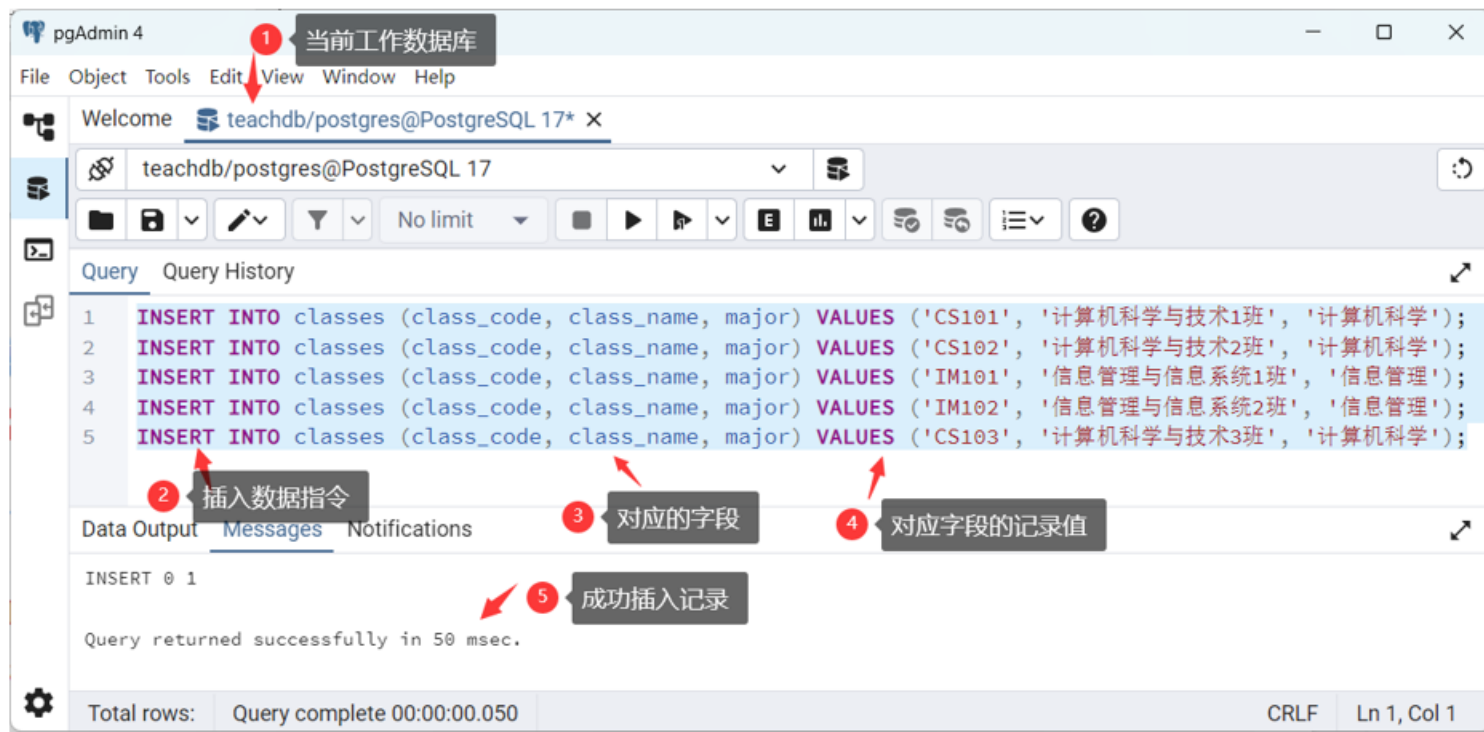
```
INSERT INTO classes (class_code, class_name, major) VALUES ('CS101', '计算机科学与技术1班', '计算机科学');
```

```
INSERT INTO classes (class_code, class_name, major) VALUES ('CS102', '计算机科学与技术2班', '计算机科学');
```

```
INSERT INTO classes (class_code, class_name, major) VALUES ('IM101', '信息管理与信息系统1班', '信息管理');
```

```
INSERT INTO classes (class_code, class_name, major) VALUES ('IM102', '信息管理与信息系统2班', '信息管理');
```

```
INSERT INTO classes (class_code, class_name, major) VALUES ('CS103', '计算机科学与技术3班', '计算机科学');
```



插入向学生表一条指令插入一条记录的示例：

```
INSERT INTO students (student_number, student_name, gender, class_id) VALUES ('20230001', '张三', 'M', 1);
```

```
INSERT INTO students (student_number, student_name, gender, class_id) VALUES ('20230002', '李四', 'F', 1);
```

```
INSERT INTO students (student_number, student_name, gender, class_id) VALUES ('20230003', '王五', 'M', 1);
```

```
INSERT INTO students (student_number, student_name, gender, class_id) VALUES ('20230004', '赵六', 'F', 2);
```

```
INSERT INTO students (student_number, student_name, gender, class_id) VALUES ('20230005', '钱七', 'M', 2);
```

```
INSERT INTO students (student_number, student_name, gender, class_id) VALUES ('20230006', '孙八', 'F', 1);
```

```
INSERT INTO students (student_number, student_name, gender, class_id) VALUES ('20230007', '周九', 'M', 1);
```

```
INSERT INTO students (student_number, student_name, gender, class_id) VALUES ('20230008', '吴十', 'F', 2);
```

```
INSERT INTO students (student_number, student_name, gender, class_id) VALUES ('20230009', '郑十一', 'M', 2);
```

```
INSERT INTO students (student_number, student_name, gender, class_id) VALUES ('20230010', '冯十二', 'F', 3);
```


执行向学生表插入数据：

The screenshot shows the pgAdmin 4 interface with the following components:

- Top Bar:** pgAdmin 4, File, Object, Tools, Edit, View, Window, Help.
- Database Connection:** teachdb/postgres@PostgreSQL 17* x
- Query Editor:** Contains 10 lines of SQL code for inserting data into the 'students' table. The code is highlighted in blue.
- Query:** `INSERT INTO students (student_number, student_name, gender, class_id) VALUES ('20230001', '张三', 'M', 1);`
`INSERT INTO students (student_number, student_name, gender, class_id) VALUES ('20230002', '李四', 'F', 1);`
`INSERT INTO students (student_number, student_name, gender, class_id) VALUES ('20230003', '王五', 'M', 1);`
`INSERT INTO students (student_number, student_name, gender, class_id) VALUES ('20230004', '赵六', 'F', 2);`
`INSERT INTO students (student_number, student_name, gender, class_id) VALUES ('20230005', '钱七', 'M', 2);`
`INSERT INTO students (student_number, student_name, gender, class_id) VALUES ('20230006', '孙八', 'F', 1);`
`INSERT INTO students (student_number, student_name, gender, class_id) VALUES ('20230007', '周九', 'M', 1);`
`INSERT INTO students (student_number, student_name, gender, class_id) VALUES ('20230008', '吴十', 'F', 2);`
`INSERT INTO students (student_number, student_name, gender, class_id) VALUES ('20230009', '郑十一', 'M', 2);`
`INSERT INTO students (student_number, student_name, gender, class_id) VALUES ('20230010', '冯十二', 'F', 3);`
- Data Output:** INSERT 0 1
- Messages:** Query returned successfully in 66 msec.
- Footer:** Total rows: Query complete 00:00:00.066, CRLF, Ln 10, Col 106

Annotations in the image:

- 1: 当前工作数据库 (Current working database)
- 2: 学生表的字段 (Fields of the students table)
- 3: 对应字段的值 (Values corresponding to the fields)
- 4: 插入成功的提示信息 (Insertion success message)

向课程表插入课程数据:

```
INSERT INTO courses (course_code, course_name, hours, credits) VALUES ('C001', '数据库系统', 48, 3.0);  
INSERT INTO courses (course_code, course_name, hours, credits) VALUES ('C002', '操作系统', 64, 4.0);  
INSERT INTO courses (course_code, course_name, hours, credits) VALUES ('C003', '计算机网络', 32, 2.0);  
INSERT INTO courses (course_code, course_name, hours, credits) VALUES ('C004', '软件工程', 48, 3.0);  
INSERT INTO courses (course_code, course_name, hours, credits) VALUES ('C005', '人工智能', 40, 3.0);  
INSERT INTO courses (course_code, course_name, hours, credits) VALUES ('C006', '数据结构', 36, 2.0);  
INSERT INTO courses (course_code, course_name, hours, credits) VALUES ('C007', '算法分析', 48, 3.0);  
INSERT INTO courses (course_code, course_name, hours, credits) VALUES ('C008', '软件测试', 40, 3.0);  
INSERT INTO courses (course_code, course_name, hours, credits) VALUES ('C009', '计算机图形学', 32, 2.0);  
INSERT INTO courses (course_code, course_name, hours, credits) VALUES ('C010', '机器学习', 48, 4.0);  
INSERT INTO courses (course_code, course_name, hours, credits) VALUES ('C011', '自然语言处理', 48, 4.0);
```

执行向课程表插入课程数据：

pgAdmin 4

File Object Tools Edit View Window Help

Welcome teachdb/postgres@PostgreSQL 17*

teachdb/postgres@PostgreSQL 17

No limit

Query Query History

```
1 -- 插入课程数据（包含课程编码、学时数和学分数）
2 INSERT INTO courses (course_code, course_name, hours, credits) VALUES ('C001', '数据库系统', 48, 3.0);
3 INSERT INTO courses (course_code, course_name, hours, credits) VALUES ('C002', '操作系统', 64, 4.0);
4 INSERT INTO courses (course_code, course_name, hours, credits) VALUES ('C003', '计算机网络', 32, 2.0);
5 INSERT INTO courses (course_code, course_name, hours, credits) VALUES ('C004', '软件工程', 48, 3.0);
6 INSERT INTO courses (course_code, course_name, hours, credits) VALUES ('C005', '人工智能', 40, 3.0);
7 INSERT INTO courses (course_code, course_name, hours, credits) VALUES ('C006', '数据结构', 36, 2.0);
8 INSERT INTO courses (course_code, course_name, hours, credits) VALUES ('C007', '算法分析', 48, 3.0);
9 INSERT INTO courses (course_code, course_name, hours, credits) VALUES ('C008', '软件测试', 40, 3.0);
10 INSERT INTO courses (course_code, course_name, hours, credits) VALUES ('C009', '计算机图形学', 32, 2.0);
11 INSERT INTO courses (course_code, course_name, hours, credits) VALUES ('C010', '机器学习', 48, 4.0);
```

Data Output Messages Notifications

INSERT 0 1

Query returned successfully in 55 msec.

Total rows: Query complete 00:00:00.055 CRLF Ln 11, Col 97

1 当前工作数据库

2 课程表

3 并不包括自增长字段的所有字段

4 与指定字段对应的值

5 插入成功的信息

2.插入多行数据的语法

INSERT INTO table_name (column1, column2, column3, ...)

VALUES

(value1a, value2a, value3a, ...),

(value1b, value2b, value3b, ...),

(value1c, value2c, value3c, ...);

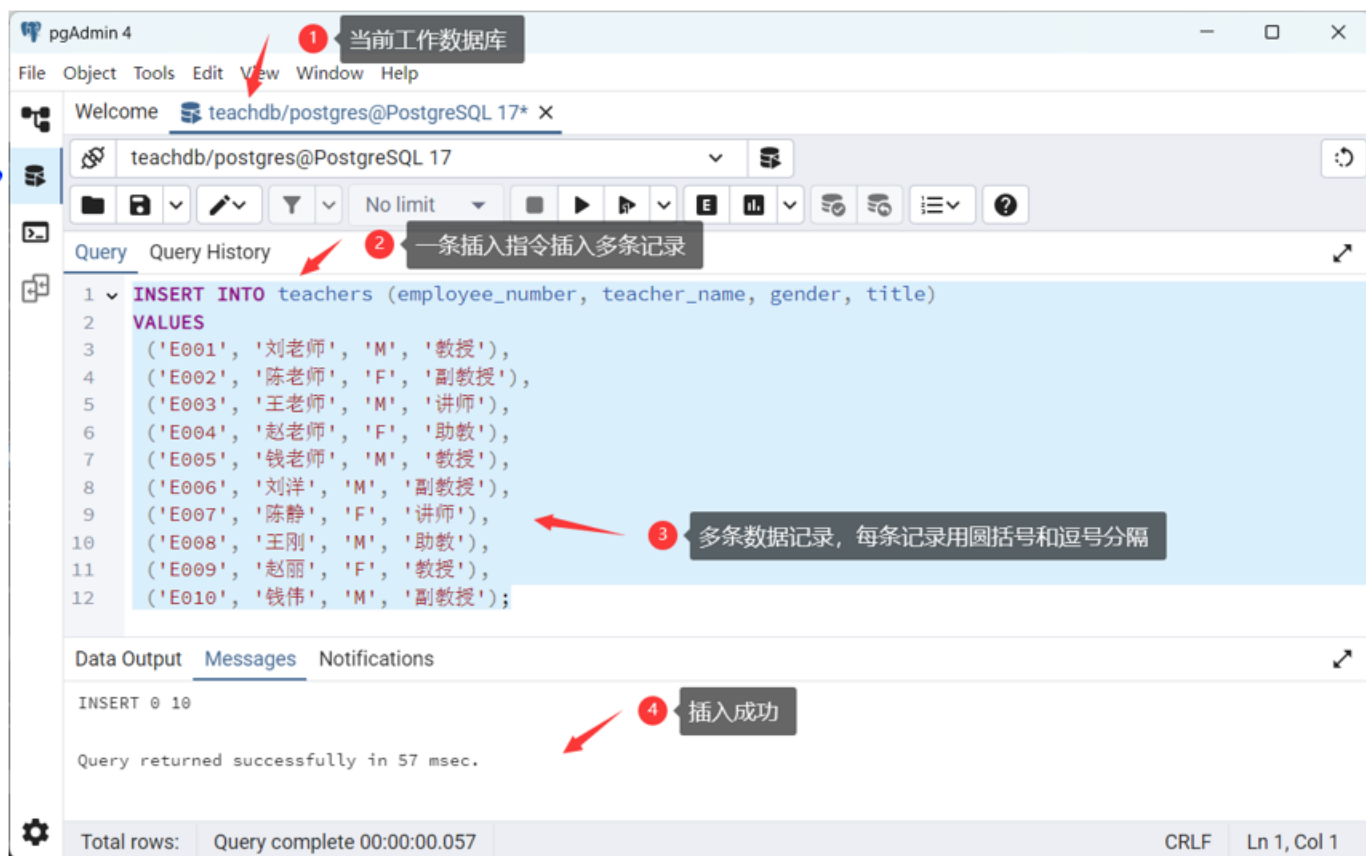
注意：可以在 **VALUES** 子句中使用逗号分隔的多个值列表来插入多行数据。

插入向教师表一条指令插入多条记录的示例：

INSERT INTO teachers (employee_number, teacher_name, gender, title)

VALUES

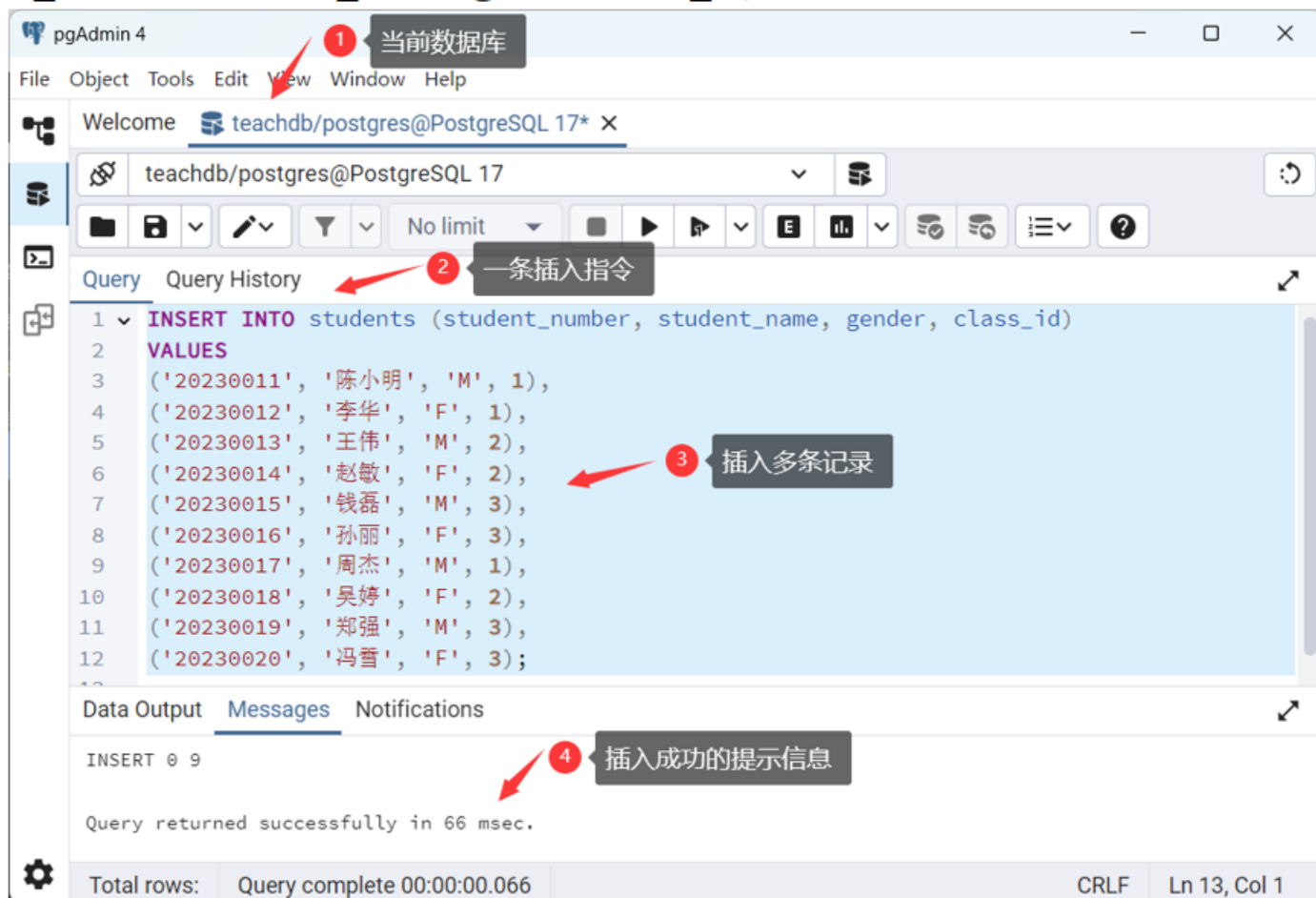
('E001', '刘老师', 'M', '教授'),
('E002', '陈老师', 'F', '副教授'),
('E003', '王老师', 'M', '讲师'),
('E004', '赵老师', 'F', '助教'),
('E005', '钱老师', 'M', '教授'),
('E006', '刘洋', 'M', '副教授'),
('E007', '陈静', 'F', '讲师'),
('E008', '王刚', 'M', '助教'),
('E009', '赵丽', 'F', '教授'),
('E010', '钱伟', 'M', '副教授');



插入向学生表插入多条记录的示例：

INSERT INTO students (student_number, student_name, gender, class_id)
VALUES

('20230011', '陈小明', 'M', 1),
('20230012', '李华', 'F', 1),
('20230013', '王伟', 'M', 2),
('20230014', '赵敏', 'F', 2),
('20230015', '钱磊', 'M', 3),
('20230016', '孙丽', 'F', 3),
('20230017', '周杰', 'M', 1),
('20230018', '吴婷', 'F', 2),
('20230019', '郑强', 'M', 3),
('20230020', '冯雪', 'F', 3);



插入向课程班级表插入多条记录的示例：

INSERT INTO course_classes (course_id, teacher_id, semester)

VALUES

(1, 1, '2023-1'), -- 刘老师教授 数据库系统

(2, 2, '2023-1'), -- 陈老师教授 操作系统

(3, 3, '2023-1'), -- 王老师教授 计算机网络

(4, 4, '2023-1'), -- 赵老师教授 软件工程

(5, 5, '2023-1'), -- 钱老师教授 人工智能

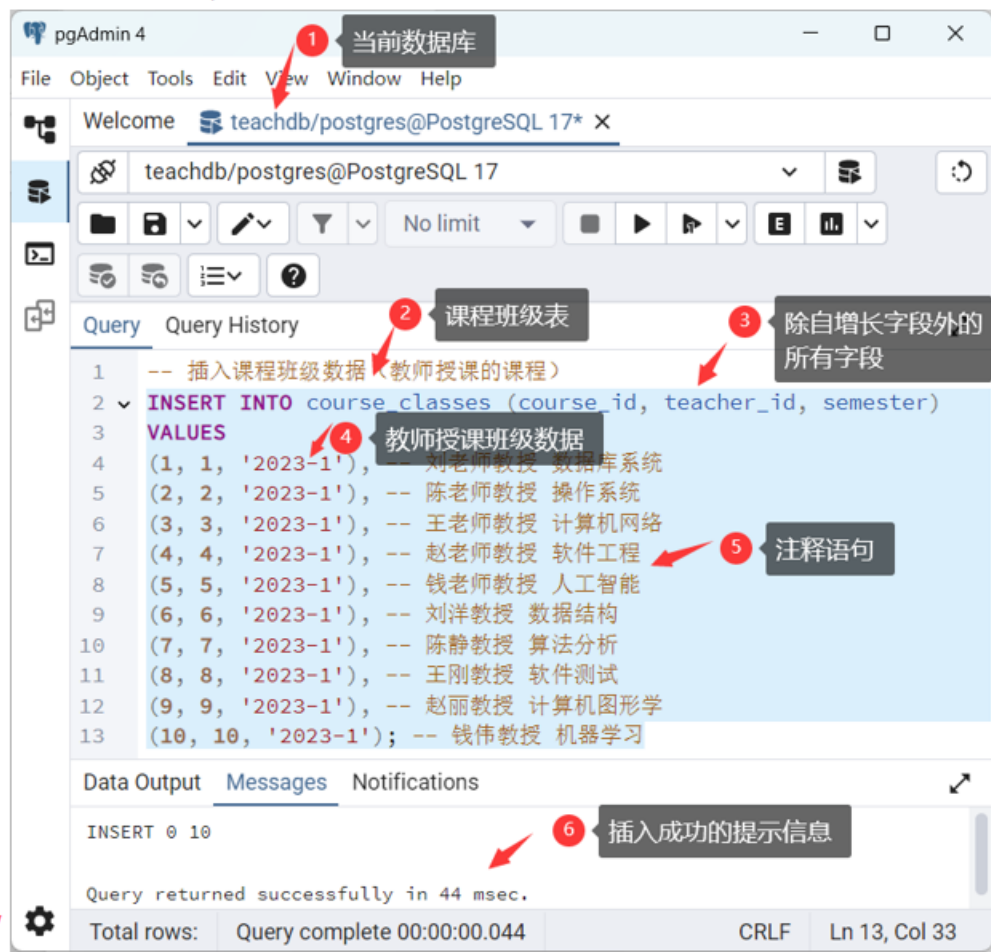
(6, 6, '2023-1'), -- 刘洋教授 数据结构

(7, 7, '2023-1'), -- 陈静教授 算法分析

(8, 8, '2023-1'), -- 王刚教授 软件测试

(9, 9, '2023-1'), -- 赵丽教授 计算机图形学

(10, 10, '2023-1'); -- 钱伟教授 机器学习

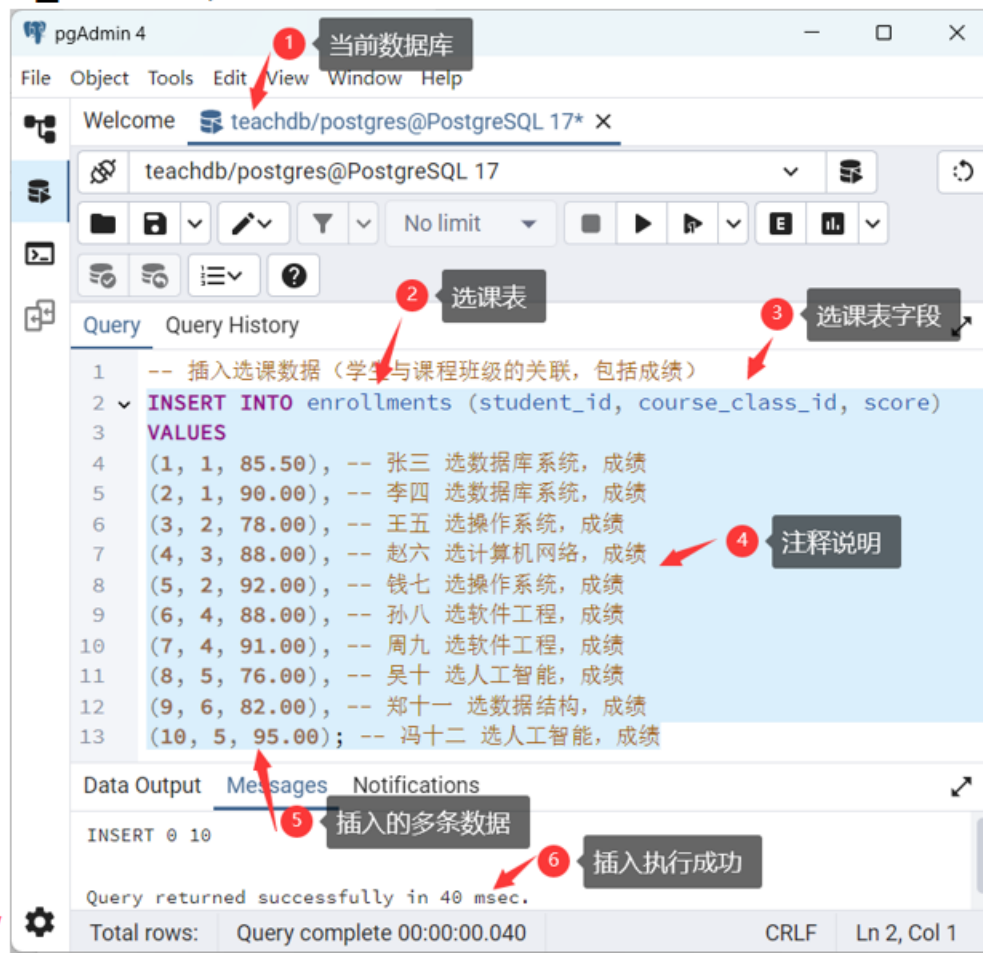


插入向选课表插入多条记录的示例：

INSERT INTO enrollments (student_id, course_class_id, score)

VALUES

(1, 1, 85.50), -- 张三 选数据库系统, 成绩
(2, 1, 90.00), -- 李四 选数据库系统, 成绩
(3, 2, 78.00), -- 王五 选操作系统, 成绩
(4, 3, 88.00), -- 赵六 选计算机网络, 成绩
(5, 2, 92.00), -- 钱七 选操作系统, 成绩
(6, 4, 88.00), -- 孙八 选软件工程, 成绩
(7, 4, 91.00), -- 周九 选软件工程, 成绩
(8, 5, 76.00), -- 吴十 选人工智能, 成绩
(9, 6, 82.00), -- 郑十一 选数据结构, 成绩
(10, 5, 95.00); -- 冯十二 选人工智能, 成绩



3.省略列名

如果要插入的所有列都有值，可以省略列名。值的顺序必须与表中列的顺序一致。

INSERT INTO enrollments

VALUES

(11, 1, 88.00), -- 陈小明 选数据库系统, 成绩

(12, 1, 91.00), -- 李华 选数据库系统, 成绩

(13, 2, 79.00), -- 王伟 选操作系统, 成绩

(14, 3, 85.00), -- 赵敏 选计算机网络, 成绩

(15, 2, 94.00), -- 钱磊 选操作系统, 成绩

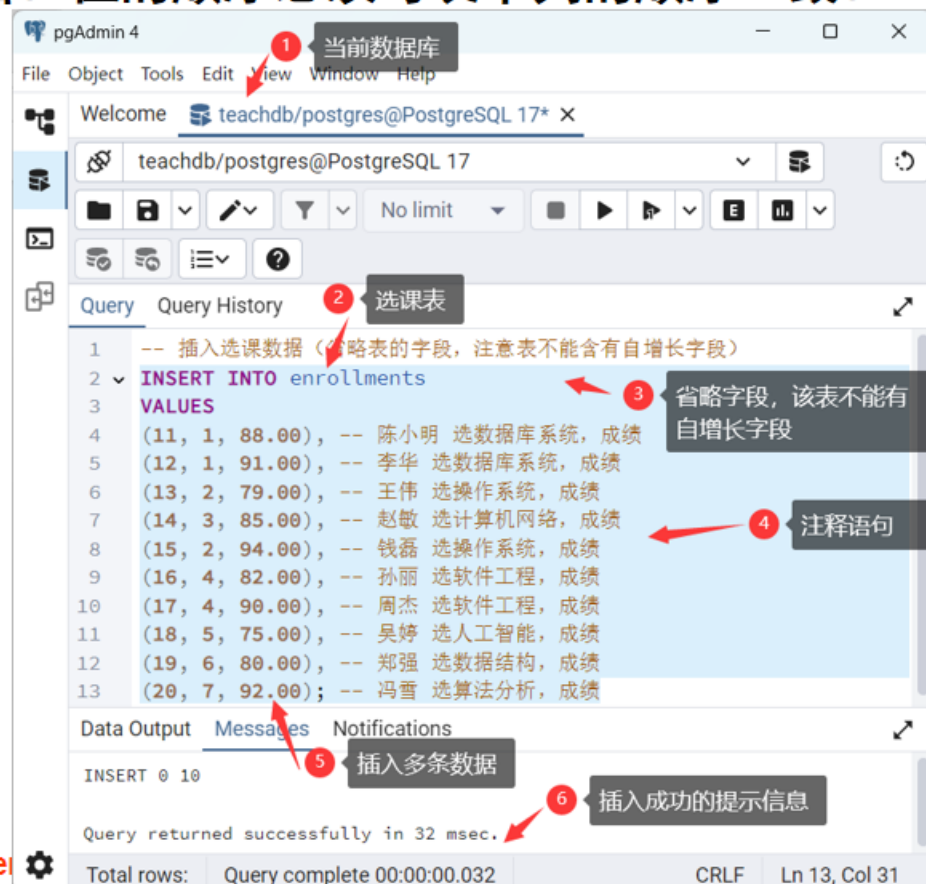
(16, 4, 82.00), -- 孙丽 选软件工程, 成绩

(17, 4, 90.00), -- 周杰 选软件工程, 成绩

(18, 5, 75.00), -- 吴婷 选人工智能, 成绩

(19, 6, 80.00), -- 郑强 选数据结构, 成绩

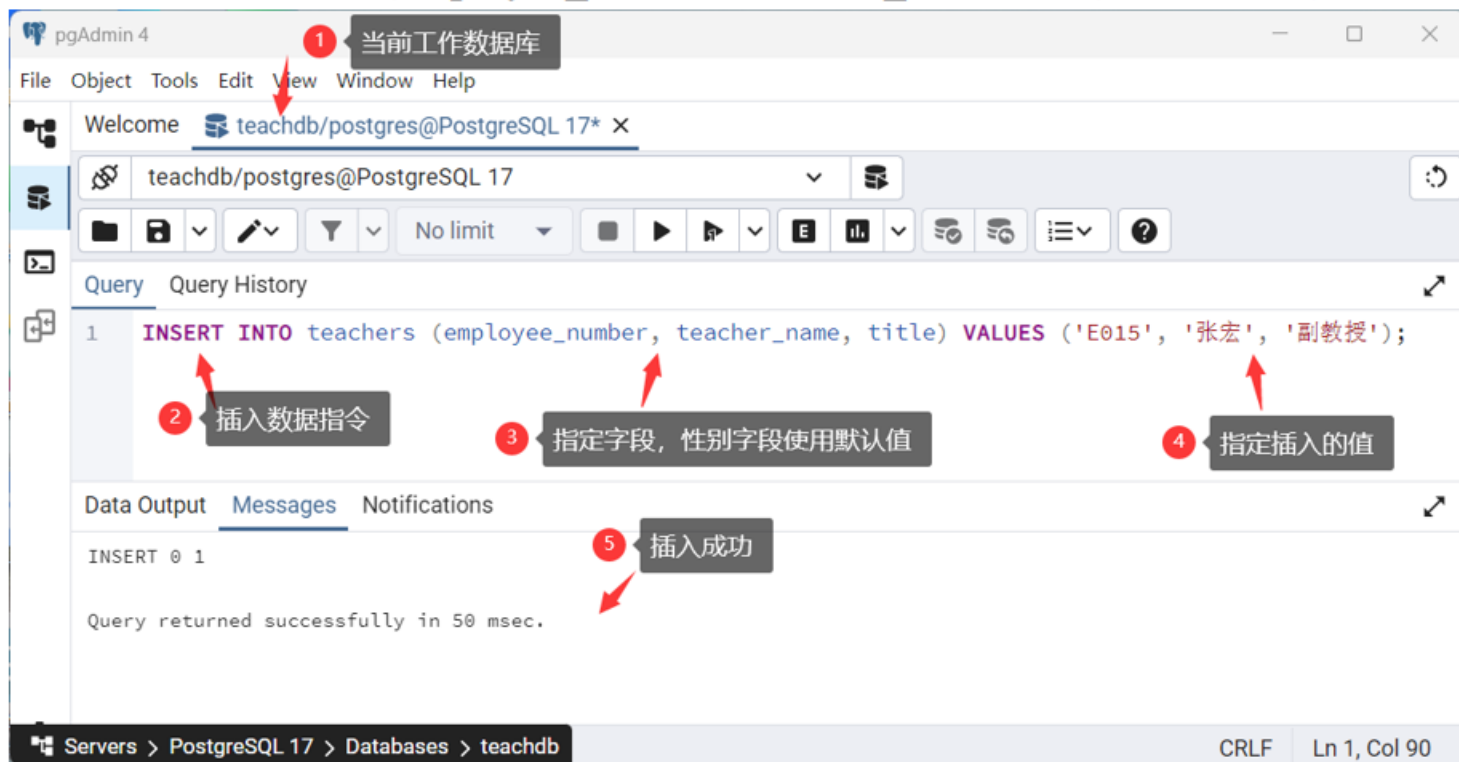
(20, 7, 92.00); -- 冯雪 选算法分析, 成绩



4.使用默认值

如果某个列有默认值，可以在插入时省略该列，PostgreSQL 将使用默认值。

INSERT INTO teachers (employee_number, teacher_name, title) **VALUES** ('E015', '张宏', '副教授');



九、SQL内置函数类型

PostgreSQL 提供了丰富的内置函数，涵盖了多种数据类型和操作。这些函数可以帮助用户在数据库中执行各种计算、数据处理和转换。以下是 PostgreSQL 中一些常见的 SQL 内置函数类型的详细讲解。

(1) 数学函数

① **ABS(x)**: 返回 x 的绝对值。

SELECT ABS(-10); -- 返回 10

② **CEIL(x)** 或 **CEILING(x)**: 返回大于或等于 x 的最小整数。

SELECT CEIL(10.5); -- 返回 11

③ **FLOOR(x)**: 返回小于或等于 x 的最大整数。

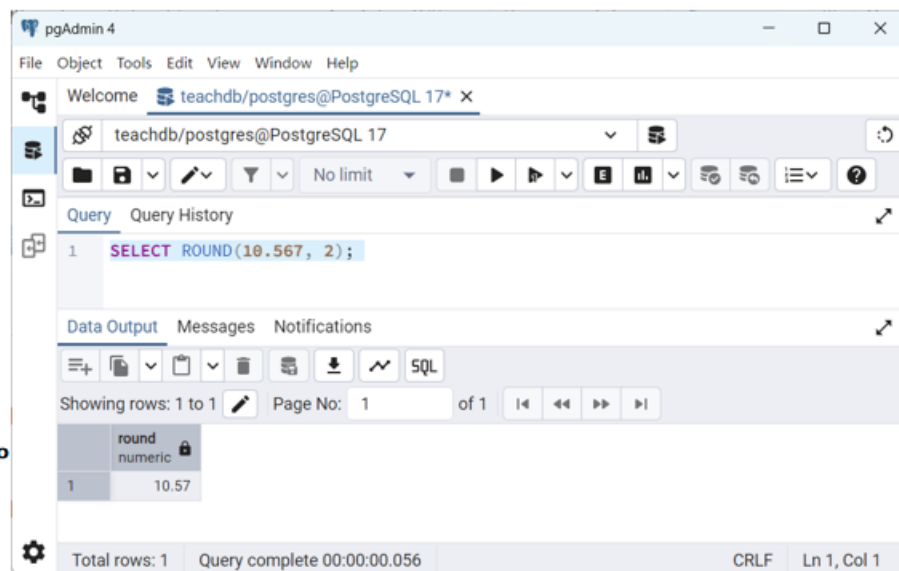
SELECT FLOOR(10.5); -- 返回 10

④ **ROUND(x, d)**: 将 x 四舍五入到 d 位小数。

SELECT ROUND(10.567, 2); -- 返回 10.57

⑤ **RANDOM()**: 返回一个 0 到 1 之间的随机数。

SELECT RANDOM(); -- 返回随机数



(2) 字符串函数

- ① **LENGTH(string)**: 返回字符串的长度。

SELECT LENGTH('Hello'); -- 返回 5

- ② **UPPER(string)**: 将字符串转换为大写。

SELECT UPPER('hello'); -- 返回 'HELLO'

- ③ **LOWER(string)**: 将字符串转换为小写。

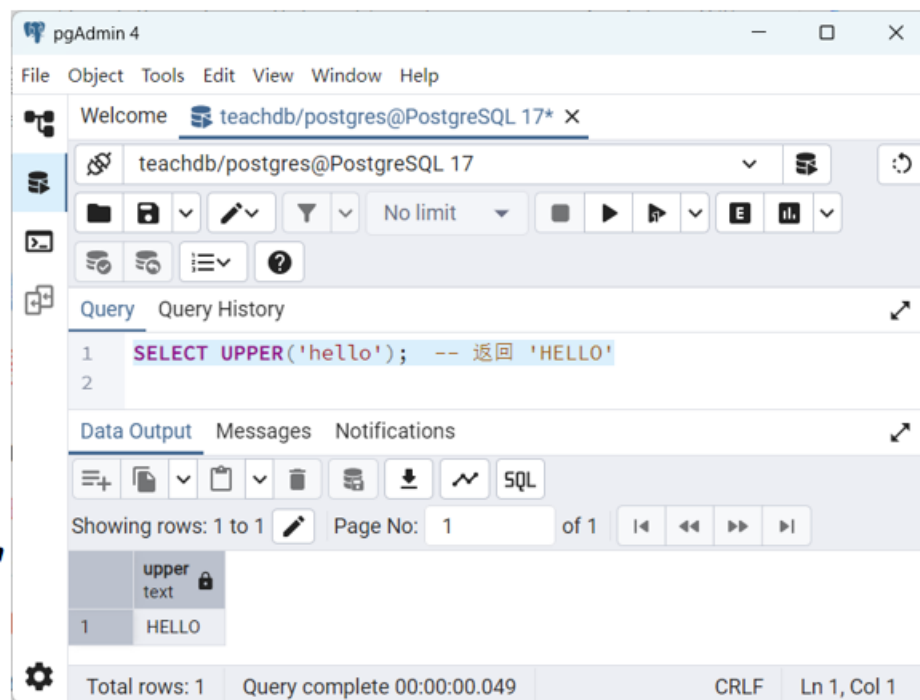
SELECT LOWER('HELLO'); -- 返回 'hello'

- ④ **TRIM(string)**: 去除字符串两端的空格。

SELECT TRIM(' Hello '); -- 返回 'Hello'

- ⑤ **SUBSTRING(string FROM start FOR length)**: 返回字符串的子串。

SELECT SUBSTRING('Hello World' FROM 1 FOR 5); -- 返回 'Hello'



(3) 日期和时间函数

- ① **CURRENT_DATE**: 返回当前日期。

SELECT CURRENT_DATE; -- 返回当前日期

- ② **CURRENT_TIME**: 返回当前时间。

SELECT CURRENT_TIME; -- 返回当前时间

- ③ **NOW()**: 返回当前的日期和时间。

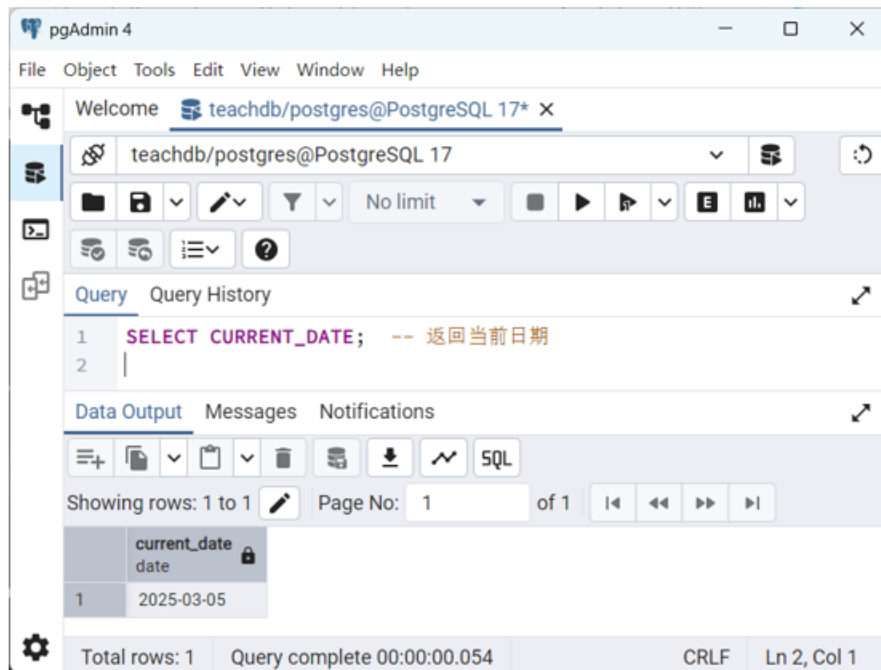
SELECT NOW(); -- 返回当前的日期和时间

- ④ **AGE(timestamp)**: 返回当前时间与给定时间之间的间隔

SELECT AGE(NOW(), '2020-01-01'); -- 返回时间间隔

- ⑤ **DATE_PART(field, source)**: 提取日期或时间的特定部分。

SELECT DATE_PART('year', NOW()); -- 返回当前年份



(4) 聚合函数

- ① **COUNT(column)**: 返回指定列的行数。

SELECT COUNT(student_id) FROM students; -- 返回学生数量

- ② **SUM(column)**: 返回指定列的总和。

SELECT SUM(score) FROM enrollments; -- 返回成绩总和

- ③ **AVG(column)**: 返回指定列的平均值。

SELECT AVG(score) FROM enrollments; -- 返回平均成绩

- ④ **MAX(column)**: 返回指定列的最大值

SELECT MAX(score) FROM enrollments; -- 返回最高成绩

- ⑤ **MIN(column)**: 返回指定列的最小值。

SELECT MIN(score) FROM enrollments; -- 返回最低成绩

(5) 条件函数

- ① COALESCE(value1, value2, ...): 返回第一个非空值。

```
SELECT COALESCE(NULL, NULL, 'Hello'); -- 返回 'Hello'
```

- ② NULLIF(value1, value2): 如果 value1 等于 value2, 则返回 NULL, 否则返回 value1。

```
SELECT NULLIF(10, 10); -- 返回 NULL
```

- ③ CASE: 用于条件判断。

```
SELECT
```

```
  CASE
```

```
    WHEN score >= 90 THEN '优秀'
```

```
    WHEN score >= 80 THEN '良好'
```

```
    ELSE '及格'
```

```
  END AS performance
```

```
FROM enrollments;
```

(6) 类型转换函数

- ① **CAST(expression AS type)**: 将表达式转换为指定类型。

SELECT CAST('123' AS INTEGER); -- 返回 123

- ② **::type**: 另一种类型转换的语法。

SELECT '123'::INTEGER; -- 返回 123

十、SQL查询语句的语法格式

SELECT column1, column2, ...

FROM table_name

WHERE condition

ORDER BY column1, column2, ...

GROUP BY column1, column2, ...

HAVING condition

LIMIT number

OFFSET number;

- ① **SELECT**: 指定要查询的列。
- ② **FROM**: 指定要查询的表。
- ③ **WHERE**: 用于过滤结果集, 指定条件。
- ④ **ORDER BY**: 用于对结果集进行排序。
- ⑤ **GROUP BY**: 用于将结果集按一个或多个列进行分组。
- ⑥ **HAVING**: 用于过滤分组后的结果集。
- ⑦ **LIMIT**: 限制返回的行数。
- ⑧ **OFFSET**: 用于跳过结果集中的前几行。

1.查询所有列

SELECT * FROM students; --该查询将返回 students 表中的所有列和所有行。

The screenshot shows the pgAdmin 4 interface. The query editor contains the SQL statement `SELECT * FROM students;`. The results pane displays the data from the `students` table. Red annotations highlight key features: (1) '当前数据库' (Current Database) points to the database connection; (2) '*代表所有列' (* represents all columns) points to the asterisk in the query; (3) '查询结果' (Query Results) points to the results pane.

Query: `SELECT * FROM students;`

Data Output:

	student_id [PK] integer	student_number character varying (20)	student_name character varying (50)	gender character (1)	class_id integer
1	1	20230001	张三	M	1
2	2	20230002	李四	F	1
3	3	20230003	王五	M	1
4	4	20230004	赵六	F	2

Servers > PostgreSQL 17 > Databases > teachdb

2.查询特定列

SELECT student_number, student_name **FROM** students;

-- 该查询将返回 students 表中的 student_number 和 student_name 列。

The screenshot shows the pgAdmin 4 interface. At the top, the title bar says 'pgAdmin 4'. Below it is a menu bar with 'File', 'Object', 'Tools', 'Edit', 'View', 'Window', and 'Help'. The main window has a tab labeled 'Welcome teachdb/postgres@PostgreSQL 17* x'. Below the tab is a toolbar with various icons. The 'Query' tab is selected, and the query editor contains the following SQL statement:

```
1 SELECT student_number, student_name FROM students;
```

Below the query editor is the 'Data Output' tab, which shows the results of the query. The results are displayed in a table with two columns: 'student_number' and 'student_name'. The table has 5 rows of data. The first row is highlighted. The table structure is as follows:

	student_number character varying (20)	student_name character varying (50)
1	20230001	张三
2	20230002	李四
3	20230003	王五
4	20230004	赵六
5	20230005	钱七

At the bottom of the interface, the status bar shows 'Servers > PostgreSQL 17 > Databases > teachdb', 'CRLF', and 'Ln 1, Col 1'.

Annotations in the image:

- 1: 当前数据库 (Current Database) - points to the database name in the tab.
- 2: 查询指定的列 (Columns specified in the query) - points to the column names in the SQL statement.
- 3: 查询结果 (Query Results) - points to the table of results.

3.使用 WHERE 子句过滤结果

SELECT * FROM students WHERE gender = 'M';

--该查询将返回所有性别为男性的学生记录。

The screenshot shows the pgAdmin 4 web interface. At the top, the 'Query' tab is active, displaying the SQL query: `SELECT * FROM students WHERE gender = 'M';`. A red arrow points to the `WHERE` clause, labeled '查询条件' (Query Condition). Below the query, the 'Data Output' tab is active, showing a table of results. A red arrow points to the table header, labeled '查询结果' (Query Result). The table has 6 columns: `student_id` (PK Integer), `student_number` (character varying (20)), `student_name` (character varying (50)), `gender` (character (1)), and `class_id` (integer). The results show 5 rows of male students.

	student_id [PK] integer	student_number character varying (20)	student_name character varying (50)	gender character (1)	class_id integer
1	1	20230001	张三	M	1
2	3	20230003	王五	M	1
3	5	20230005	钱七	M	2
4	7	20230007	周九	M	1
5	9	20230009	郑十一	M	2

Showing rows: 1 to 10 | Page No: 1 of 1

Servers > PostgreSQL 17 > Databases > teachdb | CRLF | Ln 1, Col 1

4.使用 ORDER BY 排序结果

SELECT * FROM students ORDER BY student_name ASC;

--该查询将按学生姓名的升序排列结果。

The screenshot shows the pgAdmin 4 interface. The 'Query' tab is active, displaying the SQL query: `SELECT * FROM students ORDER BY student_name ASC;`. The 'Data Output' tab is also visible, showing the results of the query. The results are sorted by student name in ascending order. The interface includes a menu bar, a toolbar, and a status bar at the bottom.

Annotations in the image:

- 1: 当前数据库 (Current Database)
- 2: 按照姓名排序 (Sort by name)
- 3: 升序 (Ascending)
- 4: 查询结果 (Query Results)

	student_id [PK] integer	student_number character varying (20)	student_name character varying (50)	gender character (1)	class_id integer
1	11	20230011	陈小明	M	1
2	10	20230010	冯十二	F	3
3	20	20230020	冯雪	F	3
4	12	20230012	李华	F	1
5	2	20230002	李四	F	1

Servers > PostgreSQL 17 > Databases > teachdb CRLF Ln 1, Col 1

4.使用 ORDER BY 排序结果

SELECT * FROM students ORDER BY class_id DESC;

--该查询将按班级 ID 的降序排列结果。

The screenshot shows the pgAdmin 4 interface with the following components:

- Current Database:** teachdb/postgres@PostgreSQL 17*
- Query Editor:** Contains the SQL query: `SELECT * FROM students ORDER BY class_id DESC;`
- Data Output:** Displays the results of the query in a table format.
- Annotations:** Four red arrows with numbers 1 through 4 point to specific elements: 1 points to the 'Current Database' dropdown, 2 points to the 'ORDER BY' clause in the query, 3 points to the 'DESC' keyword, and 4 points to the 'Query Results' tab.

	student_id [PK] integer	student_number character varying (20)	student_name character varying (50)	gender character (1)	class_id integer
1	20	20230020	冯雪	F	3
2	15	20230015	钱磊	M	3
3	10	20230010	冯十二	F	3
4	16	20230016	孙丽	F	3
5	19	20230019	郑强	M	3
6	4	20230004	赵六	F	2
7	13	20230013	王伟	M	2

Servers > PostgreSQL 17 > Databases > teachdb CRLF Ln 1, Col 1

5.使用 LIMIT 限制返回的行数

SELECT * FROM students LIMIT 5; --该查询将返回 students 表中的前 5 行记录。

The screenshot shows the pgAdmin 4 web interface. At the top, a red circle with the number 1 points to the '当前数据库' (Current Database) label. Below it, the 'Query' tab is active, and a red circle with the number 2 points to the 'LIMIT 5' in the SQL query: `SELECT * FROM students LIMIT 5;`. A tooltip '限定前5条记录' (Limit first 5 records) is visible. Below the query editor, the 'Data Output' tab shows the results. A red circle with the number 3 points to the '查询结果' (Query Results) label. The results are displayed in a table with 6 columns: student_id, student_number, student_name, gender, and class_id. The table shows 5 rows of data.

	student_id [PK] integer	student_number character varying (20)	student_name character varying (50)	gender character (1)	class_id integer
1	1	20230001	张三	M	1
2	2	20230002	李四	F	1
3	3	20230003	王五	M	1
4	4	20230004	赵六	F	2
5	5	20230005	钱七	M	2

6.使用 OFFSET 跳过前几行

SELECT * FROM students LIMIT 5 OFFSET 2; -- 该查询将跳过前 2 行，返回接下来的 5 行记录。

The screenshot shows the pgAdmin 4 interface with the following components:

- 1 当前数据库**: Points to the 'teachdb/postgres@PostgreSQL 17*' connection in the top toolbar.
- 2 跳过前2条后**: Points to the 'LIMIT 5 OFFSET 2;' part of the SQL query in the 'Query' tab.
- 3 返回接下来的5条记录**: Points to the 'LIMIT 5' part of the SQL query.
- 4 查询结果**: Points to the 'Data Output' tab showing the results of the query.

The SQL query entered is:

```
1 SELECT * FROM students LIMIT 5 OFFSET 2;
```

The results are displayed in a table with the following columns: student_id [PK] integer, student_number character varying (20), student_name character varying (50), gender character (1), and class_id integer.

	student_id [PK] integer	student_number character varying (20)	student_name character varying (50)	gender character (1)	class_id integer
1	3	20230003	王五	M	1
2	4	20230004	赵六	F	2
3	5	20230005	钱七	M	2
4	6	20230006	孙八	F	1
5	7	20230007	周九	M	1

The bottom status bar shows: Servers > PostgreSQL 17 > Databases > teachdb, CRLF, Ln 1, Col 1.

7.使用 GROUP BY 分组结果

该查询将按班级 ID 分组，并返回每个班级的学生数量。

SELECT class_id, **COUNT(*)** AS student_count

FROM students

GROUP BY class_id;

The screenshot shows the pgAdmin 4 interface with a SQL query entered in the Query editor. The query is: `SELECT class_id, COUNT(*) AS student_count FROM students GROUP BY class_id;`. The interface includes a menu bar (File, Object, Tools, Edit, View, Window, Help), a toolbar with various icons, and a status bar at the bottom. The query results are displayed in a table with two columns: `class_id` (integer) and `student_count` (bigint). The results show three rows: class_id 3 with student_count 5, class_id 2 with student_count 7, and class_id 1 with student_count 8. Red arrows and callouts highlight key parts of the interface: 1. '当前数据库' (Current Database) pointing to the database name in the toolbar. 2. '统计记录数' (Count records) pointing to the `COUNT(*)` function in the query. 3. '按照班级ID分组统计' (Group and count by class ID) pointing to the `GROUP BY class_id;` clause. 4. '统计结果列的别名' (Alias for the result column) pointing to the `AS student_count` alias. 5. '查询结果' (Query results) pointing to the table of results.

	class_id integer	student_count bigint
1	3	5
2	2	7
3	1	8

8.使用 HAVING 过滤分组后的结果

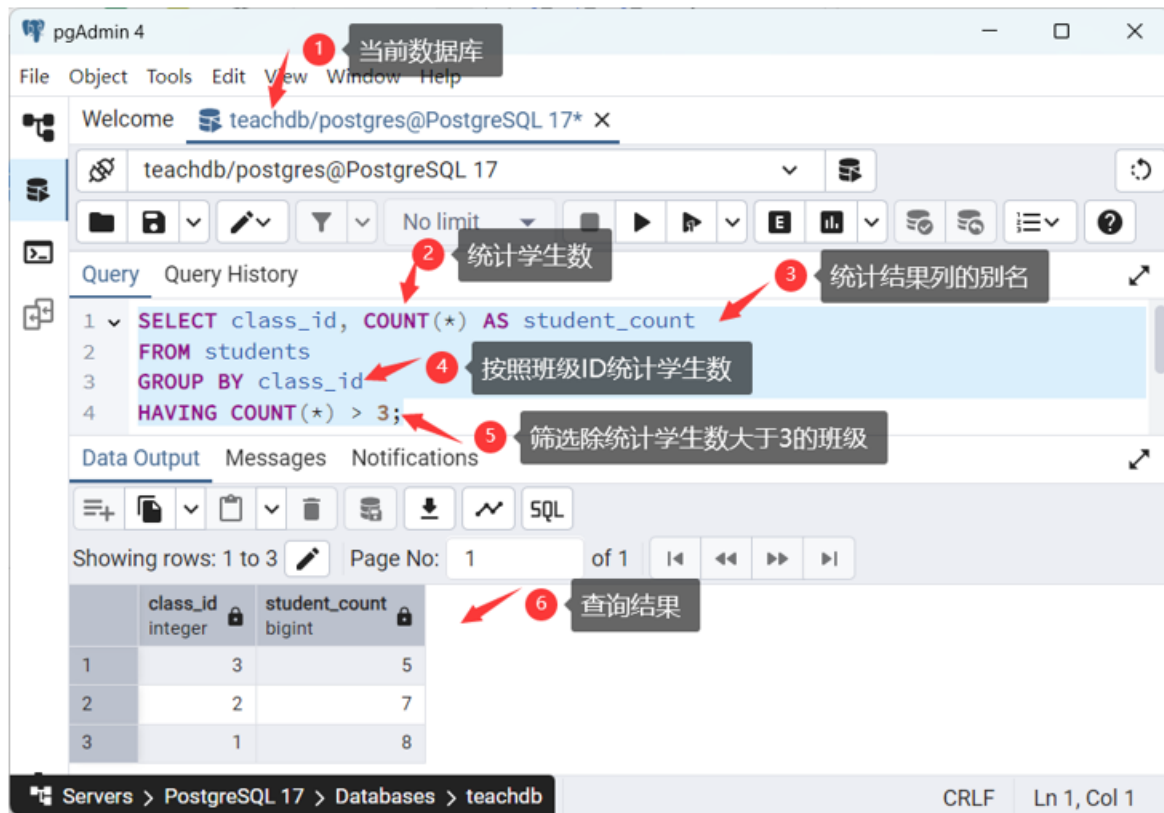
该查询将返回学生数量大于 3 的班级 ID 和对应的学生数量。

SELECT class_id, **COUNT(*)** AS student_count

FROM students

GROUP BY class_id

HAVING COUNT(*) > 3;



pgAdmin 4

File Object Tools Edit View Window Help

Welcome teachdb/postgres@PostgreSQL 17* x

teachdb/postgres@PostgreSQL 17

No limit

Query Query History

```
1 SELECT class_id, COUNT(*) AS student_count
2 FROM students
3 GROUP BY class_id
4 HAVING COUNT(*) > 3;
```

Data Output Messages Notifications

Showing rows: 1 to 3 Page No: 1 of 1

	class_id integer	student_count bigint
1	3	5
2	2	7
3	1	8

Servers > PostgreSQL 17 > Databases > teachdb

CRLF Ln 1, Col 1

9.使用 JOIN 连接多个表

该查询将返回每个学生的姓名及其所选课程的名称。

```
SELECT s.student_name, c.course_name
```

```
FROM students s
```

```
JOIN enrollments e ON s.student_id = e.student_id
```

```
JOIN course_classes cc ON e.course_class_id = cc.course_class_id
```

```
JOIN courses c ON cc.course_id = c.course_id;
```

The screenshot shows the pgAdmin 4 interface with a SQL query entered in the Query editor. The query is a multi-table JOIN query. Red arrows and callouts highlight specific parts of the query and the interface:

- 1. 当前数据库 (Current Database): Points to the database name in the top bar.
- 2. 学生表 (Students Table): Points to the `students` table in the FROM clause.
- 3. 按学生ID连接选课表 (Join course table by student ID): Points to the JOIN condition `s.student_id = e.student_id`.
- 4. 按课程班ID连接课程班表 (Join course class table by course class ID): Points to the JOIN condition `e.course_class_id = cc.course_class_id`.
- 5. 按课程ID连接课程表 (Join course table by course ID): Points to the JOIN condition `cc.course_id = c.course_id`.
- 6. 查询结果 (Query Results): Points to the table of results at the bottom.

The query results table shows the following data:

	student_name character varying (50)	course_name character varying (50)
1	张三	数据库系统
2	李四	数据库系统
3	王五	操作系统
4	赵六	计算机网络
5	钱七	操作系统

9.使用 JOIN 连接多个表（另外一种写法）

该查询将返回每个学生的姓名及其所选课程
的名称，也可以使用下列查询：

```
SELECT s.student_name, c.course_name
```

```
FROM students s,
```

```
enrollments e,
```

```
course_classes cc,
```

```
courses c
```

```
where s.student_id = e.student_id and
```

```
e.course_class_id = cc.course_class_id and
```

```
cc.course_id = c.course_id;
```

The screenshot shows the pgAdmin 4 interface with the following components:

- 1 当前数据库**: Points to the database connection 'teachdb/postgres@PostgreSQL 17* X'.
- 2 多表之间的连接**: Points to the FROM clause in the SQL query.
- 3 连接条件**: Points to the WHERE clause in the SQL query.
- 4 查询结果**: Points to the results table.

SQL Query:

```
SELECT s.student_name, c.course_name
FROM students s, enrollments e, course_classes cc, courses c
where s.student_id = e.student_id and
e.course_class_id = cc.course_class_id and
cc.course_id = c.course_id;
```

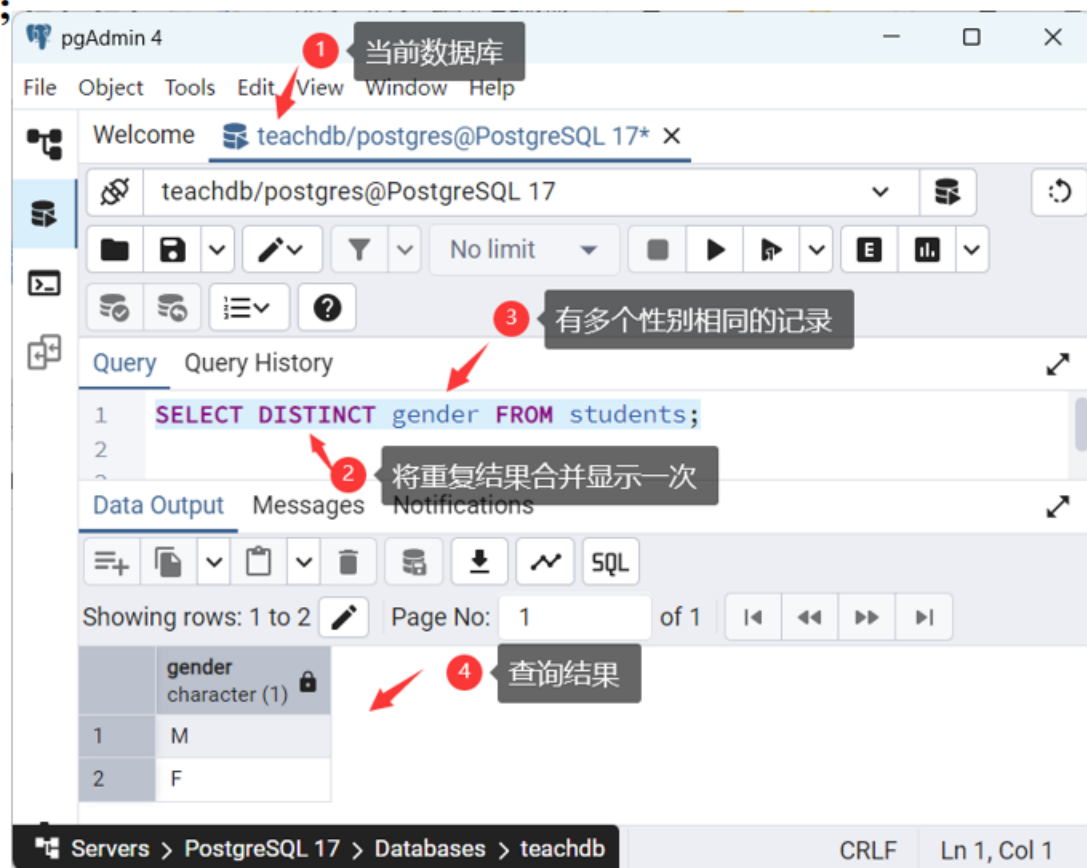
Data Output:

	student_name character varying (50)	course_name character varying (50)
1	张三	数据库系统
2	李四	数据库系统
3	王五	操作系统
4	赵六	计算机网络
5	钱七	操作系统

10.使用 DISTINCT 去重

SELECT DISTINCT gender **FROM** students;

--该查询返回 students 表中所有不同的性别。



11. 使用子查询

① 子查询返回单个值

SELECT student_name

FROM students

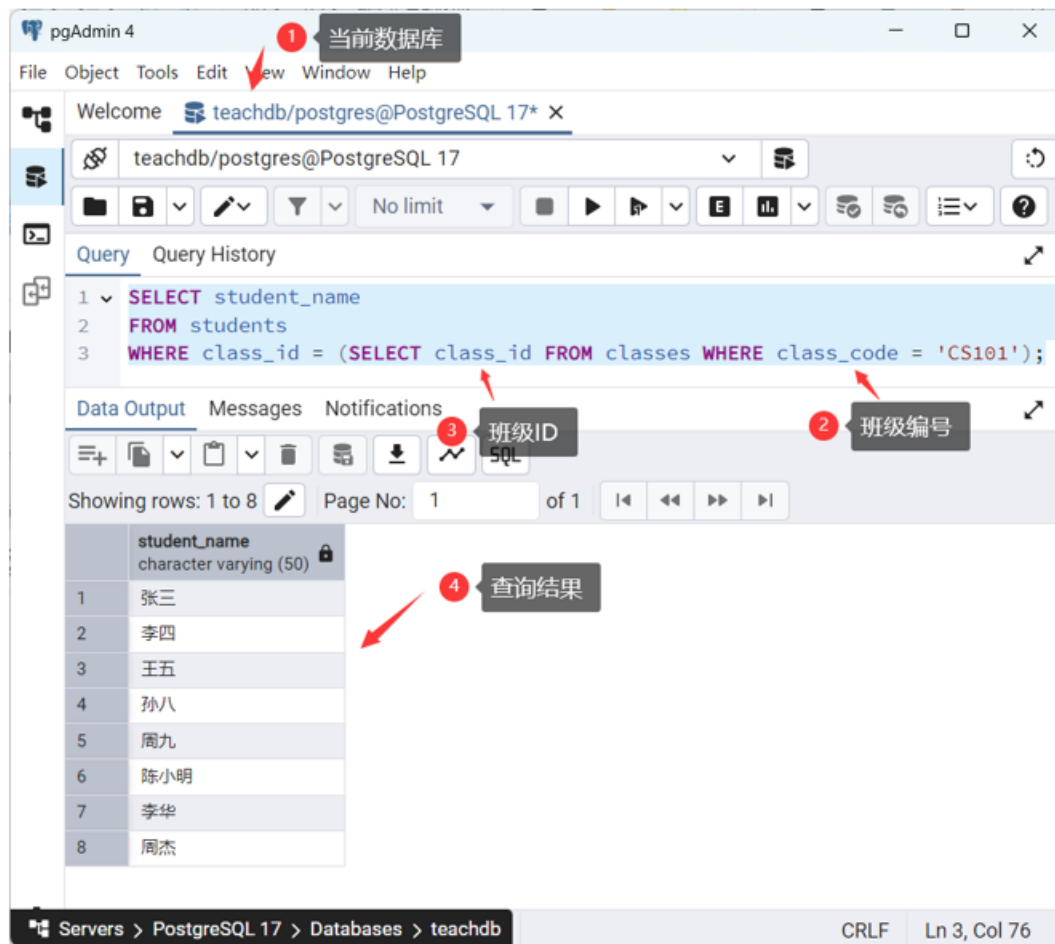
WHERE class_id = (

SELECT class_id

FROM classes

WHERE class_code = 'CS101');

--该查询将返回在 CS101 班级中的所有
学生姓名。



11. 使用子查询

② 子查询返回多个值

SELECT student_name

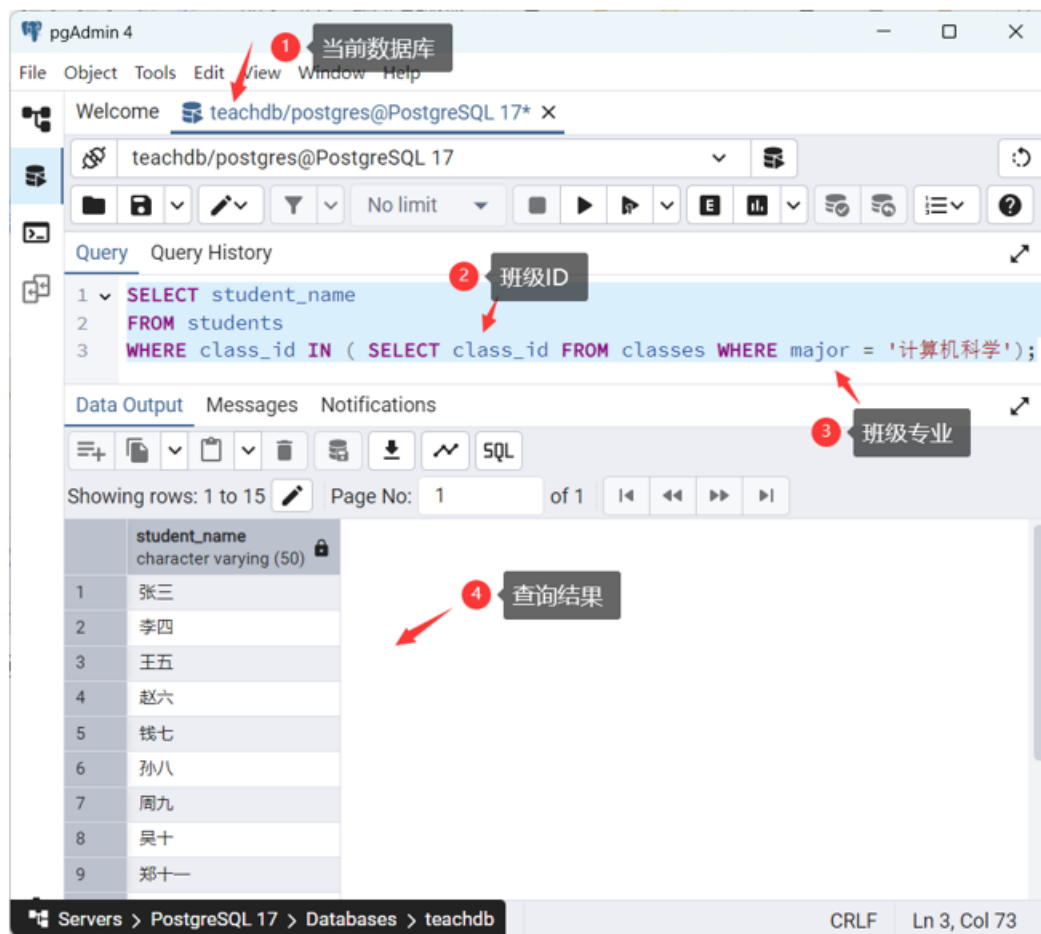
FROM students

WHERE class_id **IN** (

SELECT class_id **FROM** classes

WHERE major = '计算机科学');

--该查询将返回所有计算机科学专业的
学生姓名。



12. UNION 合并多个查询结果

```
SELECT student_name AS name, '学生' AS role  
FROM students
```

UNION

```
SELECT teacher_name AS name, '教师' AS role  
FROM teachers;
```

--该查询将返回一个包含所有学生和教师姓名的列表，并标识他们的角色。

pgAdmin 4

File Object Tools Edit View Window Help

Welcome teachdb/postgres@PostgreSQL 17*

teachdb/postgres@PostgreSQL 17

No limit

Query Query History

```
1 SELECT student_name AS name, '学生' AS role
2 FROM students
3 UNION
4 SELECT teacher_name AS name, '教师' AS role
5 FROM teachers;
```

Data Output Messages Notifications

Showing rows: 1 to 31 Page No: 1 of 1

	name character varying (50)	role text
8	陈小明	学生
9	孙丽	学生
10	吴十	学生
11	陈静	教师
12	钱磊	学生
13	刘老师	教师
14	王老师	教师
15	张三	学生
16	张宏	教师
17	钱七	学生

Servers > PostgreSQL 17 > Databases > teachdb

CRLF Ln 1, Col 1

13. 使用窗口函数

查询每个学生的学号、姓名、课程班级 ID、教师姓名、成绩及其在各自课程班级中的排名。

```
SELECT s.student_number, s.student_name, cc.course_class_id, t.teacher_name, e.score,  
       RANK() OVER (PARTITION BY cc.course_class_id ORDER BY e.score DESC) AS rank  
FROM students s  
JOIN enrollments e ON s.student_id = e.student_id  
JOIN course_classes cc ON e.course_class_id = cc.course_class_id  
JOIN teachers t ON cc.teacher_id = t.teacher_id  
ORDER BY cc.course_class_id, rank;
```

13. 使用窗口函数（续）

解释： **SELECT** 子句：

s.student_number：选择学生的学号。

s.student_name：选择学生的姓名。

cc.course_class_id：选择课程班级的 ID。

t.teacher_name：选择授课教师的姓名。

e.score：选择学生的成绩。

RANK() OVER (PARTITION BY cc.course_class_id ORDER BY e.score DESC) AS rank：

使用窗口函数计算每个学生在其课程班级中的排名，按成绩降序排列。

FROM 子句：

从 **students** 表开始，使用别名 **s**。

连接 **enrollments** 表，以获取学生的成绩，使用别名 **e**。

连接 **course_classes** 表，以获取课程班级信息，使用别名 **cc**。

连接 **teachers** 表，以获取授课教师的姓名，使用别名 **t**。

ORDER BY 子句：按课程班级 ID 和排名排序，以便结果更易于阅读。

13. 使用窗口函数（上述示例的运行结果）

pgAdmin 4

Welcome teachdb/postgres@PostgreSQL 17* X

teachdb/postgres@PostgreSQL 17

Query Query History

```
1 SELECT s.student_number, s.student_name, cc.course_class_id, t.teacher_name, e.score,
2       RANK() OVER (PARTITION BY cc.course_class_id ORDER BY e.score DESC) AS rank
3 FROM students s
4 JOIN enrollments e ON s.student_id = e.student_id
5 JOIN course_classes cc ON e.course_class_id = cc.course_class_id
6 JOIN teachers t ON cc.teacher_id = t.teacher_id
7 ORDER BY cc.course_class_id, rank;
```

Data Output Messages Notifications

Showing rows: 1 to 20 Page No: 1 of 1

	student_number character varying (20)	student_name character varying (50)	course_class_id integer	teacher_name character varying (50)	score numeric (5,2)	rank bigint
1	20230012	李华	1	刘老师	91.00	1
2	20230002	李四	1	刘老师	90.00	2
3	20230011	陈小明	1	刘老师	88.00	3
4	20230001	张三	1	刘老师	85.50	4
5	20230015	钱磊	2	陈老师	94.00	1
6	20230005	钱七	2	陈老师	92.00	2
7	20230013	王伟	2	陈老师	79.00	3
8	20230003	王五	2	陈老师	78.00	4
9	20230004	赵六	3	王老师	88.00	1

Total rows: 20 Query complete 00:00:00.051 CRLF Ln 1, Col 1

14. 使用 CASE 语句进行条件计算

编写SQL语句查询将根据学生的成绩将其分类为“优秀”、“良好”、“中等”或“及格”，显示学生姓名、课程姓名，成绩，成绩分类等；可以使用 CASE 语句来实现：

```
SELECT s.student_name, c.course_name, e.score,  
       CASE  
         WHEN e.score >= 90 THEN '优秀'  
         WHEN e.score >= 80 THEN '良好'  
         WHEN e.score >= 70 THEN '中等'  
         ELSE '及格'  
       END AS grade_category  
FROM students s  
JOIN enrollments e ON s.student_id = e.student_id  
JOIN course_classes cc ON e.course_class_id = cc.course_class_id  
JOIN courses c ON cc.course_id = c.course_id  
ORDER BY s.student_name, c.course_name;
```

14. 使用 CASE 语句进行条件计算（续）

解释：SELECT 子句：

s.student_name：选择学生的姓名。

c.course_name：选择课程的名称。

e.score：选择学生的成绩。

CASE 语句用于根据成绩分类：

WHEN e.score >= 90 THEN '优秀'：成绩大于等于 90 的学生被标记为“优秀”。

WHEN e.score >= 80 THEN '良好'：成绩大于等于 80 的学生被标记为“良好”。

WHEN e.score >= 70 THEN '中等'：成绩大于等于 70 的学生被标记为“中等”。

ELSE '及格'：其他成绩被标记为“及格”。

FROM 子句：

从 students 表开始，使用别名 s。

连接 enrollments 表，以获取学生的成绩，使用别名 e。

连接 course_classes 表，以获取课程班级信息，使用别名 cc。

连接 courses 表，以获取课程名称，使用别名 c。

ORDER BY 子句：按学生姓名和课程名称排序，以便结果更易于阅读。

14. 使用 CASE 语句进行条件计算（示例运行结果）

The screenshot shows the pgAdmin 4 interface with a SQL query executed in the 'Query' tab. The query uses a CASE statement to categorize student scores into '优秀' (Excellent), '良好' (Good), '中等' (Medium), and '及格' (Passing). The 'Data Output' tab shows the results of the query as a table with 6 rows and 4 columns: student_name, course_name, score, and grade_category.

```
SELECT s.student_name, c.course_name, e.score,
CASE
    WHEN e.score >= 90 THEN '优秀'
    WHEN e.score >= 80 THEN '良好'
    WHEN e.score >= 70 THEN '中等'
    ELSE '及格'
END AS grade_category
FROM students s
JOIN enrollments e ON s.student_id = e.student_id
JOIN course_classes cc ON e.course_class_id = cc.course_class_id
JOIN courses c ON cc.course_id = c.course_id
ORDER BY s.student_name, c.course_name;
```

	student_name character varying (50)	course_name character varying (50)	score numeric (5,2)	grade_category text
1	陈小明	数据库系统	88.00	良好
2	冯十二	人工智能	95.00	优秀
3	冯雪	算法分析	92.00	优秀
4	李华	数据库系统	91.00	优秀
5	李四	数据库系统	90.00	优秀
6	钱磊	操作系统	94.00	优秀

Total rows: 20 Query complete 00:00:00.064 CRLF Ln 12, Col 40

15. 使用公共表表达式（CTE）查询计算

编写SQL语句使用公共表表达式（CTE）计算每个课程班的平均成绩，并返回班级编号、教师姓名、课程名称和平均成绩：

```
WITH AverageScores AS (  
    SELECT cc.course_class_id,cc.teacher_id,cc.course_id, AVG(e.score) AS average_score  
    FROM enrollments e  
    JOIN course_classes cc ON e.course_class_id = cc.course_class_id  
    GROUP BY cc.course_class_id, cc.teacher_id, cc.course_id  
)  
  
SELECT ag.course_class_id,t.teacher_name,c.course_name,ag.average_score  
FROM AverageScores ag  
JOIN teachers t ON ag.teacher_id = t.teacher_id  
JOIN courses c ON ag.course_id = c.course_id  
ORDER BY ag.course_class_id;
```


15. 使用公共表表达式 (CTE) 查询计算 (续)

解释: **CTE 部分 (WITH AverageGrades AS (...))**:

计算每个课程班的平均成绩。

cc.course_class_id: 选择课程班级 ID。

cc.teacher_id: 选择教师 ID。

cc.course_id: 选择课程 ID。

AVG(e.grade) AS average_score: 计算每个课程班的平均成绩。

使用 **JOIN** 将 **enrollments** 表与 **course_classes** 表连接。

使用 **GROUP BY** 按课程班级 ID、教师 ID 和课程 ID 分组。

主查询部分:

从 **CTE AverageScores** 中选择班级编号、教师姓名、课程名称和平均成绩。

使用 **JOIN** 将 **AverageScores** 表与 **teachers** 表连接, 以获取教师姓名。

使用 **JOIN** 将 **AverageScores** 表与 **courses** 表连接, 以获取课程名称。

ORDER BY 子句: 按课程班级 ID 排序, 以便结果更易于阅读。

15. 使用公共表表达式 (CTE) 查询计算 (示例运行结果)

pgAdmin 4 当前数据库

File Object Tools Edit View Window Help

Welcome teachdb/postgres@PostgreSQL 17* x

teachdb/postgres@PostgreSQL 17

No limit

Query Query History 公共表达式名

```
1 WITH AverageScores AS (  
2     SELECT cc.course_class_id, cc.teacher_id, cc.course_id, AVG(e.score) AS average_score  
3     FROM enrollments e  
4     JOIN course_classes cc ON e.course_class_id = cc.course_class_id  
5     GROUP BY cc.course_class_id, cc.teacher_id, cc.course_id  
6 )  
7  
8 SELECT ag.course_class_id, t.teacher_name, c.course_name, ag.average_score  
9 FROM AverageScores ag  
10 JOIN teachers t ON ag.teacher_id = t.teacher_id  
11 JOIN courses c ON ag.course_id = c.course_id  
12 ORDER BY ag.course_class_id;
```

将公共表达式的查询结果作为表来做查询

Data Output Messages Notifications

Showing rows: 1 to 7 Page No: 1 of 1

	course_class_id integer	teacher_name character varying (50)	course_name character varying (50)	average_score numeric
1	1	刘老师	数据库系统	88.6250000000000000
2	2	陈老师	操作系统	85.7500000000000000
3	3	王老师	计算机网络	86.5000000000000000
4	4	赵老师	软件工程	87.7500000000000000
5	5	钱老师	人工智能	82.0000000000000000
6	6	刘洋	数据结构	81.0000000000000000

查询计算结果

Total rows: 7 Query complete 00:00:00.073 CRLF Ln 1, Col 1

16、外连接（Outer Join）

外连接（Outer Join）是 SQL 中的一种连接操作，用于从两个或多个表中检索数据，**即使在某些表中没有匹配的记录**。外连接有三种主要类型：**左外连接（LEFT JOIN）、右外连接（RIGHT JOIN）和全外连接（FULL OUTER JOIN）**。

(1) 左外连接 (LEFT JOIN)

左外连接返回左表（第一个表）中的所有记录，以及右表（第二个表）中匹配的记录。如果右表中没有匹配的记录，则结果中右表的字段将返回 NULL。语法格式：

SELECT columns

FROM left_table

LEFT JOIN right_table **ON** left_table.column = right_table.column;

(1) 左外连接 (LEFT JOIN)

例如：编写SQL语句使用左外连接查询所有老师的排课情况，即使某些课程没有排课，包括老师姓名、课程名、开课学期及学时数。

```
SELECT t.teacher_name, c.course_name, cc.semester, c.hours  
FROM courses c  
LEFT JOIN course_classes cc ON cc.course_id = c.course_id  
LEFT JOIN teachers t ON t.teacher_id = cc.teacher_id  
ORDER BY t.teacher_name;
```

(1) 左外连接 (LEFT JOIN)

解释: SELECT 子句:

t.teacher_name: 选择教师的姓名。

c.course_name: 选择课程的名称。

cc.semester: 选择开课的学期。

c.hours: 选择课程的学时数。

FROM 子句: 从 courses 表开始, 使用别名 c。

LEFT JOIN 子句:

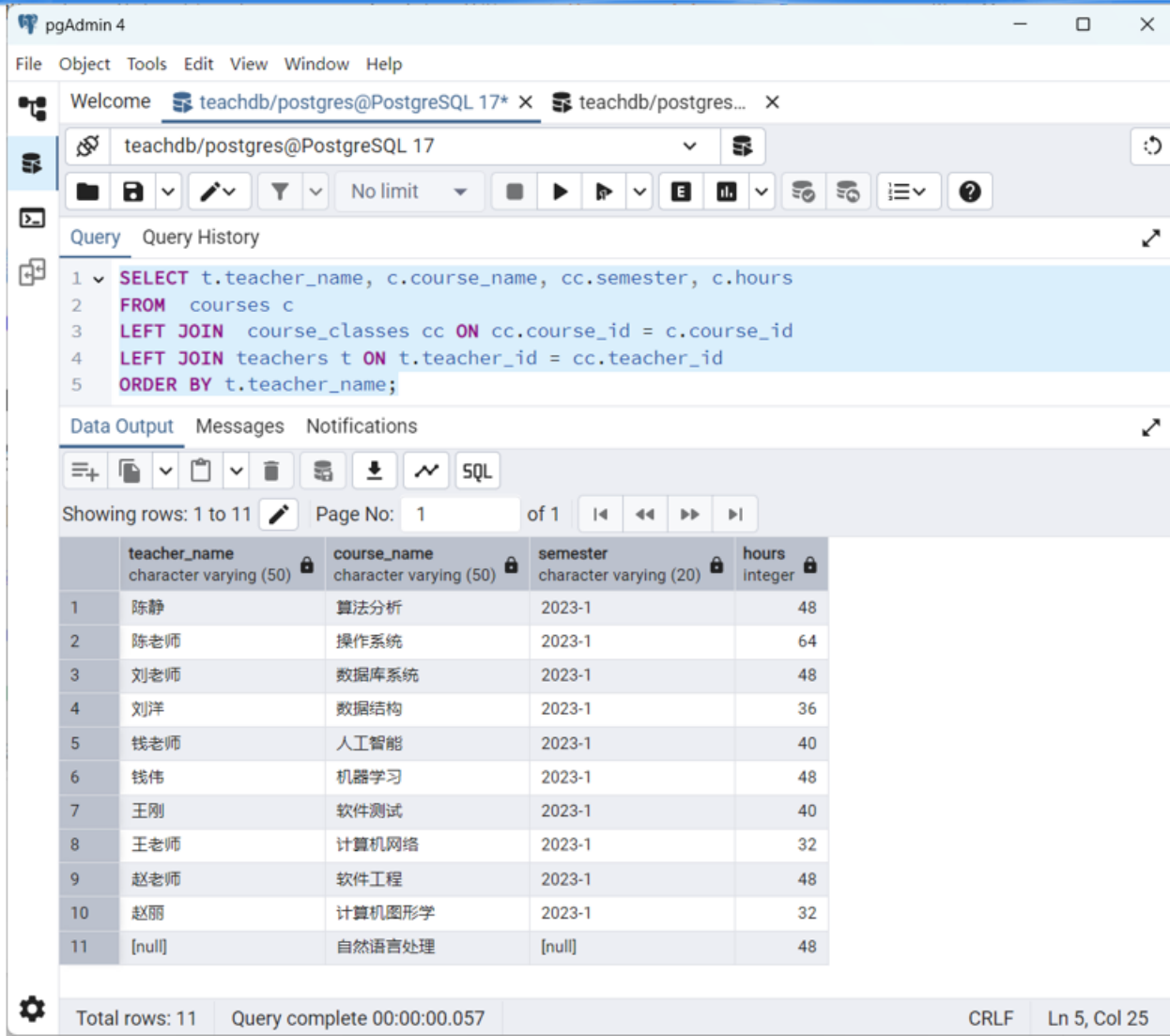
LEFT JOIN course_classes cc ON cc.course_id = c.course_id 连接课程班表, 获取开课学期。

LEFT JOIN teachers t ON t.teacher_id = cc.teacher_id 连接教师表, 以获取教师姓名。

ORDER BY 子句: 按教师姓名排序, 以便结果更易于阅读。

(1) 左外连接 (LEFT JOIN)

例如：编写SQL语句
使用左外连接查询所有老师的排课情况，
即使某些课程没有排
课，包括老师姓名、
课程名、开课学期及
学时数。(运行结果)



The screenshot shows the pgAdmin 4 interface. The query editor displays the following SQL query:

```
1 SELECT t.teacher_name, c.course_name, cc.semester, c.hours
2 FROM courses c
3 LEFT JOIN course_classes cc ON cc.course_id = c.course_id
4 LEFT JOIN teachers t ON t.teacher_id = cc.teacher_id
5 ORDER BY t.teacher_name;
```

The query results are displayed in a table with the following columns: teacher_name, course_name, semester, and hours. The results show 11 rows of data, including teachers and their associated courses and semesters.

	teacher_name character varying (50)	course_name character varying (50)	semester character varying (20)	hours integer
1	陈静	算法分析	2023-1	48
2	陈老师	操作系统	2023-1	64
3	刘老师	数据库系统	2023-1	48
4	刘洋	数据结构	2023-1	36
5	钱老师	人工智能	2023-1	40
6	钱伟	机器学习	2023-1	48
7	王刚	软件测试	2023-1	40
8	王老师	计算机网络	2023-1	32
9	赵老师	软件工程	2023-1	48
10	赵丽	计算机图形学	2023-1	32
11	[null]	自然语言处理	[null]	48

The status bar at the bottom indicates: Total rows: 11, Query complete 00:00:00.057, CRLF, Ln 5, Col 25.

(2) 右外连接 (RIGHT JOIN)

右外连接返回右表中的所有记录，以及左表中匹配的记录。如果左表中没有匹配的记录，则结果中左表的字段将返回 NULL。

SELECT columns

FROM left_table

RIGHT JOIN right_table **ON** left_table.column = right_table.column;

(2) 右外连接 (RIGHT JOIN)

编写SQL语句使用右外连接查询所有老师的排课情况，即使某些老师没有排课，包括老师姓名、课程名、开课学期及学时数。

```
SELECT t.teacher_name, c.course_name, cc.semester, c.hours  
FROM courses c  
right JOIN course_classes cc ON cc.course_id = c.course_id  
right JOIN teachers t ON t.teacher_id = cc.teacher_id  
ORDER BY t.teacher_name;
```


(2) 右外连接 (RIGHT JOIN)

编写SQL语句使用右外连接查询所有老师的排课情况，即使某些老师没有排课，包括老师姓名、课程名、开课学期及学时数。

(运行结果)

The screenshot shows the pgAdmin 4 interface. The SQL query editor contains the following query:

```
1 SELECT t.teacher_name, c.course_name, cc.semester, c.hours
2 FROM courses c
3 right JOIN course_classes cc ON cc.course_id = c.course_id
4 right JOIN teachers t ON t.teacher_id = cc.teacher_id
5 ORDER BY t.teacher_name;
```

The Data Output tab shows the results of the query, displaying 11 rows. The columns are teacher_name, course_name, semester, and hours.

	teacher_name	course_name	semester	hours
1	陈静	算法分析	2023-1	48
2	陈老师	操作系统	2023-1	64
3	刘老师	数据库系统	2023-1	48
4	刘洋	数据结构	2023-1	36
5	钱老师	人工智能	2023-1	40
6	钱伟	机器学习	2023-1	48
7	王刚	软件测试	2023-1	40
8	王老师	计算机网络	2023-1	32
9	张宏	[null]	[null]	[null]
10	赵老师	软件工程	2023-1	48
11	赵丽	计算机图形学	2023-1	32

The status bar at the bottom indicates: Total rows: 11, Query complete 00:00:00.045, CRLF, Ln 2, Col 7.

(3) 全外连接 (REFT JOIN)

全外连接返回左表和右表中的所有记录。如果在左表中没有匹配的记录，则左表的字段将返回 NULL；如果在右表中没有匹配的记录，则右表的字段将返回 NULL。

SELECT columns

FROM left_table

FULL OUTER JOIN right_table **ON** left_table.column = right_table.column;

(3) 全外连接 (REFT JOIN)

编写SQL语句使用全外连接查询所有老师和所有课程的排课情况，即使某些老师没有排课，或者某些课程没有排课，都需要显示。包括老师姓名、课程名、开课学期及学时数。

```
SELECT t.teacher_name, c.course_name, cc.semester, c.hours  
FROM courses c  
FULL OUTER JOIN course_classes cc ON cc.course_id = c.course_id  
FULL OUTER JOIN teachers t ON t.teacher_id = cc.teacher_id  
ORDER BY t.teacher_name;
```

(3) 全外连接 (LEFT JOIN)

编写SQL语句使用全外连接查询所有老师和所有课程的排课情况，即使某些老师没有排课，或者某些课程没有排课，都需要显示。包括老师姓名、课程名、开课学期及学时数。

(运行情况)

2026年3月11日

The screenshot shows the pgAdmin 4 web interface. The query editor displays a SQL query using a full outer join to combine the 'teachers' and 'courses' tables with the 'course_classes' table. The query is as follows:

```
1 SELECT t.teacher_name, c.course_name, cc.semester, c.hours
2 FROM   courses c
3 FULL OUTER JOIN course_classes cc ON cc.course_id = c.course_id
4 FULL OUTER JOIN teachers t ON t.teacher_id = cc.teacher_id
5 ORDER BY t.teacher_name;
```

The 'Data Output' tab shows the results of the query, displaying 12 rows. The columns are 'teacher_name', 'course_name', 'semester', and 'hours'. The results are as follows:

	teacher_name character varying (50)	course_name character varying (50)	semester character varying (20)	hours integer
1	陈静	算法分析	2023-1	48
2	陈老师	操作系统	2023-1	64
3	刘老师	数据库系统	2023-1	48
4	刘洋	数据结构	2023-1	36
5	钱老师	人工智能	2023-1	40
6	钱伟	机器学习	2023-1	48
7	王刚	软件测试	2023-1	40
8	王老师	计算机网络	2023-1	32
9	张宏	[null]	[null]	[null]
10	赵老师	软件工程	2023-1	48
11	赵丽	计算机图形学	2023-1	32
12	[null]	自然语言处理	[null]	48

The interface also shows the status 'Total rows: 12' and 'Query complete 00:00:00.052'.

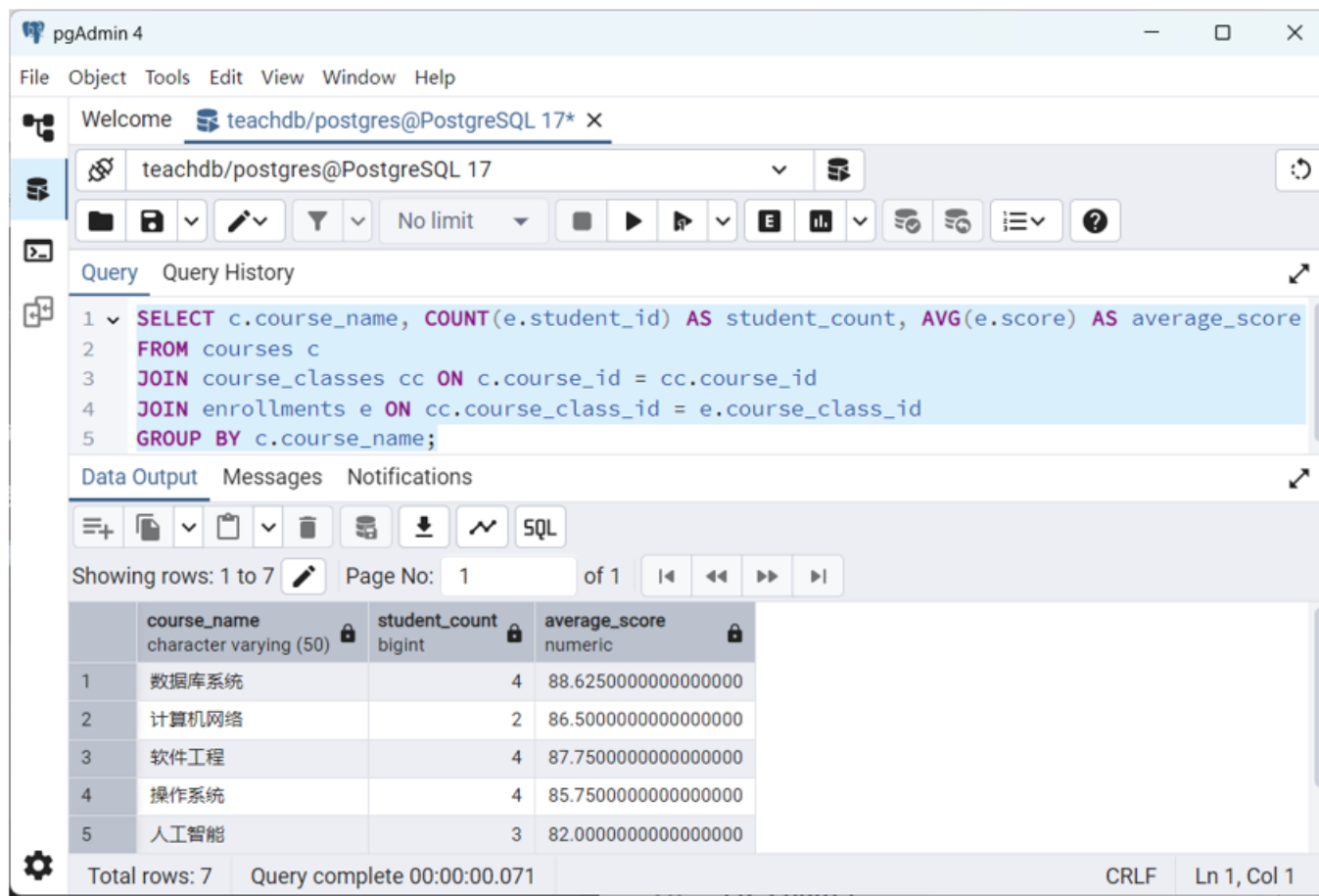
17、较复杂的统计查询示例：

(1) 查询每门课程的学生人数和平均成绩。

```
SELECT c.course_name, COUNT(e.student_id) AS student_count, AVG(e.score) AS average_score
FROM courses c
JOIN course_classes cc ON c.course_id = cc.course_id
JOIN enrollments e ON cc.course_class_id = e.course_class_id
GROUP BY c.course_name;
```

17、较复杂的统计查询示例：

(1) 查询每门课程的学生人数和平均成绩（统计查询运行结果）。



The screenshot shows the pgAdmin 4 interface. The query editor contains the following SQL query:

```
1 SELECT c.course_name, COUNT(e.student_id) AS student_count, AVG(e.score) AS average_score
2 FROM courses c
3 JOIN course_classes cc ON c.course_id = cc.course_id
4 JOIN enrollments e ON cc.course_class_id = e.course_class_id
5 GROUP BY c.course_name;
```

The query results are displayed in the Data Output tab, showing 5 rows of data:

	course_name character varying (50)	student_count bigint	average_score numeric
1	数据库系统	4	88.6250000000000000
2	计算机网络	2	86.5000000000000000
3	软件工程	4	87.7500000000000000
4	操作系统	4	85.7500000000000000
5	人工智能	3	82.0000000000000000

The status bar at the bottom indicates: Total rows: 7, Query complete 00:00:00.071, CRLF, Ln 1, Col 1.

17、较复杂的统计查询示例：

(2) 查询每个学生及其所选课程的详细信息，包括：学生姓名、课程名、老师姓名和开课学期。

```
SELECT s.student_name, c.course_name, t.teacher_name, cc.semester  
FROM students s  
JOIN enrollments e ON s.student_id = e.student_id  
JOIN course_classes cc ON e.course_class_id = cc.course_class_id  
JOIN courses c ON cc.course_id = c.course_id  
JOIN teachers t ON cc.teacher_id = t.teacher_id;
```

17、较复杂的统计查询示例：

(2) 查询每个学生及其所选课程的详细信息（学生姓名、课程名、老师姓名和开课学期）；（运行结果）：

The screenshot shows the pgAdmin 4 interface. The query editor contains the following SQL query:

```
1 SELECT s.student_name, c.course_name, t.teacher_name, cc.semester
2 FROM students s
3 JOIN enrollments e ON s.student_id = e.student_id
4 JOIN course_classes cc ON e.course_class_id = cc.course_class_id
5 JOIN courses c ON cc.course_id = c.course_id
6 JOIN teachers t ON cc.teacher_id = t.teacher_id;
```

The query results are displayed in a table with the following columns: student_name, course_name, teacher_name, and semester. The results show 12 rows of data.

	student_name character varying (50)	course_name character varying (50)	teacher_name character varying (50)	semester character varying (20)
1	张三	数据库系统	刘老师	2023-1
2	李四	数据库系统	刘老师	2023-1
3	王五	操作系统	陈老师	2023-1
4	赵六	计算机网络	王老师	2023-1
5	钱七	操作系统	陈老师	2023-1
6	孙八	软件工程	赵老师	2023-1
7	周九	软件工程	赵老师	2023-1
8	吴十	人工智能	钱老师	2023-1
9	郑十一	数据结构	刘洋	2023-1
10	冯十二	人工智能	钱老师	2023-1
11	陈小明	数据库系统	刘老师	2023-1
12	李华	数据库系统	刘老师	2023-1

The bottom status bar indicates: Total rows: 20, Query complete 00:00:00.077, CRLF, Ln 1, Col 1.

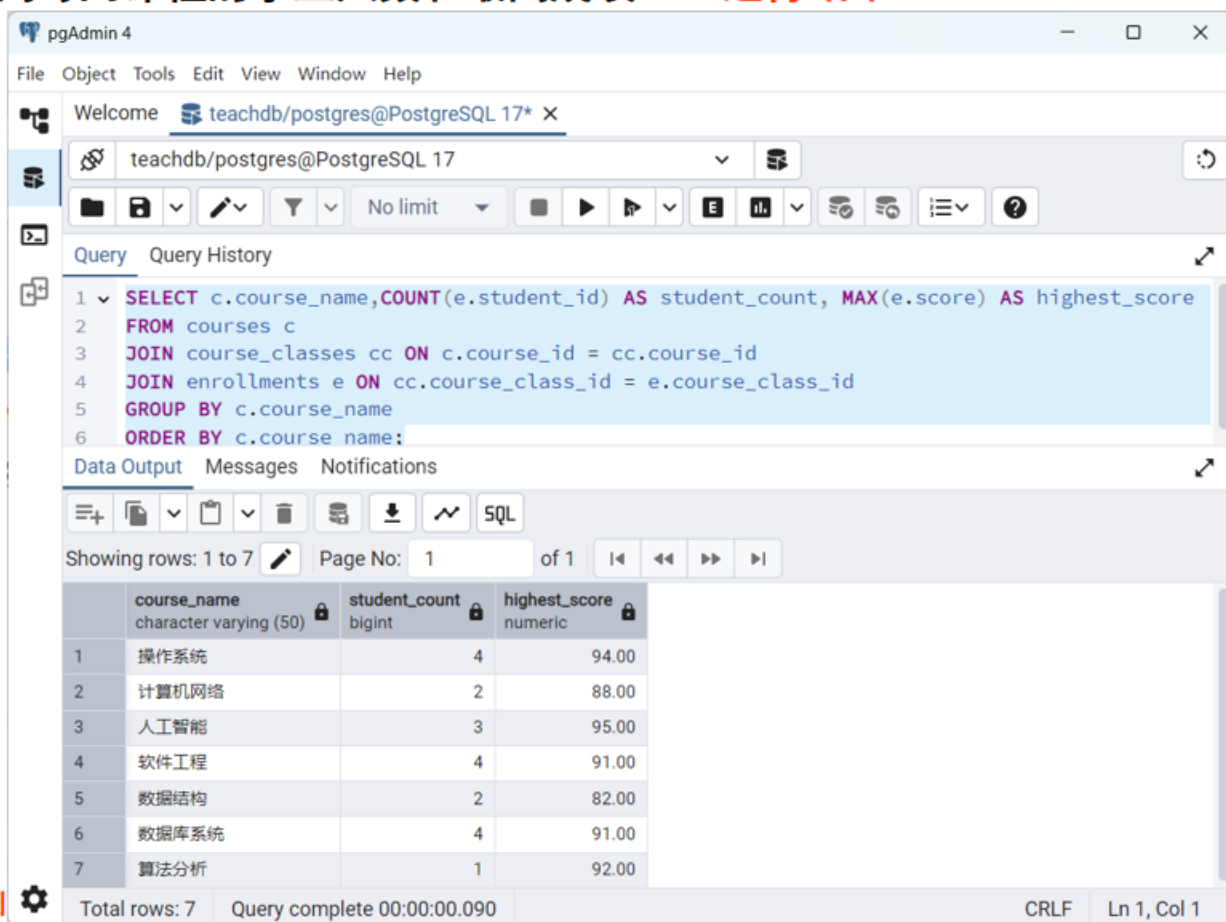
17、较复杂的统计查询示例：

(3) 查询每门课程的学生人数和最高成绩。

```
SELECT c.course_name,COUNT(e.student_id) AS student_count, MAX(e.score) AS highest_score
FROM courses c
JOIN course_classes cc ON c.course_id = cc.course_id
JOIN enrollments e ON cc.course_class_id = e.course_class_id
GROUP BY c.course_name
ORDER BY c.course_name;
```

17、较复杂的统计查询示例：

(3) 查询每门课程的学生人数和最高成绩。（运行结果）



The screenshot shows the pgAdmin 4 interface. The query editor contains the following SQL query:

```
1 SELECT c.course_name, COUNT(e.student_id) AS student_count, MAX(e.score) AS highest_score
2 FROM courses c
3 JOIN course_classes cc ON c.course_id = cc.course_id
4 JOIN enrollments e ON cc.course_class_id = e.course_class_id
5 GROUP BY c.course_name
6 ORDER BY c.course_name;
```

The query results are displayed in the Data Output tab, showing a table with 7 rows and 3 columns: course_name, student_count, and highest_score.

	course_name character varying (50)	student_count bigint	highest_score numeric
1	操作系统	4	94.00
2	计算机网络	2	88.00
3	人工智能	3	95.00
4	软件工程	4	91.00
5	数据结构	2	82.00
6	数据库系统	4	91.00
7	算法分析	1	92.00

The status bar at the bottom indicates: Total rows: 7, Query complete 00:00:00.090, CRLF, Ln 1, Col 1.

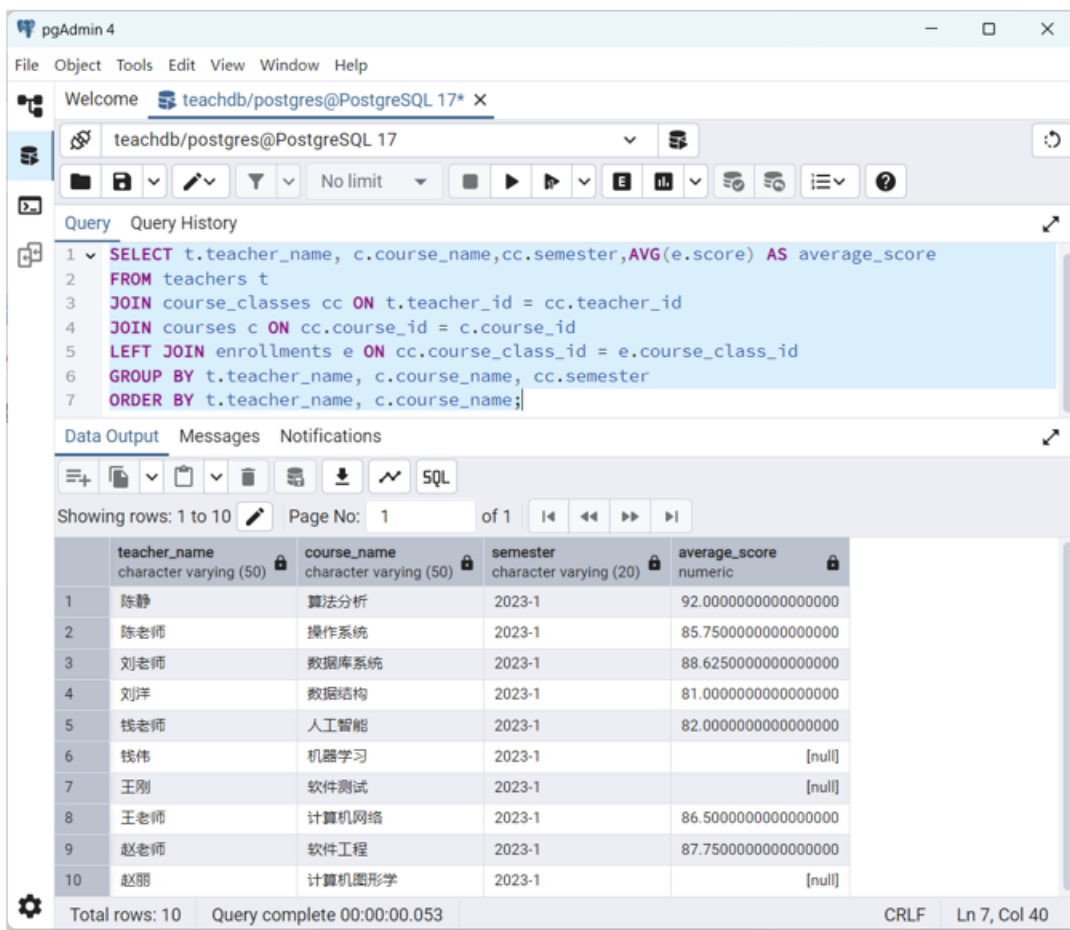
17、较复杂的统计查询示例：

(4) 查询每个教师教授的课程、授课学期及其学生的平均成绩。

```
SELECT t.teacher_name, c.course_name, cc.semester, AVG(e.score) AS average_score
FROM teachers t
JOIN course_classes cc ON t.teacher_id = cc.teacher_id
JOIN courses c ON cc.course_id = c.course_id
LEFT JOIN enrollments e ON cc.course_class_id = e.course_class_id
GROUP BY t.teacher_name, c.course_name, cc.semester
ORDER BY t.teacher_name, c.course_name;
```

17、较复杂的统计查询示例：

(4) 查询每个教师教授的课程、授课学期及其学生的平均成绩。（运行结果）



The screenshot shows the pgAdmin 4 interface. The SQL query is as follows:

```
1 SELECT t.teacher_name, c.course_name, cc.semester, AVG(e.score) AS average_score
2 FROM teachers t
3 JOIN course_classes cc ON t.teacher_id = cc.teacher_id
4 JOIN courses c ON cc.course_id = c.course_id
5 LEFT JOIN enrollments e ON cc.course_class_id = e.course_class_id
6 GROUP BY t.teacher_name, c.course_name, cc.semester
7 ORDER BY t.teacher_name, c.course_name;
```

The results are displayed in a table with the following columns: teacher_name, course_name, semester, and average_score. The table shows 10 rows of data.

	teacher_name character varying (50)	course_name character varying (50)	semester character varying (20)	average_score numeric
1	陈静	算法分析	2023-1	92.0000000000000000
2	陈老师	操作系统	2023-1	85.7500000000000000
3	刘老师	数据库系统	2023-1	88.6250000000000000
4	刘洋	数据结构	2023-1	81.0000000000000000
5	钱老师	人工智能	2023-1	82.0000000000000000
6	钱伟	机器学习	2023-1	[null]
7	王刚	软件测试	2023-1	[null]
8	王老师	计算机网络	2023-1	86.5000000000000000
9	赵老师	软件工程	2023-1	87.7500000000000000
10	赵丽	计算机图形学	2023-1	[null]

At the bottom of the interface, it indicates: Total rows: 10, Query complete 00:00:00.053, CRLF, Ln 7, Col 40.

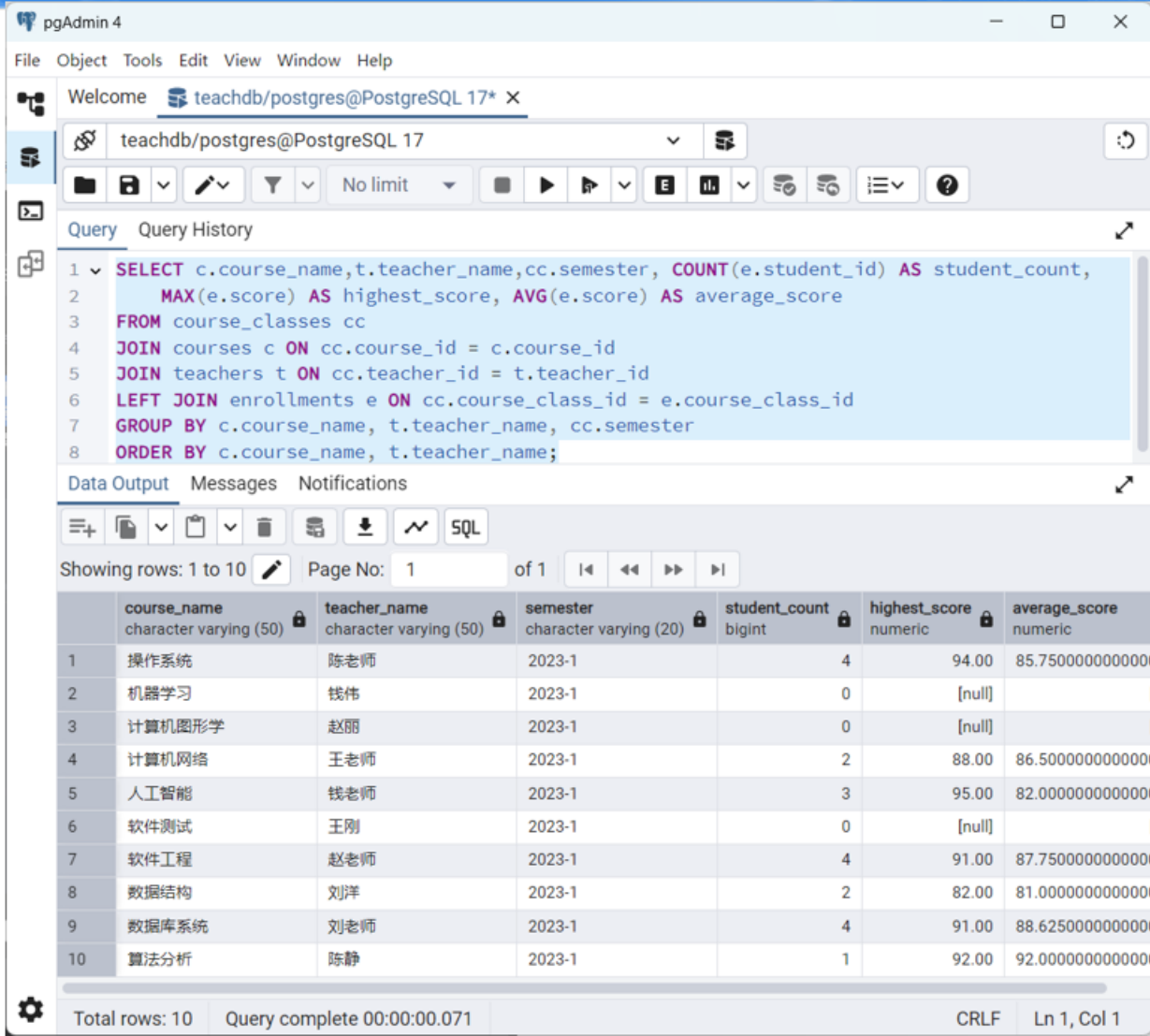
17、较复杂的统计查询示例：

(4) 查询每个课程班级的课程、教师姓名、授课学期、学生人数、班级最高分及其学生的平均成绩。

```
SELECT c.course_name,t.teacher_name,cc.semester,  
       COUNT(e.student_id) AS student_count,  
       MAX(e.score) AS highest_score,  
       AVG(e.score) AS average_score  
FROM course_classes cc  
JOIN courses c ON cc.course_id = c.course_id  
JOIN teachers t ON cc.teacher_id = t.teacher_id  
LEFT JOIN enrollments e ON cc.course_class_id = e.course_class_id  
GROUP BY c.course_name, t.teacher_name, cc.semester  
ORDER BY c.course_name, t.teacher_name;
```

17、较复杂的统计查询示例:

(4)、查询每个课程班级的课程、教师姓名、授课学期、学生人数、班级最高分及其学生的平均成绩。(运行结果)



The screenshot shows the pgAdmin 4 interface. The SQL query is as follows:

```
1 SELECT c.course_name,t.teacher_name,cc.semester, COUNT(e.student_id) AS student_count,
2       MAX(e.score) AS highest_score, AVG(e.score) AS average_score
3 FROM course_classes cc
4 JOIN courses c ON cc.course_id = c.course_id
5 JOIN teachers t ON cc.teacher_id = t.teacher_id
6 LEFT JOIN enrollments e ON cc.course_class_id = e.course_class_id
7 GROUP BY c.course_name, t.teacher_name, cc.semester
8 ORDER BY c.course_name, t.teacher_name;
```

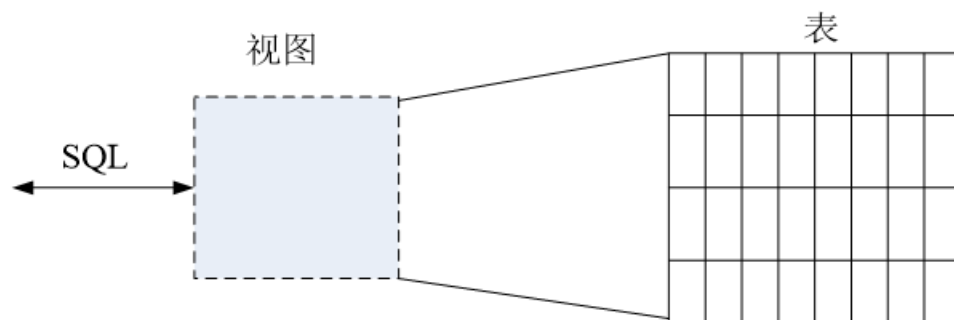
The results are displayed in a table with the following columns: course_name, teacher_name, semester, student_count, highest_score, and average_score. The table shows 10 rows of data.

	course_name character varying (50)	teacher_name character varying (50)	semester character varying (20)	student_count bigint	highest_score numeric	average_score numeric
1	操作系统	陈老师	2023-1	4	94.00	85.75000000000000
2	机器学习	钱伟	2023-1	0	[null]	
3	计算机图形学	赵丽	2023-1	0	[null]	
4	计算机网络	王老师	2023-1	2	88.00	86.50000000000000
5	人工智能	钱老师	2023-1	3	95.00	82.00000000000000
6	软件测试	王刚	2023-1	0	[null]	
7	软件工程	赵老师	2023-1	4	91.00	87.75000000000000
8	数据结构	刘洋	2023-1	2	82.00	81.00000000000000
9	数据库系统	刘老师	2023-1	4	91.00	88.62500000000000
10	算法分析	陈静	2023-1	1	92.00	92.00000000000000

At the bottom, the status bar indicates: Total rows: 10, Query complete 00:00:00.071, CRLF, Ln 1, Col 1.

十一、数据库视图

视图是一个**虚拟表**，它是基于 SQL 查询结果的命名查询。**视图并不存储数据，而是存储查询的 SQL 语句。**当你查询视图时，PostgreSQL 会动态执行视图定义中的 SQL 语句，并返回结果。**视图可以简化复杂的查询，提供数据的安全性和抽象层。**



(1) 视图的优点

- ① **简化复杂查询**：通过将复杂的 SQL 查询封装在视图中，用户可以更简单地访问数据。
- ② **数据安全性**：可以限制用户对基础表的访问，只允许他们访问视图中的数据。
- ③ **逻辑数据独立性**：如果基础表的结构发生变化，只需更新视图定义，而不需要修改所有使用该表的查询。

(2) 创建视图的 SQL 语法

```
CREATE VIEW view_name AS  
SELECT column1, column2, ...  
FROM table_name  
WHERE condition;
```

① **view_name**: 视图的名称。

② **SELECT**: 定义视图的查询语句，可以是任何有效的 **SELECT** 查询。

(3) 创建视图示例

创建一个视图，显示每个学生的姓名、课程名称、教师姓名和成绩。

```
CREATE VIEW student_course_info AS
```

```
SELECT s.student_name,c.course_name,t.teacher_name,e.score
```

```
FROM students s
```

```
JOIN enrollments e ON s.student_id = e.student_id
```

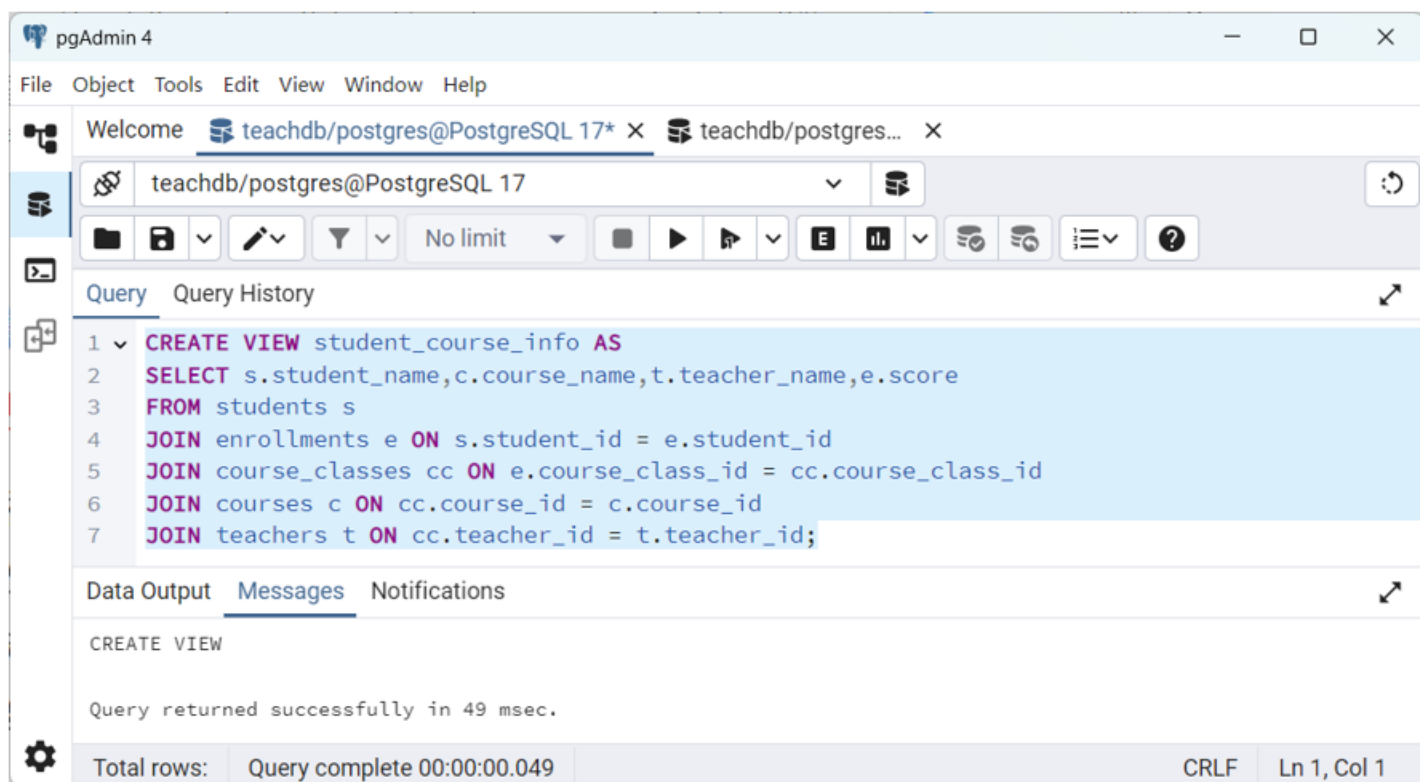
```
JOIN course_classes cc ON e.course_class_id = cc.course_class_id
```

```
JOIN courses c ON cc.course_id = c.course_id
```

```
JOIN teachers t ON cc.teacher_id = t.teacher_id;
```

(3) 创建视图示例

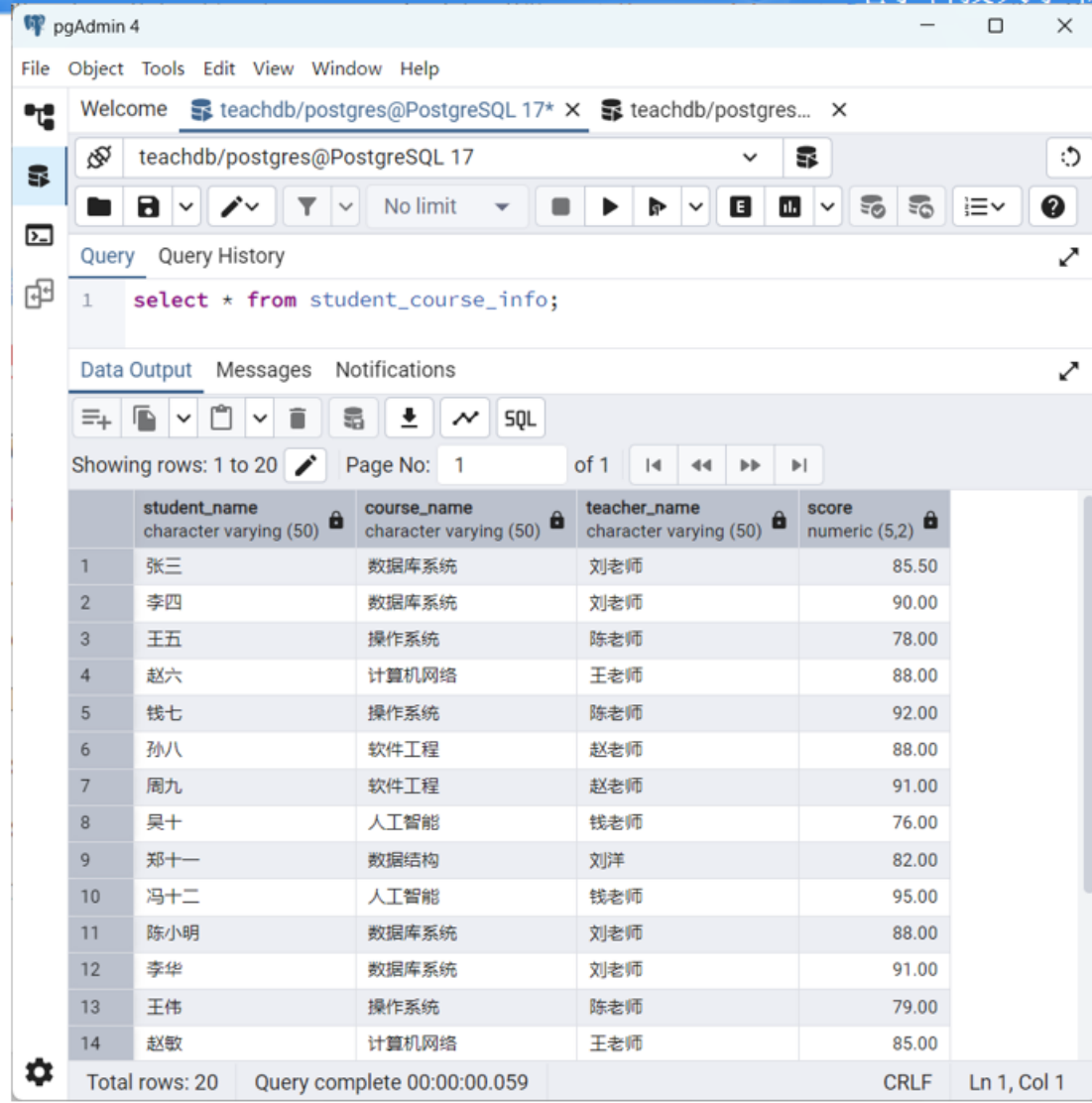
创建一个视图，显示每个学生的姓名、课程名称、教师姓名和成绩。（执行结果）



(3) 创建视图示例

在该视图上执行查询：

```
select * from student_course_info;
```



The screenshot shows the pgAdmin 4 web interface. The query editor at the top contains the SQL statement: `select * from student_course_info;`. Below the editor, the 'Data Output' tab is active, displaying a table with 14 rows of data. The table has four columns: `student_name`, `course_name`, `teacher_name`, and `score`. The data is as follows:

	student_name character varying (50)	course_name character varying (50)	teacher_name character varying (50)	score numeric (5,2)
1	张三	数据库系统	刘老师	85.50
2	李四	数据库系统	刘老师	90.00
3	王五	操作系统	陈老师	78.00
4	赵六	计算机网络	王老师	88.00
5	钱七	操作系统	陈老师	92.00
6	孙八	软件工程	赵老师	88.00
7	周九	软件工程	赵老师	91.00
8	吴十	人工智能	钱老师	76.00
9	郑十一	数据结构	刘洋	82.00
10	冯十二	人工智能	钱老师	95.00
11	陈小明	数据库系统	刘老师	88.00
12	李华	数据库系统	刘老师	91.00
13	王伟	操作系统	陈老师	79.00
14	赵敏	计算机网络	王老师	85.00

At the bottom of the interface, a status bar indicates 'Total rows: 20', 'Query complete 00:00:00.059', and 'Ln 1, Col 1'.

(4) 删除视图

如果需要删除视图，可以使用以下 SQL 语句：

```
DROP VIEW student_course_info;
```

十二、PostgreSQL 数据库的物化视图

物化视图（Materialized View）是数据库中的一种特殊视图，它在创建时会
将查询结果存储在数据库中，而不是每次查询时动态计算。物化视图可以提
高查询性能，尤其是在处理复杂查询或大数据集时，因为它们可以避免重复
计算。

(1) 物化视图的特点

- ① **存储数据**：物化视图将查询结果存储在数据库中，因此可以快速访问。
- ② **定期刷新**：物化视图的数据不会自动更新，需要手动或定期刷新，以确保数据的准确性。
- ③ **提高性能**：对于复杂的查询，物化视图可以显著提高性能，因为查询结果已经预先计算并存储。

(2) 创建物化视图的 SQL 语法

```
CREATE MATERIALIZED VIEW view_name AS  
SELECT column1, column2, ...  
FROM table_name  
WHERE condition;
```

- ① view_name: 物化视图的名称。
- ② SELECT: 定义物化视图的查询语句, 可以是任何有效的 SELECT 查询。

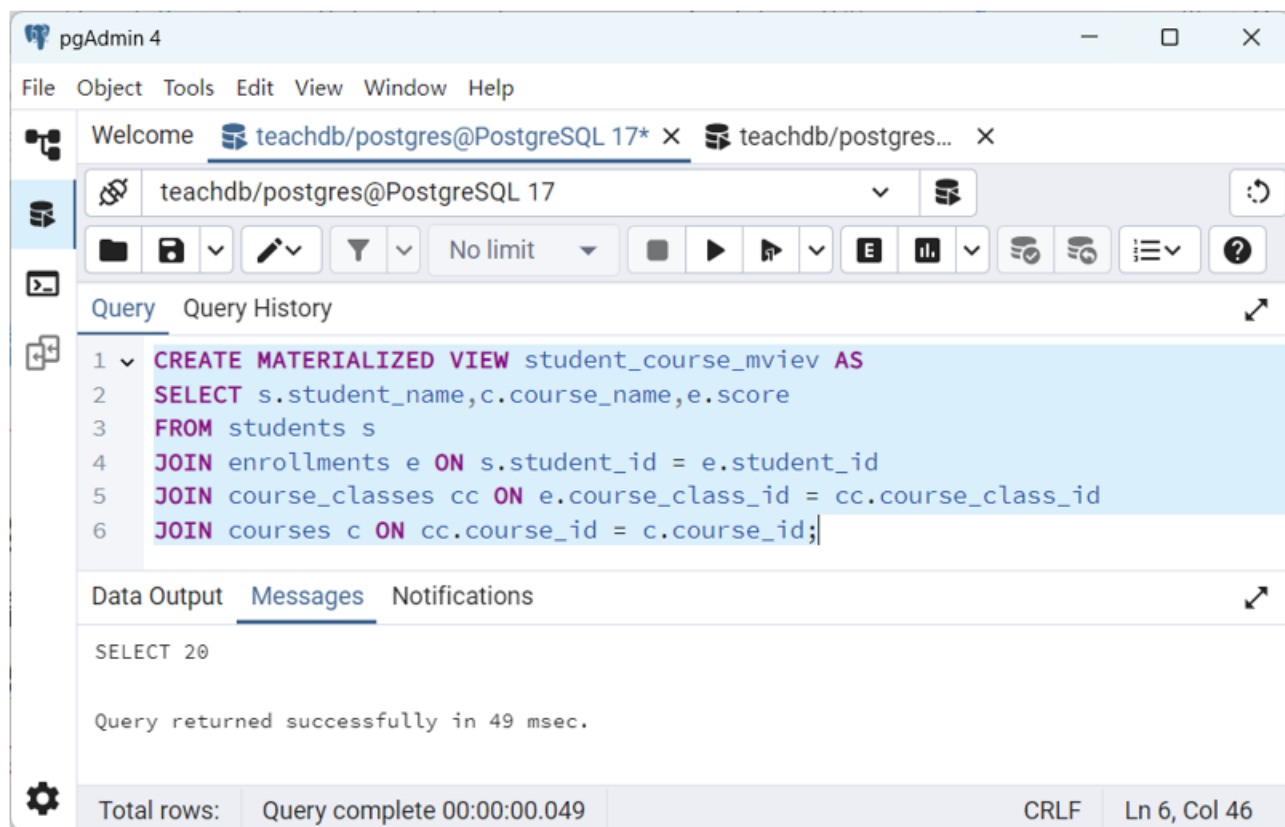
(3) 创建物化视图的示例

创建一个物化视图，显示每个学生的姓名、课程名称和成绩。

```
CREATE MATERIALIZED VIEW student_course_mvview AS  
SELECT s.student_name,c.course_name,e.score  
FROM students s  
JOIN enrollments e ON s.student_id = e.student_id  
JOIN course_classes cc ON e.course_class_id = cc.course_class_id  
JOIN courses c ON cc.course_id = c.course_id;
```

(3) 创建物化视图的示例

创建一个物化视图，
显示每个学生的姓名
、课程名称和成绩。
(执行创建)



(4) 执行物化视图查询

select *

from student_course_mview ;

The screenshot shows the pgAdmin 4 web interface. The top navigation bar includes 'File', 'Object', 'Tools', 'Edit', 'View', 'Window', and 'Help'. The main window has a tab for 'teachdb/postgres@PostgreSQL 17*'. The left sidebar shows a tree view with 'teachdb/postgres@PostgreSQL 17' selected. The central pane displays a SQL query: `select * from student_course_mview ;`. Below the query editor, the 'Data Output' tab is active, showing a table with 11 rows. The table has three columns: 'student_name' (character varying (50)), 'course_name' (character varying (50)), and 'score' (numeric (5,2)). The status bar at the bottom indicates 'Total rows: 20', 'Query complete 00:00:00.045', and 'Ln 1, Col 1'.

	student_name character varying (50)	course_name character varying (50)	score numeric (5,2)
1	张三	数据库系统	85.50
2	李四	数据库系统	90.00
3	王五	操作系统	78.00
4	赵六	计算机网络	88.00
5	钱七	操作系统	92.00
6	孙八	软件工程	88.00
7	周九	软件工程	91.00
8	吴十	人工智能	76.00
9	郑十一	数据结构	82.00
10	冯十二	人工智能	95.00
11	陈小明	数据库系统	88.00

(5) 刷新物化视图

由于物化视图的数据是静态的，因此需要定期刷新以确保数据的准确性。可以使用以下 SQL 语句刷新物化视图：

```
REFRESH MATERIALIZED VIEW student_course_info;
```

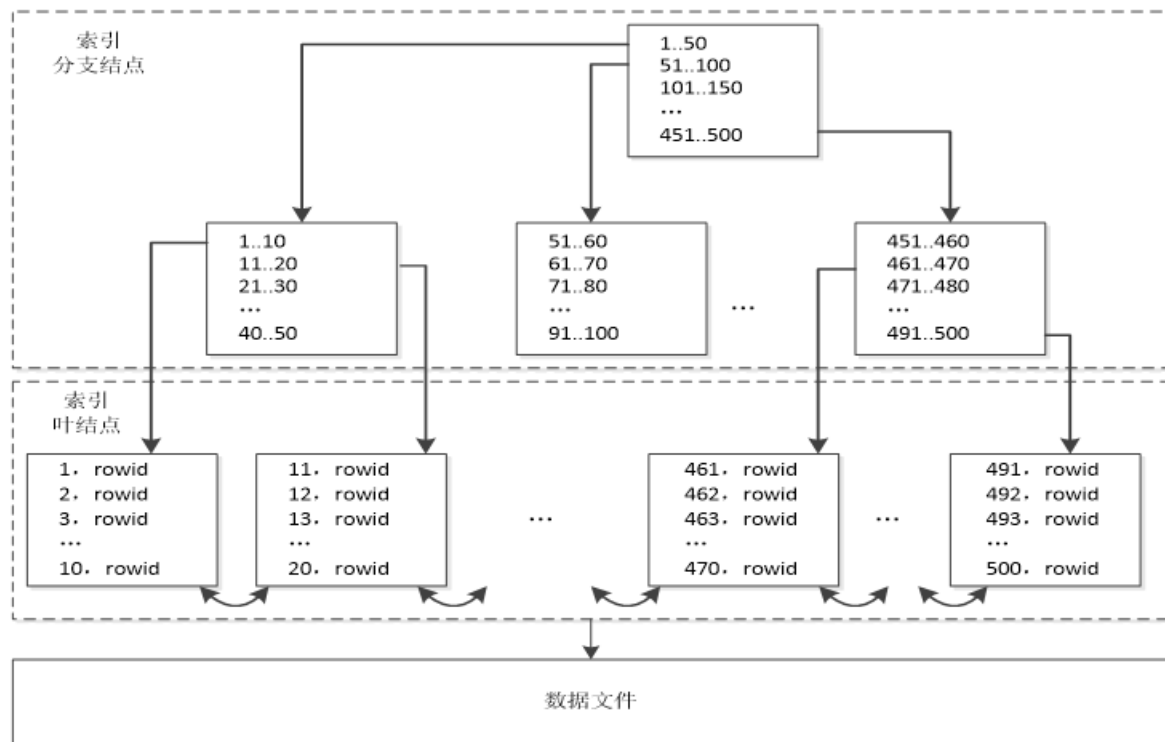
(5) 删除物化视图

如果需要删除物化视图，可以使用以下 SQL 语句：

```
DROP MATERIALIZED VIEW student_course_mview;
```

十三、数据库的索引

索引是数据库中用于提高查询性能的一种数据结构。它类似于书籍的目录，可以加速数据检索。通过创建索引，数据库可以更快地找到所需的数据，而不必扫描整个表。



(1) 数据库索引的优点

- ① **提高查询性能**：索引可以显著加快 `SELECT` 查询的速度，尤其是在处理大数据集时。
- ② **加速排序和过滤**：索引可以加速 `ORDER BY` 和 `WHERE` 子句中的条件判断。
- ③ **支持唯一性约束**：索引可以确保某些列的唯一性，例如主键和唯一约束。

(2) 数据库索引的缺点

- ① **占用存储空间**: 索引会占用额外的存储空间, 尤其是在大型表上。
- ② **影响写入性能**: 每当对表进行 INSERT、UPDATE 或 DELETE 操作时, 相关的索引也需要更新, 这可能会导致性能下降。
- ③ **维护成本**: 索引需要定期维护和优化, 以确保其性能。

(3) PostgreSQL 中的索引类型

- ① **B-tree 索引**：默认的索引类型，适用于大多数查询。它支持等值查询和范围查询。
- ② **Hash 索引**：适用于等值查询，但不支持范围查询。通常不推荐使用，因为 B-tree 索引在大多数情况下表现更好。
- ③ **GIN (Generalized Inverted Index) 索引**：适用于数组、JSONB 和全文搜索等数据类型，支持快速查找。
- ④ **GiST (Generalized Search Tree) 索引**：适用于地理数据和其他复杂数据类型，支持范围查询。
- ⑤ **BRIN (Block Range INdex) 索引**：适用于大数据集，尤其是数据按顺序存储的情况，能够有效地减少存储空间。

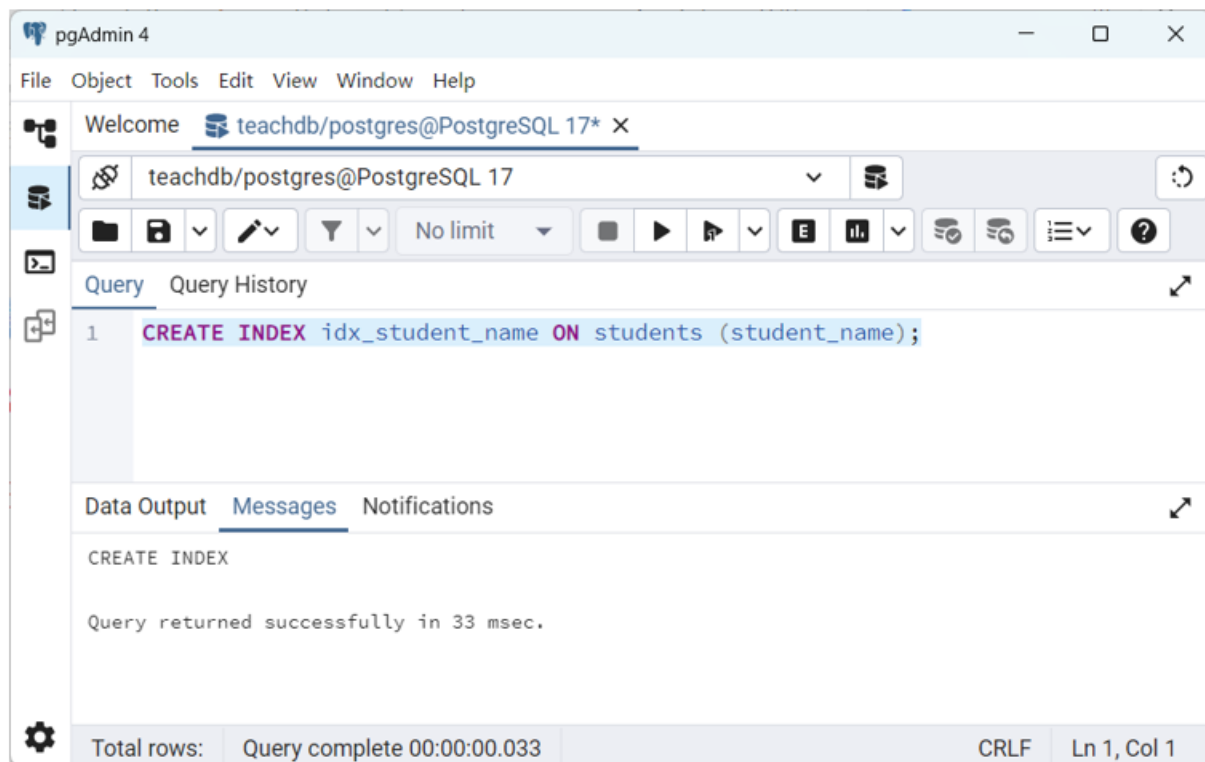
(4) 创建索引的 SQL 语法

CREATE INDEX *index_name* **ON** *table_name* (*column1*, *column2*, ...);

- ① *index_name*: 索引的名称。
- ② *table_name*: 要创建索引的表的名称。
- ③ *column1*, *column2*, ...: 要索引的列。

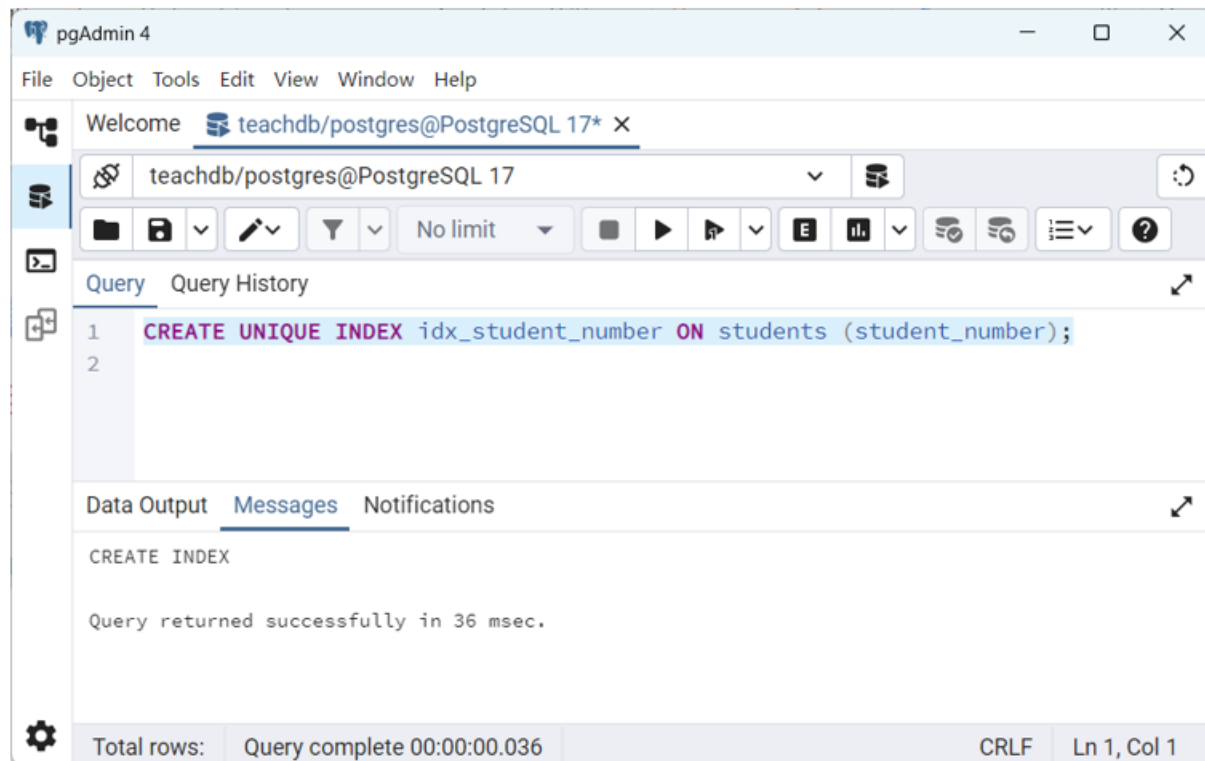
(5) 创建 B-tree 索引

CREATE INDEX **idx_student_name** **ON** **students** (**student_name**);



(6) 创建唯一索引

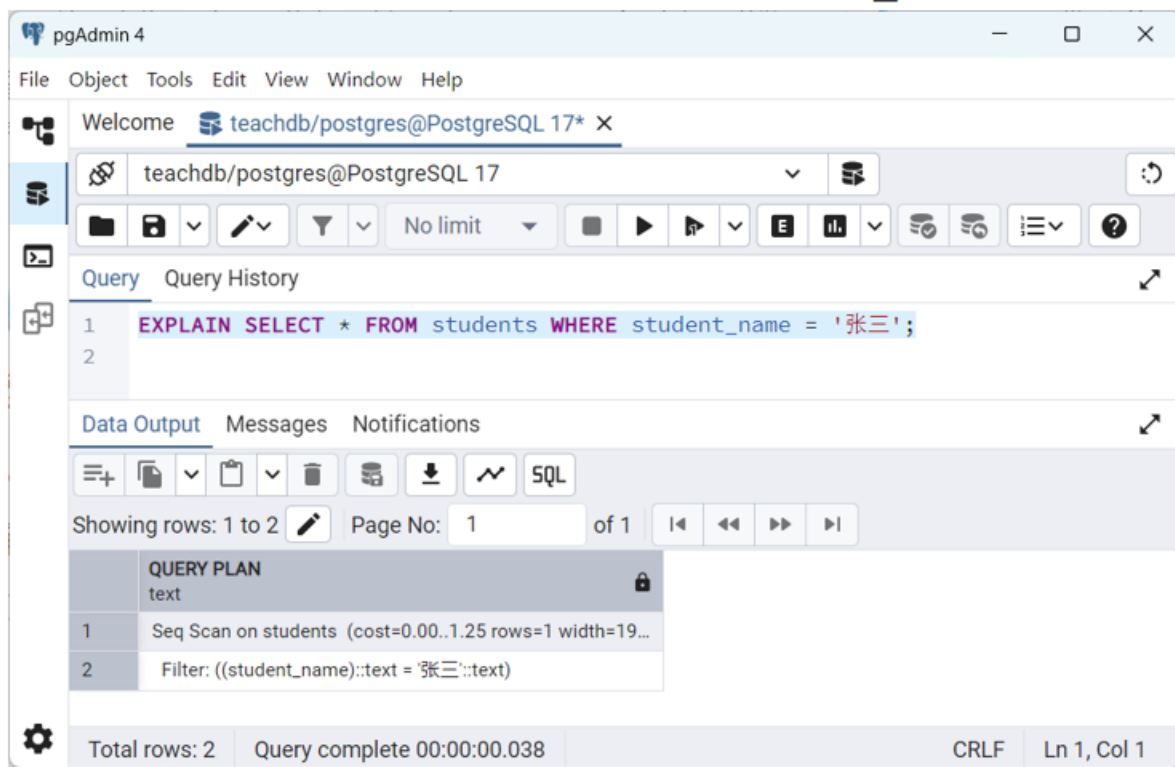
CREATE UNIQUE INDEX idx_student_number **ON** students (student_number);



(7) 查询使用索引

创建索引后，PostgreSQL 会自动使用索引来优化查询。使用 EXPLAIN 语句查看查询计划，以确认索引是否被使用：

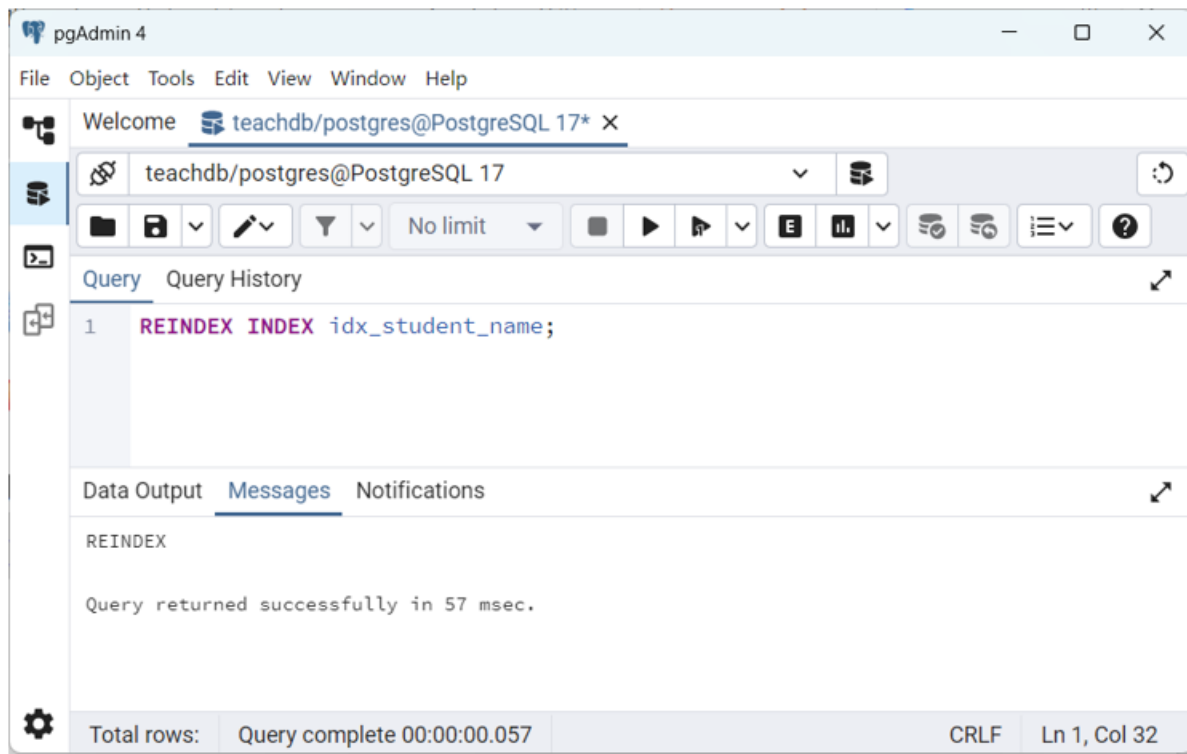
EXPLAIN SELECT * FROM students WHERE student_name = '张三';



(8) 索引的维护

随着数据的变化，索引可能会变得不够高效，使用 **REINDEX** 命令重建索引：

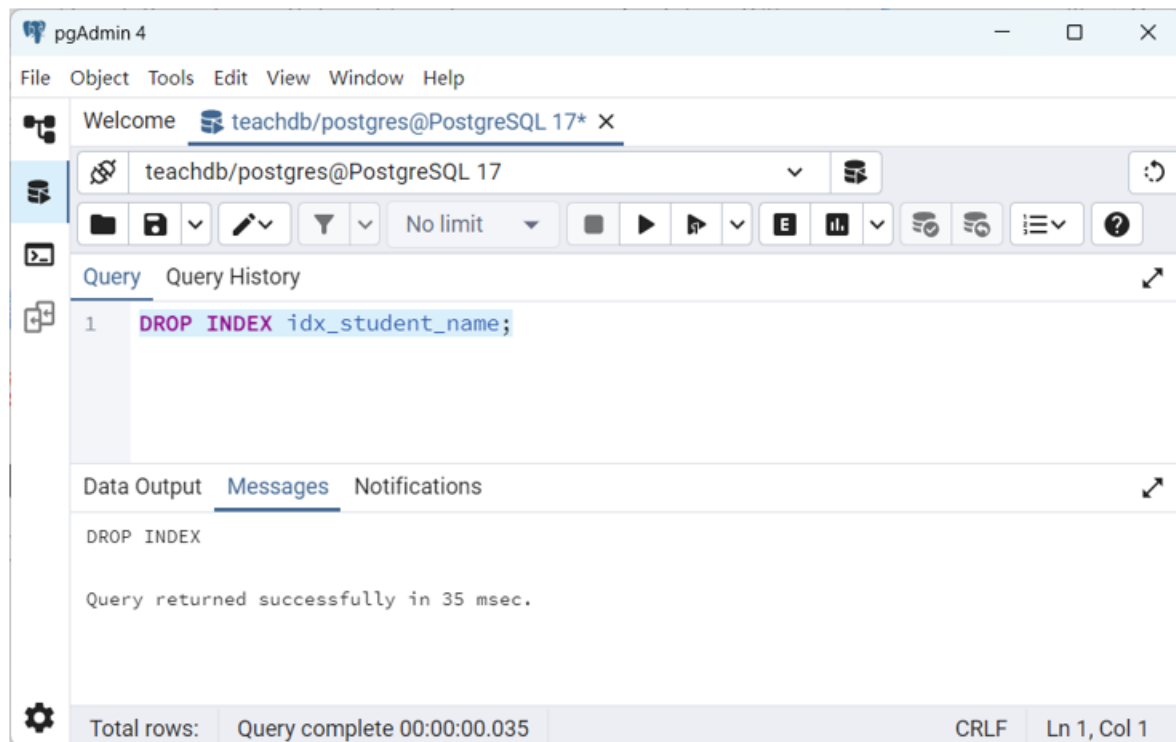
REINDEX INDEX idx_student_name;



(9) 删除索引

如果需要删除索引，可以使用以下 SQL 语句：

DROP INDEX idx_student_name;



十四、删除表的数据

在 PostgreSQL 数据库中，删除表中数据的主要语句是 **DELETE**。该语句用于从表中删除一行或多行数据。除了 **DELETE** 语句外，还有 **TRUNCATE** 语句用于快速删除表中的所有数据。

(1) 使用 DELETE 语句

DELETE 语句用于根据指定条件删除表中的一行或多行数据。

其基本语法如下：

DELETE FROM table_name

WHERE condition;

- ① table_name：要删除数据的表名。
- ② condition：用于指定要删除的行的条件。如果没有指定条件，则会删除表中的所有行。

删除特定行

如果我们想删除学生姓名为“李四”的记录，可以使用以下 SQL 语句：

DELETE FROM students

WHERE student_name = '李四';

删除多行

如果想删除学生表所有 class_id 为 1 的学生记录，可以使用以下 SQL 语句：

```
DELETE FROM students  
WHERE class_id = 1;
```

(2) 使用 TRUNCATE 语句

TRUNCATE 语句用于快速删除表中的所有数据，而不记录每一行的删除操作。

其基本语法如下：

TRUNCATE TABLE table_name;

table_name: 要清空数据的表名。

TRUNCATE 的特点

- ① **快速**：TRUNCATE 通常比 DELETE 更快，因为它不逐行删除数据，而是直接释放数据页。
- ② **不可回滚**：在大多数情况下，TRUNCATE 是不可回滚的，意味着一旦执行，数据将无法恢复。
- ③ **不触发触发器**：TRUNCATE 不会触发 DELETE 触发器。

使用 TRUNCATE 语句

如果我们想要清空 students 表中的所有数据，可以使用以下 SQL 语句：

```
TRUNCATE TABLE students;
```

执行后，students 表将变为空表，表结构仍然存在，但没有任何数据。

十五、数据修改（更新）

在 PostgreSQL 数据库中，数据更新操作主要通过 **UPDATE** 语句实现。**UPDATE** 语句用于修改表中现有记录的值。以下是对 PostgreSQL 数据更新操作的详细讲解，包括语法、示例和注意事项。

(1) UPDATE 语句的基本语法

UPDATE table_name

SET column1 = value1, column2 = value2, ...

WHERE condition;

- ① table_name: 要更新数据的表名。
- ② SET: 指定要更新的列及其新值。
- ③ WHERE: 可选部分, 用于指定要更新的行。如果没有 WHERE 子句, 表中的所有行都会被更新。

(2) 更新单个字段

如果要将学生表中姓名为“张三”的学号 `class_id` 更新为2, 可使用以下 SQL 语句:

```
UPDATE students
```

```
SET class_id = 2
```

```
WHERE student_name = '张三';
```

(3) 更新多个字段

如果想要同时更新多个字段，例如将学生姓名为“李四”的学生的 `class_id` 和 `student_number` 更新，可以使用以下 SQL 语句：

```
UPDATE students
```

```
SET class_id = 2, student_number = '20230004'
```

```
WHERE student_name = '李四';
```

(5) 注意事项

- ① **WHERE 子句的重要性**：在使用 UPDATE 语句时，**务必使用 WHERE 子句来限制更新的行**。如果省略 WHERE 子句，所有行都会被更新，这可能导致数据丢失或错误。
- ② **事务处理**：在执行更新操作时，**建议使用事务（BEGIN 和 COMMIT）来确保数据的一致性和完整性**。如果更新操作失败，可以使用 ROLLBACK 撤销更改。

使用事务处理执行更新的示例

BEGIN;

UPDATE students

SET class_id = 3

WHERE student_name = '王五';

COMMIT; -- 或者使用 **ROLLBACK;** 来撤销更改

本章结束！